

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

-----***-----



ĐỒ ÁN TỐT NGHIỆP ĐẠI HỌC

ĐỀ TÀI:

**PHÁT TRIỂN ỨNG DỤNG HỌC TIẾNG ANH TÍCH HỢP AI
CHO PHÂN TÍCH NGỮ PHÁP, GỢI Ý TỪ VỰNG VÀ
CHATBOT LUYỆN PHẢN XẠ**

Giảng viên hướng dẫn : TS. Đỗ Thị Liên

Họ và tên sinh viên : Nguyễn Minh Hiếu - B21DCCN365

: Nguyễn Huy Tú - B21DCCN750

: Đỗ Hoàng Trọng - B21DCCN725

Hà Nội – 2025

LỜI CẢM ƠN

Đầu tiên, chúng em xin gửi lời cảm ơn sâu sắc đến trường Học Viện Công Nghệ Bưu Chính Viễn Thông, các thầy, các cô trong học viện nói chung và trong khoa Công Nghệ Thông Tin nói riêng đã luôn nhiệt huyết, tận tình trong từng bài giảng, đã truyền đạt cho chúng em những kiến thức bổ ích trong thời gian học tập tại trường. Đây sẽ là những kiến thức quý báu, là hành trang tiếp bước cho chúng em sau này.

Đặc biệt, chúng em xin gửi lời cảm ơn chân thành đến cô TS. Đỗ Thị Liên đã tận tình chỉ bảo, hướng dẫn, truyền đạt kiến thức, kinh nghiệm để giúp em thực hiện đề tài đồ án này.

Vì thời gian có hạn, trình độ hiểu biết của bản thân còn hạn chế, cho nên trong đồ án sẽ có những thiếu sót khi tìm hiểu, đánh giá, trình bày về đề tài. chúng em rất mong được các thầy cô giáo đóng góp ý kiến để đồ án của chúng em được đầy đủ và hoàn thiện hơn.

Chúng em xin chân thành cảm ơn!

MỤC LỤC

LỜI CẢM ƠN	i
MỤC LỤC	ii
DANH MỤC HÌNH ẢNH	v
DANH MỤC CÁC BẢNG	vii
DANH MỤC CÁC TỪ VIẾT TẮT	ix
MỞ ĐẦU	1
CHƯƠNG 1. TỔNG QUAN VỀ ỨNG DỤNG HỌC TIẾNG ANH TÍCH HỢP AI CHO PHÂN TÍCH NGỮ PHÁP, GỢI Ý TỪ VỰNG VÀ CHATBOT LUYỆN PHẢN XẠ.....	2
1.1. Khảo sát các hệ thống học tiếng Anh hiện nay	2
1.1.1. Duolingo.....	2
1.1.2. Study4	3
1.1.3. Grammarly	4
1.2. Các hướng tiếp cận xây dựng ứng dụng học tiếng Anh tích hợp AI	5
1.2.1. Hướng tiếp cận cho phân tích ngữ pháp chuyên sâu.....	5
1.2.2. Hướng tiếp cận cho gợi ý từ vựng cá nhân hóa	6
1.2.3. Hướng tiếp cận cho Chatbot luyện phản xạ và đối thoại	6
1.3. Các công nghệ phân tích ngữ pháp, gợi ý từ vựng và chatbot	7
1.4. Đề xuất tích hợp AI cho phân tích ngữ pháp, gợi ý từ vựng và chatbot luyện phản xạ cho ứng dụng học tiếng anh	7
1.4.1. Quy trình xử lý (pipeline)	8
1.4.2. Chi tiết công nghệ và thuật toán cho từng chức năng	8
1.5. Kết luận chương 1.....	11
CHƯƠNG 2. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG.....	13
2.1. Mô hình tổng quan hệ thống.....	13
2.1.1 Giới thiệu hệ thống.....	13
2.1.2. Mô hình tổng quát hệ thống	13
2.1.3. Các tác nhân của hệ thống.....	14
2.1.4. Các thành phần chính của hệ thống	14
2.2. Phương pháp tiếp cận và xây dựng phần mềm.....	15

2.2.1. Phương pháp xây dựng phần mềm.....	15
2.2.2. Thiết kế kiến trúc tổng thể	17
2.2.3. Kiến trúc phần mềm	17
2.2.4. Công nghệ xây dựng hệ thống	20
2.3. Phân tích hệ thống	26
2.3.1. Mục tiêu của hệ thống	26
2.3.2. Đối tượng sử dụng hệ thống.....	27
2.3.3. Xác định và miêu tả các usecase sử dụng	27
2.3.4. Biểu đồ usecase tổng quan	29
2.3.5. Biểu đồ Use case và kịch bản một số chức năng chính	29
2.3.6. Biểu đồ lớp phân tích	42
2.3.7. Biểu đồ tuần tự	45
2.4. Thiết kế hệ thống	51
2.4.1. Thiết kế các module chức năng.....	51
2.4.2. Thiết kế biểu đồ lớp (Class Diagram)	51
2.5. Cơ sở dữ liệu.....	52
2.5.1. Lược đồ cơ sở dữ liệu.....	52
2.5.2. Xây dựng và mô tả các bảng trong cơ sở dữ liệu.....	52
2.5.3. Quan hệ giữa các bảng	59
2.6. Kết luận chương 2.....	59
CHƯƠNG 3. THỬ NGHIỆM VÀ ĐÁNH GIÁ ỨNG DỤNG HỌC TIẾNG ANH	
TÍCH HỢP AI CHO PHÂN TÍCH NGỮ PHÁP, GỢI Ý TỪ VỰNG VÀ	
CHATBOT LUYỆN PHẢN XẠ.....	61
3.1. Dữ liệu thực nghiệm	62
3.2. Cài đặt thực nghiệm	63
3.2.1. Độ đo	63
3.2.2. Phương pháp thực nghiệm	64
3.2.3. Các phương pháp được sử dụng để so sánh	64
3.3. Kết quả thực nghiệm	66
3.4. Thử nghiệm ứng dụng học tiếng Anh tích hợp AI cho phân tích ngữ pháp,	
gợi ý từ vựng và chatbot luyện phản xạ.....	70

3.4.1. Module kiểm tra ngữ pháp và chat cơ bản	70
3.4.2. Module gợi ý từ vựng.....	71
3.5. Kết luận chương 3.....	73
Phụ lục cài đặt và triển khai.....	75
Kết luận và hướng phát triển	86
Tài liệu tham khảo	88

DANH MỤC HÌNH ẢNH

Hình 1.1. Duolingo.....	2
Hình 1.2. Study4	3
Hình 1.3. Grammarly	4
Hình 1.4. Kiến trúc tích hợp AI cho ứng dụng học tiếng anh.....	7
Hình 1.5. Tổng quan kiến trúc LightRAG	9
Hình 2.1. Mô hình tổng quát hệ thống	13
Hình 2.2. Kiến trúc Client - Server	17
Hình 2.3. Kiến trúc phân tầng kết hợp tư duy MVC.....	19
Hình 2.4. Usecase tổng quan.....	29
Hình 2.5. Biểu đồ usecase học và luyện tập bài học	29
Hình 2.6. Biểu đồ sửa thông tin cá nhân	30
Hình 2.7. Biểu đồ xem danh sách bài học.....	31
Hình 2.8. Biểu đồ bình luận và đánh giá bài học	33
Hình 2.9. Biểu đồ luyện nói tiếng anh với AI.....	34
Hình 2.10. Biểu đồ hỗ trợ viết tiếng Anh với AI	35
Hình 2.11. Biểu đồ học tiếng anh bằng flashcard	36
Hình 2.12. Biểu đồ hỗ trợ từ vựng với AI.....	37
Hình 2.13. Biểu đồ xem thống kê	38
Hình 2.14. Biểu đồ sửa thông tin người dùng.....	39
Hình 2.15. Biểu đồ sửa khóa học	40
Hình 2.16. Biểu đồ xem thống kê của hệ thống.....	41
Hình 2.17. Biểu đồ lớp phân tích	42
Hình 2.18. Biểu đồ tuần tự chức năng người dùng đăng ký hệ thống	45
Hình 2.19. Biểu đồ tuần tự chức năng người dùng đăng nhập hệ thống	46
Hình 2.20. Biểu đồ tuần tự của sửa thông tin cá nhân	46
Hình 2.21. Biểu đồ tuần tự của xem danh sách bài học	47
Hình 2.22. Biểu đồ tuần tự của học và luyện tập bài học	47
Hình 2.23. Biểu đồ tuần tự của bình luận và đánh giá bài học	48
Hình 2.24. Biểu đồ tuần tự của học tiếng anh bằng flashcard	48
Hình 2.25. Biểu đồ tuần tự của hỗ trợ từ vựng với AI.....	49

Hình 2.26. Biểu đồ tuần tự của xem thống kê.....	49
Hình 2.27. Biểu đồ tuần tự của quản lý sửa thông tin người dùng	50
Hình 2.28. Biểu đồ tuần tự của quản lý sửa flashcard	50
Hình 2.29. Biểu đồ lớp	51
Hình 2.30. Lược đồ cơ sở dữ liệu	52
Hình 3.1. Hình ảnh 1 trang trong sách GrammarInUse Intermediate	62
Hình 3.2. Bảng so sánh hiệu suất LightRAG và các phương pháp baseline theo paper	68
Hình 3.3. RAG	70
Hình 3.4. LLM	70
Hình 3.5. Mối quan hệ các từ sau khi đưa vào graph.....	71
Hình 3.6. Naive RAG.....	72
Hình 3.7. Graph-Based Text Indexing / LightRAG.....	72

DANH MỤC CÁC BẢNG

Bảng 2.1. Danh sách usecase cho Người dùng	28
Bảng 2.2. Danh sách usecase cho Quản trị viên	28
Bảng 2.3. Kịch bản chức năng học và luyện tập bài học	30
Bảng 2.4. Kịch bản chức năng sửa thông tin cá nhân	31
Bảng 2.5. Kịch bản chức năng xem danh sách bài học	32
Bảng 2.6. Kịch bản chức năng bình luận và đánh giá bài học	33
Bảng 2.7. Kịch bản chức năng luyện nói tiếng anh với AI	34
Bảng 2.8. Kịch bản chức năng hỗ trợ viết tiếng anh với AI	35
Bảng 2.9. Kịch bản chức năng học tiếng anh bằng flashcard	36
Bảng 2.10. Kịch bản chức năng hỗ trợ từ vựng với AI	37
Bảng 2.11. Kịch bản chức năng xem thống kê	38
Bảng 2.12. Kịch bản chức năng quản lý sửa thông tin người dùng	39
Bảng 2.13. Kịch bản chức năng quản lý sửa khóa học	40
Bảng 2.14. Kịch bản chức năng quản lý xem thống kê hệ thống	41
Bảng 2.15. Bảng trích xuất danh từ	43
Bảng 2.16. Bảng Role	52
Bảng 2.17. Bảng User	53
Bảng 2.18. Bảng Language	53
Bảng 2.19. Bảng Course	53
Bảng 2.20. Bảng Unit	54
Bảng 2.21. Bảng Lesson	54
Bảng 2.22. Bảng Exercise	54
Bảng 2.23. Bảng Dialogue	55
Bảng 2.24. Bảng Video	55
Bảng 2.25. Bảng Vocabulary	55
Bảng 2.26. Bảng Grammar	56
Bảng 2.27. Bảng GrammarFormula	56
Bảng 2.28. Bảng GrammarExample	56
Bảng 2.29. Bảng Comment	57
Bảng 2.30. Bảng Notification	57

Bảng 2.31. Bảng Deck	58
Bảng 2.32. Bảng Flashcard	58
Bảng 2.33. Bảng User_Deck.....	58
Bảng 2.34. Bảng User_Lesson.....	58
Bảng 3.1. Bảng tóm tắt đánh giá thực nghiệm các module tích hợp AI	61
Bảng 3.2. Bảng đo thực nghiệm Hit rate và MRR.....	66
Bảng 3.3. Đo thực nghiệm Faithfulness, Answer Relevance, Context Precision.....	67
Bảng 3.4. Bảng đo thực nghiệm cho những câu hỏi về từ vựng.....	69

DANH MỤC CÁC TỪ VIẾT TẮT

LLM	Large Language Model
RAG	Retrieval Augmented Generation
NLP	Natural Language Processing
MRR	Mean Reciprocal Rank
GEC	Grammar Error Correction

MỞ ĐẦU

Trong kỷ nguyên toàn cầu hóa, tiếng Anh đã trở thành ngôn ngữ quốc tế và là chìa khóa mở ra cánh cửa tri thức, cơ hội nghề nghiệp và giao tiếp xuyên biên giới. Nhu cầu học và làm chủ tiếng Anh của người Việt Nam ngày càng tăng cao. Tuy nhiên, các phương pháp học truyền thống và các ứng dụng hiện có thường gặp phải những hạn chế như: thiếu tính tương tác cá nhân hóa, khả năng phân tích lỗi ngữ pháp và gợi ý từ vựng còn sơ sài, và môi trường luyện nói thiếu tự nhiên.

Sự phát triển vượt bậc của Trí tuệ Nhân tạo (AI) đã mở ra tiềm năng cách mạng hóa giáo dục ngôn ngữ. Việc tích hợp AI có thể cung cấp trải nghiệm học tập cá nhân hóa, chính xác và hiệu quả hơn nhiều so với trước đây.

Đề án “ Phát triển ứng dụng học tiếng anh tích hợp AI cho phân tích ngữ pháp, gợi ý từ vựng và chatbot luyện phản xạ” ra đời nhằm mục đích khắc phục những hạn chế trên và cung cấp một giải pháp học tập toàn diện. Đề án này sẽ trình bày về xây dựng một website cùng với việc giới thiệu và trình bày các Framework, thư viện và cơ sở lý thuyết liên quan. Phần cài đặt, thử nghiệm, đánh giá hệ thống sẽ dựa trên bộ dữ liệu sản phẩm của các website học tiếng anh.

CHƯƠNG 1. TỔNG QUAN VỀ ỨNG DỤNG HỌC TIẾNG ANH TÍCH HỢP AI CHO PHÂN TÍCH NGỮ PHÁP, GỢI Ý TỪ VỰNG VÀ CHATBOT LUYỆN PHẢN XẠ

Chương này tập trung vào việc nghiên cứu bối cảnh hiện tại của các ứng dụng học tiếng Anh trên thị trường, từ đó chỉ ra những ưu điểm và hạn chế của các nền tảng phổ biến như Duolingo, Study4 và Grammarly. Dựa trên các khoảng trống công nghệ hiện có, chương 1 sẽ đề xuất các hướng tiếp cận kỹ thuật và hệ thống các công nghệ AI then chốt (LLM, GEC) nhằm xây dựng một giải pháp học tập tích hợp, cá nhân hóa và có tính tương tác cao.

1.1. Khảo sát các hệ thống học tiếng Anh hiện nay

1.1.1. Duolingo



Hình 1.1. Duolingo

Duolingo là nền tảng học ngôn ngữ toàn cầu, nổi tiếng với phương pháp học tập được "game hóa" (gamification), thu hút người dùng thông qua các bài học ngắn và hệ thống điểm thưởng.

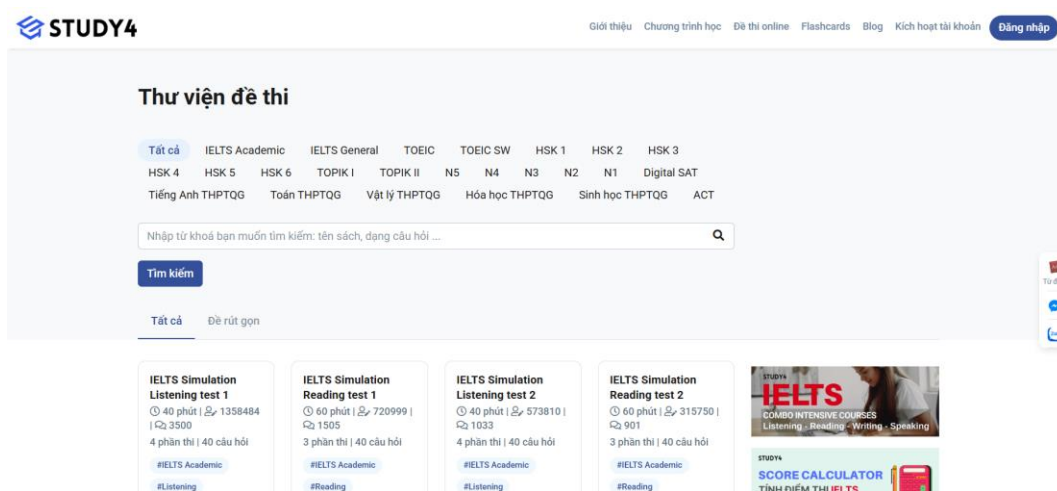
Phân tích Ngữ pháp, Duolingo chủ yếu tích hợp các thuật toán Machine Learning để đánh giá câu trả lời của người dùng trong các bài tập dịch thuật hoặc sắp

xếp từ. Hệ thống tập trung vào việc nhận diện và sửa lỗi sai trực tiếp (Explicit Error Correction) liên quan đến cấu trúc câu cơ bản và từ vựng, đồng thời củng cố kiến thức thông qua cơ chế lặp lại ngắt quãng (Spaced Repetition). Tuy nhiên, tính năng này thường giới hạn ở việc kiểm tra lỗi theo cấu trúc bài học định sẵn, chưa cung cấp phản hồi ngữ pháp chuyên sâu hay phân tích lỗi theo ngữ cảnh phức tạp.

Đối với Gợi ý Từ vựng, Duolingo sử dụng mô hình Adaptive Learning để theo dõi tiến trình cá nhân, xác định những từ đã nắm vững và những từ cần ôn tập. Từ vựng được lồng ghép tự nhiên trong các bài học theo chủ đề và được lưu trữ trong Kho từ vựng cá nhân.

Về khả năng Luyện phản xạ/Tương tác, Duolingo đã từng triển khai tính năng Chatbot (Duolingo Bots) để người học đối thoại theo kịch bản. Hiện tại, ứng dụng đã cải tiến bằng cách sử dụng Công nghệ Nhận dạng Giọng nói và Xử lý Ngôn ngữ Tự nhiên (NLP) để mô phỏng các tình huống đối thoại, tập trung vào việc đánh giá phát âm và tốc độ, giúp rèn luyện phản xạ nghe – nói, mặc dù tính linh hoạt trong hội thoại vẫn còn hạn chế so với các mô hình ngôn ngữ lớn (LLM) hiện đại.

1.1.2. Study4



Hình 1.2. Study4

Study4 là một nền tảng học tập trực tuyến tập trung vào việc đào tạo và luyện thi các chứng chỉ ngôn ngữ quốc tế như IELTS và TOEIC, cung cấp các khóa học có cấu trúc và đề thi mô phỏng.

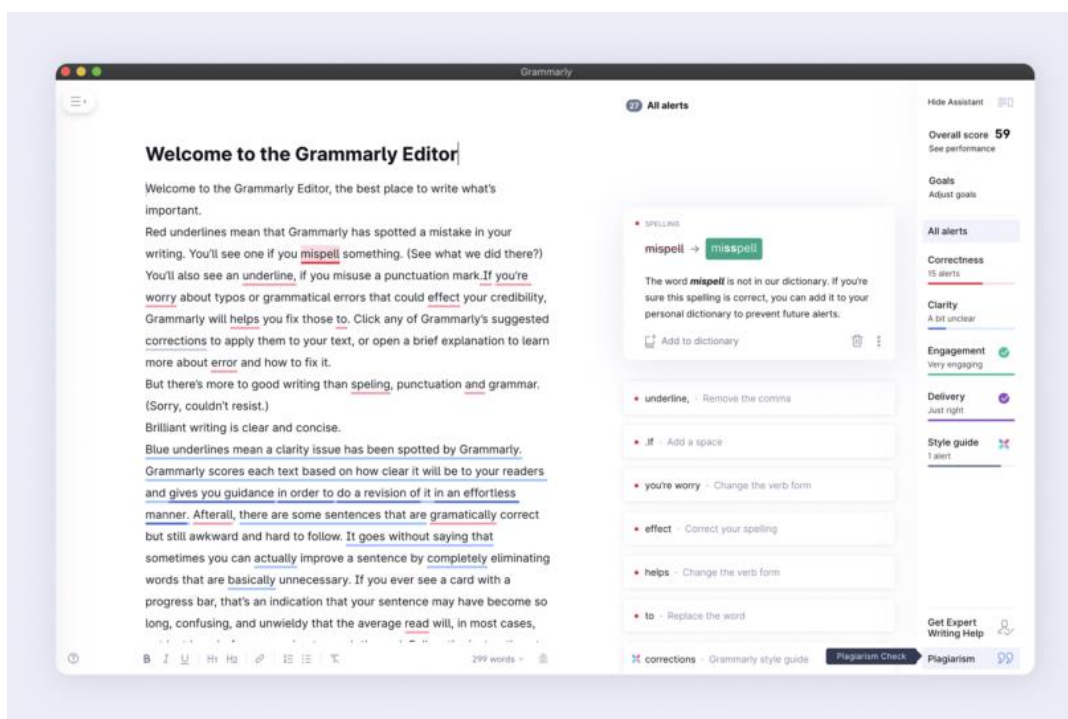
Trong lĩnh vực Phân tích Ngữ pháp, Study4 tích hợp các công cụ kiểm tra ngữ pháp tự động, đặc biệt hữu ích trong các bài tập luyện Writing. Phản hồi thường chi tiết hơn Duolingo, tập trung vào các lỗi ngữ pháp mang tính học thuật như lỗi sử dụng

thì, cấu trúc câu phức, và sự mạch lạc trong văn bản. Một số tính năng tiên tiến có thể sử dụng các mô hình AI cơ bản để chấm điểm bài viết và phân tích các dạng lỗi sai phổ biến trong môi trường thi cử.

Về Gợi ý Từ vựng, nền tảng này sử dụng Phân tích Dữ liệu Lớn từ các đề thi thực tế để xây dựng các danh sách từ vựng trọng tâm theo từng cấp độ và chủ đề (ví dụ: Academic Vocabulary). Tính năng này hỗ trợ người học tạo flashcard và làm bài tập trắc nghiệm, giúp cá nhân hóa lộ trình học từ vựng dựa trên mục tiêu điểm số và trình độ.

Về Luyện phản xạ/Tương tác, Study4 tập trung vào Luyện thi Speaking với các câu hỏi mô phỏng đề thi chính thức. Nền tảng sử dụng AI Chấm Speaking dựa trên Công nghệ Nhận dạng Giọng nói để đánh giá các tiêu chí như Độ trôi chảy (Fluency), Sự mạch lạc (Coherence), và Độ chính xác Ngữ pháp/Từ vựng, từ đó rèn luyện phản xạ trả lời theo định dạng chuẩn.

1.1.3. Grammarly



Hình 1.3. Grammarly

Grammarly là một công cụ hỗ trợ viết hàng đầu, là ví dụ tiêu biểu cho việc ứng dụng AI/LLM mạnh mẽ vào việc phân tích ngôn ngữ, mặc dù nó không phải là một ứng dụng học tập toàn diện theo nghĩa truyền thống.

Khả năng Phân tích Ngữ pháp của Grammarly vượt trội nhờ sử dụng các mô

hình Neural Networks và Deep Learning. Công cụ không chỉ sửa các lỗi cơ bản mà còn thực hiện kiểm tra ngữ pháp theo ngữ cảnh (Contextual Grammar Checking). Nó phân tích độ rõ ràng, sự mạch lạc, và tông giọng của văn bản, từ đó đưa ra các gợi ý thay thế cấu trúc câu dài dòng hoặc khó hiểu bằng các cách diễn đạt hiệu quả hơn.

Đối với Gợi ý Từ vựng, Grammarly sử dụng Xử lý Ngôn ngữ Tự nhiên (NLP) để phân tích phong cách viết và đề xuất từ, cụm từ thay thế nhằm nâng cao tính chính xác, rõ ràng, và hấp dẫn của văn bản. Ví dụ, nó có thể gợi ý sử dụng từ học thuật hơn hoặc thay thế các từ bị lặp lại, giúp người dùng cải thiện chất lượng từ vựng một cách chủ động.

Grammarly không có Chatbot luyện phản xạ nói, nhưng với sự ra đời của Grammarly Go (dựa trên Generative AI), nó cho phép người dùng tương tác viết bằng cách yêu cầu viết lại, tóm tắt, hoặc tạo bản nháp phản hồi. Điều này giúp người dùng học được cách diễn đạt tự nhiên và hiệu quả trong ngữ cảnh viết lách thực tế, gián tiếp nâng cao phản xạ viết.

1.2. Các hướng tiếp cận xây dựng ứng dụng học tiếng Anh tích hợp AI

1.2.1. Hướng tiếp cận cho phân tích ngữ pháp chuyên sâu

Mục tiêu của hướng tiếp cận này là vượt qua các công cụ kiểm tra lỗi ngữ pháp cơ bản để cung cấp phản hồi mang tính học thuật và ngữ cảnh.

- Sử dụng Mô hình Ngôn ngữ Lớn (LLM) và Mô hình Transformer[\[1\]](#): Thay vì sử dụng các thuật toán dựa trên luật (Rule-based), ứng dụng nên sử dụng các mô hình như BERT, GPT hoặc các biến thể tinh chỉnh (fine-tuned models). Các mô hình này có khả năng Phân tích Cây cú pháp (Syntactic Parsing) và hiểu được ngữ cảnh toàn cầu để phát hiện lỗi ngữ pháp phức tạp (ví dụ: lỗi về sự hòa hợp chủ-vị trong câu phức, lỗi cấu trúc câu bị động không đúng ngữ cảnh).

- Phát hiện và Phân loại Lỗi (Error Detection and Classification): Xây dựng một mô hình chuyên biệt để không chỉ báo lỗi mà còn phân loại chính xác loại lỗi (ví dụ: Tense Error, Subject-Verb Agreement, Punctuation Error). Dữ liệu huấn luyện cần bao gồm các lỗi thường gặp của người học không phải người bản xứ (Non-Native Speaker Corpus).

- Phản hồi Tạo sinh (Generative Feedback): Tận dụng khả năng tạo sinh của LLM để không chỉ sửa lỗi mà còn giải thích lỗi sai bằng ngôn ngữ tự nhiên và đề xuất

các cách viết khác nhau để đạt hiệu quả giao tiếp cao hơn (tương tự như tính năng của Grammarly Premium).

1.2.2. Hướng tiếp cận cho gợi ý từ vựng cá nhân hóa

Việc gợi ý từ vựng cần phải thích ứng với trình độ hiện tại, tốc độ tiếp thu và mục tiêu học tập (ví dụ: từ vựng học thuật, từ vựng giao tiếp, từ vựng chuyên ngành).

- Hệ thống Khuyến nghị (Recommendation Systems): Áp dụng các thuật toán Học Máy Thích ứng (Adaptive Machine Learning) để xây dựng hồ sơ từ vựng chi tiết cho từng người học. Hệ thống sẽ sử dụng kỹ thuật Hệ thống Khuyến nghị Dựa trên Nội dung (Content-based Recommendation) và Hệ thống Khuyến nghị Cộng tác (Collaborative Filtering) để đề xuất các từ vựng mới dựa trên lĩnh vực mà người học quan tâm hoặc từ vựng mà những người học cùng trình độ đang học.

- Vector hóa Từ vựng (Word Embedding): Sử dụng các mô hình nhúng từ (như Word2Vec, GloVe) để hiểu mối quan hệ ngữ nghĩa giữa các từ. Khi người học sử dụng một từ cơ bản, hệ thống có thể gợi ý các từ đồng nghĩa (synonyms) hoặc các từ vựng nâng cao (collocations) có cùng ngữ nghĩa nhưng phù hợp với ngữ cảnh cụ thể hơn.

- Tích hợp Lặp lại Ngắt quãng (Spaced Repetition System - SRS) Thông minh: Sử dụng AI để tối ưu hóa thời điểm ôn tập từ vựng, tính toán khoảng thời gian tối ưu cho từng từ dựa trên độ khó từ, mức độ thành thạo và tần suất sử dụng của người học, thay vì áp dụng một lịch trình cố định.

1.2.3. Hướng tiếp cận cho Chatbot luyện phản xạ và đối thoại

Thành phần này đóng vai trò là giáo viên ảo, cung cấp môi trường luyện nói và tương tác linh hoạt, mô phỏng cuộc hội thoại tự nhiên.

- Sử dụng Mô hình Ngôn ngữ Lớn (LLM) với Kỹ thuật Tinh chỉnh: Xây dựng Chatbot dựa trên các LLM (như mô hình của Google, Meta) được tinh chỉnh trên các bộ dữ liệu hội thoại tiếng Anh chất lượng cao. Điều này cho phép Chatbot duy trì được tính thống nhất ngữ cảnh (Context Coherence) qua nhiều lượt trao đổi và trả lời các câu hỏi mở mà không bị giới hạn bởi kịch bản.

- Quản lý Đối thoại Chủ động (Active Dialogue Management): Chatbot không chỉ trả lời mà còn chủ động dẫn dắt, đặt câu hỏi phản biện, và sửa lỗi sai của người học ngay trong khi đối thoại. Điều này được thực hiện bằng cách sử dụng các thuật

toán xác định ý định (Intent Detection) và trạng thái đối thoại hiện tại.

1.3. Các công nghệ phân tích ngữ pháp, gợi ý từ vựng và chatbot

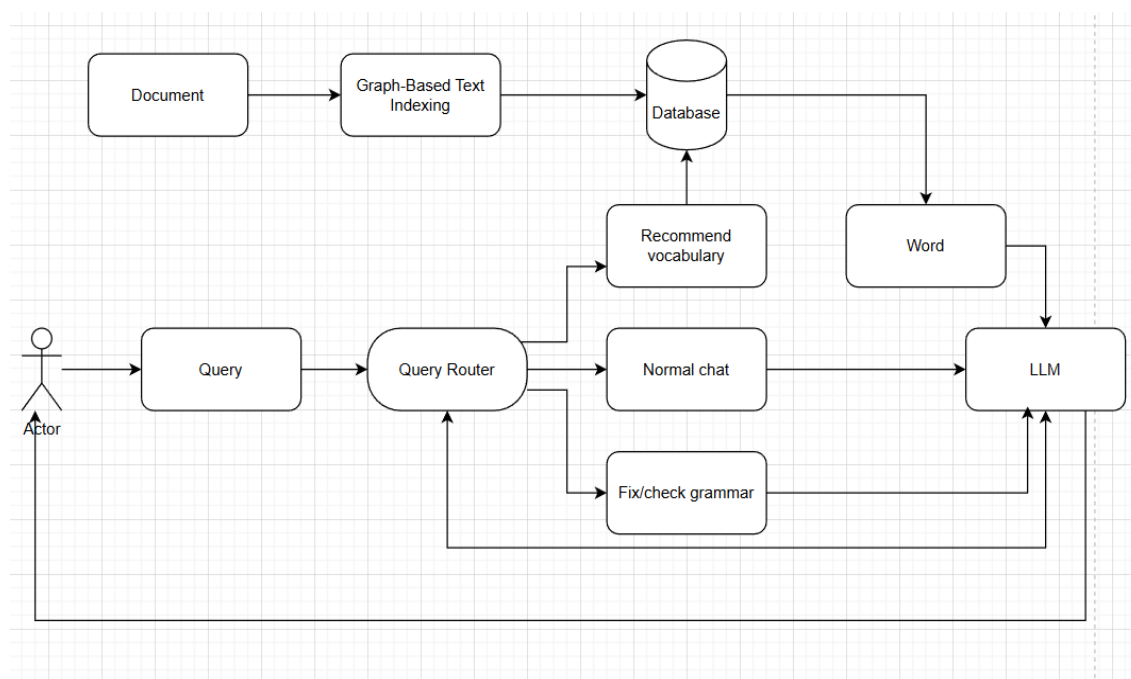
Mô hình Ngôn ngữ Lớn (LLM - Generative AI): Đây là công nghệ cốt lõi. Các mô hình như GPT-4, Gemini hoặc các LLM mã nguồn mở được tinh chỉnh (fine-tuned) cho nhiệm vụ đối thoại. Chúng giúp Chatbot tạo ra các phản hồi linh hoạt, duy trì ngữ cảnh dài, và xử lý các câu hỏi mở, mô phỏng cuộc trò chuyện thực tế một cách hiệu quả.

Mô hình Xử lý Lỗi Ngữ pháp (Grammar Error Correction - GEC): Các mô hình GEC được tinh chỉnh (fine-tuned) để tự động phát hiện, phân loại, và sửa lỗi ngữ pháp. Chúng hoạt động như một công cụ hiệu chỉnh tự động, cung cấp các gợi ý sửa chữa chính xác và tự nhiên hơn các công cụ dựa trên từ điển.

Học Tăng cường (Reinforcement Learning - RL): Có thể được sử dụng để tối ưu hóa việc lựa chọn các đề xuất sửa lỗi từ mô hình GEC, đảm bảo đề xuất sửa lỗi là tốt nhất và phù hợp với ý định của người viết.

Hệ thống Khuyến nghị (Recommendation Systems): Áp dụng thuật toán Học Máy Thích ứng (Adaptive Learning) để xây dựng hồ sơ từ vựng cá nhân. Hệ thống sử dụng dữ liệu về từ vựng đã học, đã quên, và mục tiêu của người học để đề xuất từ vựng mới theo nguyên tắc Lặp lại Ngắt quãng (SRS) tối ưu.

1.4. Đề xuất tích hợp AI cho phân tích ngữ pháp, gợi ý từ vựng và chatbot luyện phản xạ cho ứng dụng học tiếng anh



Hình 1.4. Kiến trúc tích hợp AI cho ứng dụng học tiếng anh

Thay vì xây dựng lại các mô hình máy học từ đầu, nhóm nghiên cứu tập trung vào việc tích hợp các mô hình tiên tiến sẵn có thông qua API và các thư viện mã nguồn mở để tối ưu hóa hiệu năng. Dưới đây là quy trình và các thuật toán nền tảng được sử dụng cho từng chức năng.

1.4.1. Quy trình xử lý (pipeline)

Hệ thống hoạt động theo luồng xử lý 4 bước:

- Bước 1 - Input: Nhận câu hỏi từ người dùng.
- Bước 2 - Phân loại (Router): Gửi câu hỏi kèm hướng dẫn (prompt) cho Gemini để xác định xem người dùng đang muốn "Sửa lỗi ngữ pháp", "Trò chuyện thông thường" hay "Hỏi từ vựng".
 - Bước 3 - Xử lý:
 - Nếu là Ngữ pháp hoặc Trò chuyện thông thường : Gọi trực tiếp API Gemini để nhận câu trả lời.
 - Với Ngữ pháp, đầu vào cho LLM sẽ là: [System Prompt: You are a grammar checker. Find errors in this sentence...] + [User Query]
 - Với Trò chuyện thông thường, đầu vào cho LLM thường là: [System Prompt: You are a helpful assistant] + [Chat History] + [User Query]
 - Nếu là Từ vựng: Chạy thư viện LightRAG để tìm kiếm thông tin trong Database, sau đó mới gửi kết quả tìm được cho Gemini tổng hợp.
 - Bước 4 - Output: Hiển thị kết quả định dạng JSON lên màn hình.

1.4.2. Chi tiết công nghệ và thuật toán cho từng chức năng

Dưới đây là các công nghệ cụ thể nhóm đã cài đặt và thuật toán lõi vận hành bên trong chúng:

1.4.2.1. Chức năng Sửa lỗi Ngữ pháp & Trò chuyện

Công cụ thực hiện: Gọi API của Gemini-2.5-Flash, Gemini-2.5-flash-lite, Ollama.

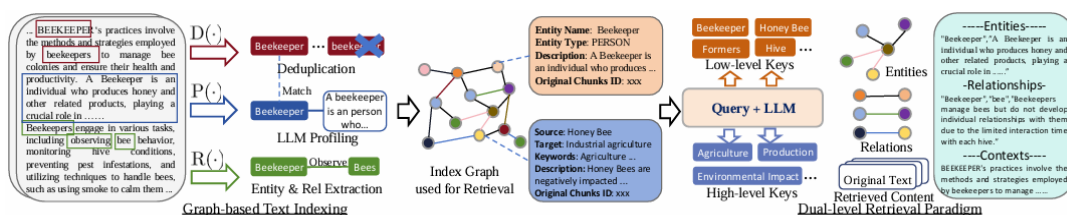
Thuật toán nền tảng: Generative AI dựa trên kiến trúc LLM.

Giải thích: Nhóm không tự train model mà sử dụng mô hình ngôn ngữ lớn đã được Google huấn luyện trước (Pre-trained).

1.4.2.2. Chức năng Gợi ý Từ vựng

Module Gợi ý Từ vựng được thiết kế nhằm khắc phục các hạn chế của phương

pháp RAG truyền thống (Naive RAG) trong việc duy trì mạch liên kết tri thức và ngữ cảnh. Hệ thống tích hợp thư viện LightRAG làm nòng cốt, kết hợp với các thuật toán vector hóa và đồ thị tri thức để tối ưu hóa độ chính xác khi truy xuất thông tin.



Hình 1.5. Tổng quan kiến trúc LightRAG

Để đảm bảo khả năng hiểu và xử lý ngôn ngữ tự nhiên, module sử dụng các thuật toán cốt lõi sau:

- **Vector hóa (Embedding):**

- **Công cụ:** Sử dụng mô hình text-embedding-004

- **Cơ chế:** Chuyển đổi dữ liệu đầu vào (từ vựng, định nghĩa, câu hỏi người dùng) thành các vector (dãy số thực) trong không gian nhiều chiều. Quá trình này giúp hệ thống chuẩn hóa dữ liệu dạng văn bản sang dạng số học để thực hiện các phép toán so sánh.

- **Tính độ tương đồng (Cosine Similarity):**

- **Cơ chế:** Tính toán khoảng cách góc giữa vector câu hỏi và vector từ vựng trong cơ sở dữ liệu.

- **Mục đích:** Xác định mức độ liên quan về mặt ngữ nghĩa. Khoảng cách càng nhỏ (giá trị Cosine càng gần 1), độ tương đồng càng cao, giúp hệ thống lọc ra các từ vựng sát nghĩa nhất với truy vấn.

- **Duyệt đồ thị (Graph Traversal):**

- **Cơ chế:** LightRAG xây dựng cấu trúc đồ thị nơi các từ vựng là các nút (nodes) và mối quan hệ giữa chúng là các cạnh (edges). Thuật toán sẽ duyệt qua các cạnh này để khai thác thông tin.

- **Mục đích:** Tìm kiếm các từ vựng liên quan gián tiếp (từ đồng nghĩa, trái nghĩa, cùng trường từ vựng) mà phương pháp so sánh vector đơn thuần có thể bỏ sót, từ đó mở rộng ngữ cảnh câu trả lời.

Khác với phương pháp phân mảnh (chunking) truyền thống thường gây đứt gãy ngữ cảnh, kiến trúc LightRAG áp dụng quy trình xử lý dữ liệu ba bước để bảo toàn

tính toàn vẹn của tri thức (Xem Hình 3.5):

- **Bước 1: Trích xuất Thực thể và Quan hệ (Entity & Relation Extraction):**

Sử dụng Mô hình Ngôn ngữ Lớn (LLM) để phân tích tài liệu thô, tự động định danh các thực thể (từ vựng, cụm từ) và xác định mối quan hệ ngữ nghĩa giữa chúng (ví dụ: là danh từ của, trái nghĩa với). Dữ liệu phi cấu trúc được chuyển đổi thành cấu trúc đồ thị liên kết chặt chẽ.

- **Bước 2: Tạo hồ sơ thông tin (LLM Profiling):** Hệ thống tạo các cặp *Khóa - Giá trị (Key-Value)* cho mỗi thực thể. Trong đó, "Key" phục vụ việc định danh và tìm kiếm nhanh, còn "Value" là đoạn tóm tắt ngữ cảnh do LLM sinh ra, giúp tăng cường khả năng hiểu sâu dữ liệu khi truy xuất.

- **Bước 3: Khử trùng lặp (Deduplication):** Các thực thể trùng lặp từ nhiều nguồn tài liệu khác nhau được hợp nhất thành một nút duy nhất trên đồ thị. Quá trình này giúp loại bỏ dư thừa dữ liệu, tối ưu hóa kích thước chỉ mục và tài nguyên lưu trữ.

Để giải quyết vấn đề độ chính xác ngữ cảnh (Context Precision) thấp (0.53 - 0.7) đã phân tích trong các thực nghiệm trước, hệ thống áp dụng cơ chế truy xuất kép:

- **Truy xuất cấp thấp (Low-level Retrieval):**

- **Mục tiêu:** Tập trung vào các thực thể cụ thể và mối quan hệ trực tiếp.

- **Ứng dụng:** Giải quyết các truy vấn chi tiết, cụ thể (ví dụ: "*Định nghĩa của từ 'negotiation'?*", "*Ví dụ về 'compromise'?*").

- **Truy xuất cấp cao (High-level Retrieval):**

- **Mục tiêu:** Tập trung vào các chủ đề rộng và khái niệm bao trùm.

- **Ứng dụng:** Xử lý các câu hỏi tổng hợp, trừu tượng (ví dụ: "*Liệt kê các từ vựng chuyên ngành đàm phán thương mại*").

Sự kết hợp này cho phép hệ thống thu thập thông tin từ cả nút đích và các nút lân cận (neighboring nodes), cung cấp ngữ cảnh toàn diện cho LLM sinh câu trả lời chính xác hơn.

Trong bối cảnh ứng dụng giáo dục cần liên tục bổ sung bài học mới, việc tái xây dựng toàn bộ chỉ mục (như GraphRAG truyền thống) gây lãng phí tài nguyên. LightRAG giải quyết bài toán này thông qua thuật toán cập nhật tăng cường:

- Hệ thống chỉ xử lý các tài liệu mới nhập vào.

- Tự động hợp nhất các nút/cạnh mới vào đồ thị hiện có mà không phá vỡ cấu

trúc cũ.

- **Kết quả:** Đảm bảo tính cập nhật của dữ liệu với chi phí vận hành thấp nhất và độ trễ tối thiểu.

1.4.2.3. Chức năng Phân loại ý định (Router)

Công cụ thực hiện: Prompt Engineering trên Gemini.

Thuật toán nền tảng: Zero-shot Classification (Phân loại không cần mẫu).

Giải thích: Thay vì dùng các thuật toán phân loại cũ (như SVM, Naive Bayes), nhóm tận dụng khả năng hiểu ngôn ngữ của Gemini. Bằng cách viết một câu lệnh (prompt) rõ ràng: "*Hãy phân loại câu sau vào nhóm Grammar hoặc Vocabulary*", AI có thể tự suy luận và phân loại chính xác mà không cần huấn luyện thêm.

1.5. Kết luận chương 1

Chương 1 đã hoàn thành việc thiết lập nền tảng lý thuyết và tổng quan cho đề tài nghiên cứu về việc xây dựng ứng dụng học tiếng Anh tích hợp AI. Nội dung của chương này đã cung cấp một cái nhìn toàn diện về bối cảnh, nhu cầu và định hướng kỹ thuật cho việc phát triển hệ thống.

Chương đã làm rõ vai trò thiết yếu của Trí tuệ Nhân tạo (AI) trong việc cá nhân hóa và nâng cao hiệu quả giáo dục ngôn ngữ. Mục tiêu trọng tâm là khắc phục những hạn chế của các phương pháp học truyền thống, đặc biệt trong việc rèn luyện phản xạ và cung cấp phản hồi ngôn ngữ theo ngữ cảnh.

Qua việc phân tích các nền tảng học và hỗ trợ ngôn ngữ hàng đầu như Duolingo, Study4, và Grammarly đã giúp xác định rõ các khoảng trống công nghệ. Các hệ thống hiện tại có xu hướng mạnh ở một khía cạnh (ví dụ: Game hóa, Hỗ trợ viết chuyên sâu) nhưng thiếu tính tích hợp toàn diện, đặc biệt là sự kết hợp của phân tích ngữ pháp chuyên sâu, gợi ý từ vựng thích ứng và một Chatbot luyện phản xạ linh hoạt dựa trên công nghệ hiện đại.

Chương đã đề xuất các hướng tiếp cận kỹ thuật tiên tiến, dựa trên việc ứng dụng các mô hình Mô hình Ngôn ngữ Lớn (LLM), kiến trúc Transformer (cho GEC) để giải quyết ba chức năng cốt lõi của ứng dụng. Cấu trúc hệ thống được đề xuất đã minh họa rõ ràng cách thức tích hợp các module AI này, trong đó LLM đóng vai trò là lõi xử lý trung tâm, đảm bảo sự phối hợp nhịp nhàng giữa các tính năng.

Những phân tích và đề xuất được trình bày trong Chương 1 đã xác lập cơ sở

vững chắc cho các công đoạn tiếp theo của đồ án. Chúng đã cung cấp định hướng rõ ràng về mặt kỹ thuật và yêu cầu chức năng. Đây là nền tảng để chuyển sang Chương 2, nơi sẽ tập trung vào việc thiết kế chi tiết hệ thống, xác định kiến trúc phần mềm, lựa chọn công nghệ cụ thể và lên kế hoạch triển khai cho từng module AI đã đề xuất.

CHƯƠNG 2. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

Chương này tập trung phân tích yêu cầu và thiết kế kiến trúc hệ thống, lựa chọn các công nghệ triển khai và xây dựng mô hình cơ sở dữ liệu, nhằm thiết lập nền tảng kỹ thuật chi tiết cho việc hiện thực hóa ứng dụng.

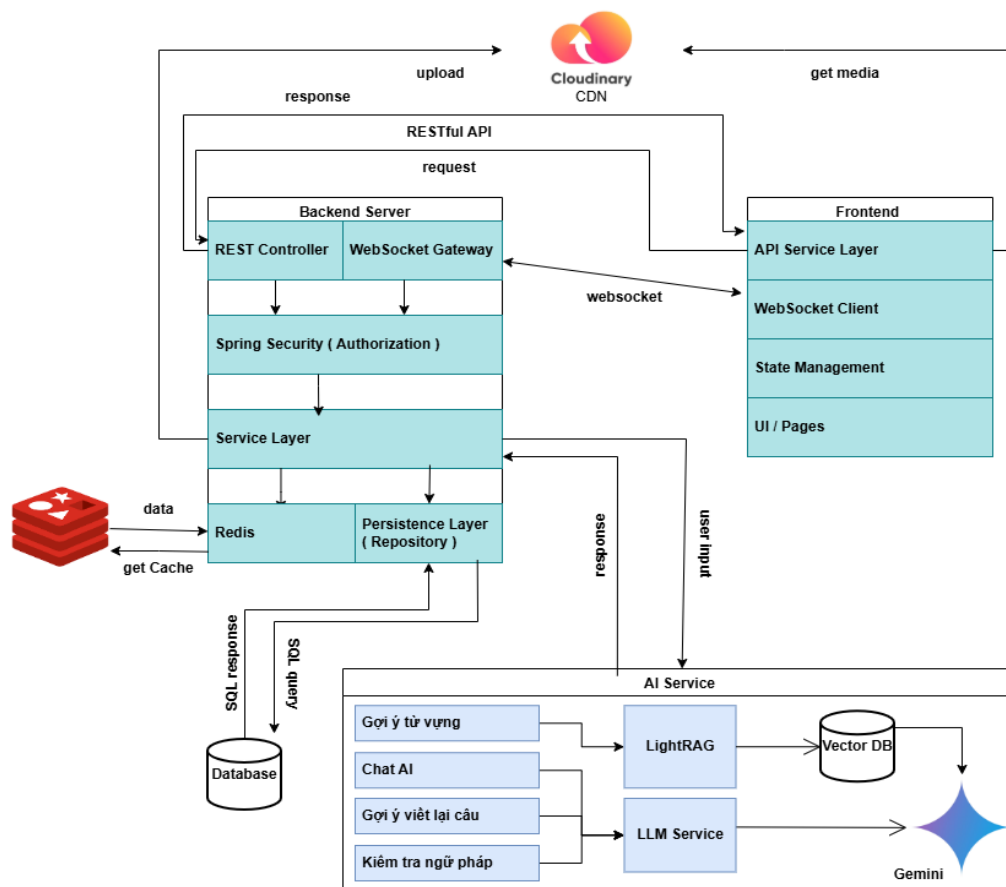
2.1. Mô hình tổng quan hệ thống

2.1.1 Giới thiệu hệ thống

Hệ thống web học tiếng Anh là một nền tảng học tập trực tuyến được xây dựng nhằm hỗ trợ người học nâng cao kỹ năng tiếng Anh thông qua các khóa học, bài học, từ vựng, video và bài luyện tập. Hệ thống cho phép người học truy cập nội dung học tập mọi lúc, mọi nơi thông qua trình duyệt web, đồng thời hỗ trợ quản trị viên quản lý nội dung, người dùng và theo dõi hoạt động của hệ thống.

Bên cạnh các chức năng học tập cơ bản, hệ thống còn tích hợp dịch vụ trí tuệ nhân tạo nhằm hỗ trợ gợi ý học tập, trả lời câu hỏi và cá nhân hóa trải nghiệm học tiếng Anh cho người dùng.

2.1.2. Mô hình tổng quát hệ thống



Hình 2.1. Mô hình tổng quát hệ thống

Hệ thống bao gồm ba khối chính là Frontend, Backend và các dịch vụ hỗ trợ bên ngoài.

Frontend là ứng dụng web cho phép người dùng tương tác với hệ thống thông qua các chức năng học tập và quản lý cá nhân. Các yêu cầu từ Frontend được gửi đến Backend để xử lý và nhận kết quả phản hồi tương ứng.

Backend đóng vai trò trung tâm, thực hiện xử lý logic nghiệp vụ, quản lý dữ liệu và điều phối các dịch vụ hỗ trợ. Backend kết nối với cơ sở dữ liệu để lưu trữ thông tin hệ thống, đồng thời tích hợp với Media Storage để lưu trữ nội dung đa phương tiện, Redis để hỗ trợ lưu trữ tạm thời và tối ưu hiệu năng, cũng như AI Service nhằm cung cấp các chức năng thông minh cho hệ thống.

2.1.3. Các tác nhân của hệ thống

Hệ thống web học tiếng Anh bao gồm các tác nhân chính sau:

- Học viên (User)

Là người sử dụng chính của hệ thống, có các chức năng:

- Đăng ký và đăng nhập tài khoản
- Tham gia các khóa học tiếng Anh
- Học bài, xem video, học từ vựng
- Làm bài luyện tập và kiểm tra
- Theo dõi tiến độ và kết quả học tập
- Nhận hỗ trợ học tập từ hệ thống AI

- Quản trị viên (Admin)

Là người quản lý và vận hành hệ thống, có các chức năng:

- Quản lý tài khoản người dùng
- Quản lý khóa học, bài học và nội dung học tập
- Cập nhật, chỉnh sửa hoặc xóa dữ liệu trong hệ thống
- Theo dõi thống kê và tình trạng hoạt động của hệ thống

2.1.4. Các thành phần chính của hệ thống

Hệ thống được xây dựng theo mô hình client-server và bao gồm các thành phần chính sau:

- Frontend

Là giao diện web cho phép học viên và quản trị viên tương tác với hệ thống thông

qua trình duyệt. Frontend chịu trách nhiệm hiển thị nội dung học tập, tiếp nhận thao tác của người dùng và gửi yêu cầu đến backend.

- Backend Server

Là thành phần xử lý nghiệp vụ chính của hệ thống, chịu trách nhiệm:

- Xác thực và phân quyền người dùng
- Xử lý logic học tập và quản lý nội dung
- Cung cấp các API để frontend giao tiếp

- AI Service

Là dịch vụ độc lập hỗ trợ trí tuệ nhân tạo, có nhiệm vụ:

- Hỗ trợ trả lời câu hỏi liên quan đến bài học
- Hệ thống sử dụng AI để phân tích, kiểm tra ngữ pháp và sửa lại câu
- Chatbot để luyện hội thoại cơ bản

- Database

Lưu trữ toàn bộ dữ liệu của hệ thống, bao gồm:

- Thông tin người dùng
- Khóa học, bài học, từ vựng
- Kết quả và tiến độ học tập

2.2. Phương pháp tiếp cận và xây dựng phần mềm

2.2.1. Phương pháp xây dựng phần mềm

Phương pháp được lựa chọn

Hệ thống được phát triển theo phương pháp Agile, một phương pháp phát triển phần mềm linh hoạt, cho phép chia nhỏ quá trình phát triển thành các vòng lặp ngắn (sprint).

Mỗi vòng lặp tập trung triển khai và hoàn thiện một hoặc một nhóm chức năng cụ thể trong mô hình tổng quát của hệ thống, bao gồm: Frontend, Backend và AI Service.

Lý do lựa chọn phương pháp Agile

Việc lựa chọn phương pháp Agile xuất phát từ các lý do sau:

- Hệ thống tích hợp AI (chatbot, kiểm tra ngữ pháp, gợi ý từ vựng) đòi hỏi quá trình thử nghiệm, đánh giá và tinh chỉnh liên tục dựa trên chất lượng phản hồi của mô hình.

- Các module trong hệ thống (Frontend, Backend, AI Service) có thể được phát triển song song và độc lập, phù hợp với cách tổ chức công việc theo sprint của Agile.
- Agile cho phép phát hiện sớm lỗi kiến trúc và logic nghiệp vụ, đặc biệt trong quá trình kết nối giữa Backend và AI Service thông qua API.

Quy trình phát triển phần mềm

Quy trình phát triển hệ thống theo phương pháp Agile bao gồm các giai đoạn chính sau:

- **Phân tích yêu cầu**

Thu thập và phân tích yêu cầu từ người dùng và quản trị viên, tập trung vào các chức năng chính trong mô hình tổng quát của hệ thống. Kết quả của giai đoạn này là danh sách các chức năng được ưu tiên triển khai theo từng sprint.

- **Thiết kế hệ thống**

Dựa trên yêu cầu đã phân tích, hệ thống được thiết kế theo kiến trúc Client–Server, trong đó:

1. Frontend và Backend được tách biệt và giao tiếp thông qua RESTful API và WebSocket.
2. Backend được tổ chức theo Layered Architecture (N-tier) nhằm tách biệt rõ ràng các tầng xử lý.
3. AI Service được thiết kế như một module độc lập, giao tiếp với Backend thông qua API.

- **Phát triển hệ thống**

Quá trình phát triển được thực hiện theo từng sprint, mỗi sprint tập trung vào một nhóm chức năng cụ thể:

1. Sprint phát triển các API Backend và kết nối cơ sở dữ liệu.
2. Sprint xây dựng giao diện Frontend và tích hợp API.
3. Sprint triển khai các chức năng AI như chatbot, kiểm tra ngữ pháp hoặc gợi ý từ vựng.

- **Kiểm thử**

Sau mỗi sprint, hệ thống kiểm thử chức năng các API Backend, luồng giao tiếp giữa Frontend và Backend và tích hợp giữa Backend và AI Service để đánh giá độ chính xác và thời gian phản hồi của các chức năng AI

- Triển khai

Đưa hệ thống vào vận hành thử nghiệm, tiếp nhận phản hồi từ người dùng và thực hiện các điều chỉnh cần thiết.

2.2.2. Thiết kế kiến trúc tổng thể

Hệ thống được thiết kế theo kiến trúc Client–Server nhằm tách biệt rõ ràng giữa giao diện người dùng và xử lý nghiệp vụ, giúp tăng khả năng mở rộng và bảo trì.

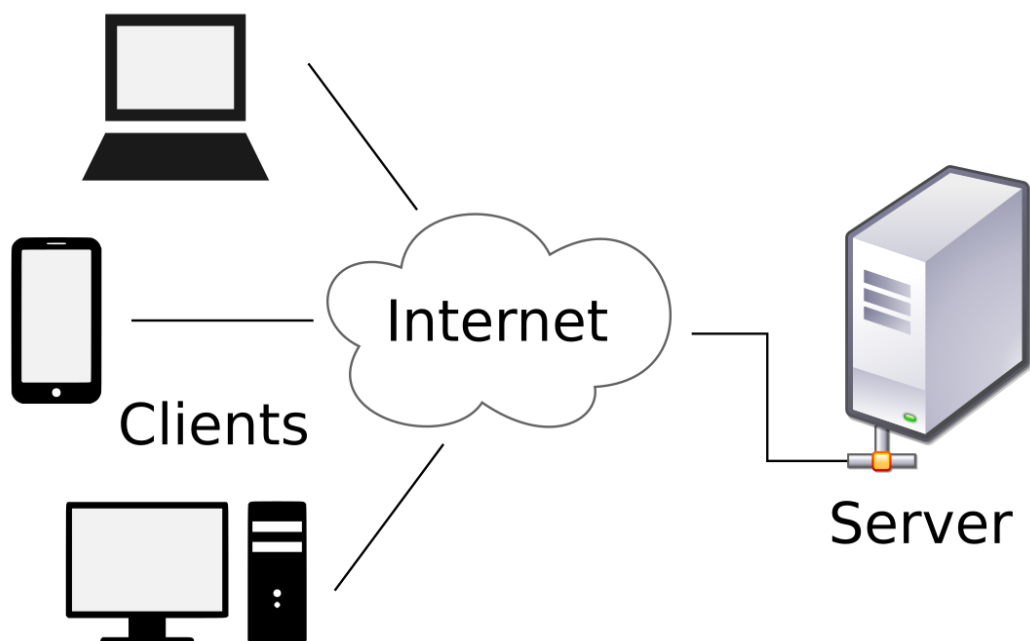
Frontend đóng vai trò tương tác với người dùng, gửi yêu cầu và nhận dữ liệu từ Backend thông qua RESTful API và WebSocket.

Backend được tổ chức theo kiến trúc phân tầng (Layered Architecture). Presentation Layer tiếp nhận và xử lý các yêu cầu từ Frontend, Service Layer đảm nhiệm xử lý logic nghiệp vụ và điều phối các chức năng của hệ thống, trong khi Persistence Layer chịu trách nhiệm truy xuất và quản lý dữ liệu. Ngoài ra, Backend tích hợp Redis để lưu trữ dữ liệu tạm thời và cache, giúp cải thiện hiệu năng hệ thống.

Các dịch vụ bên ngoài như Media Storage và AI Service được Backend truy cập thông qua Service Layer, đảm bảo tuân thủ nguyên tắc phân tầng và tách biệt trách nhiệm.

2.2.3. Kiến trúc phần mềm

2.2.3.1 Kiến trúc Client–Server Architecture



Hình 2.2. Kiến trúc Client - Server

Hệ thống được thiết kế và triển khai theo kiến trúc Client–Server, trong đó các thành phần được phân tách rõ ràng giữa phía người dùng (Client) và phía xử lý trung tâm (Server). Kiến trúc này giúp hệ thống dễ dàng mở rộng, bảo trì và đảm bảo tính bảo mật trong quá trình trao đổi dữ liệu.

Client Side (Frontend)

Client là ứng dụng web được xây dựng bằng React, chạy trên trình duyệt của người dùng. Client có nhiệm vụ:

- Hiển thị giao diện người dùng (UI) và các trang chức năng học tập.
- Gửi yêu cầu đến Backend Server thông qua RESTful API và WebSocket.
- Nhận phản hồi từ Server và cập nhật trạng thái giao diện tương ứng.

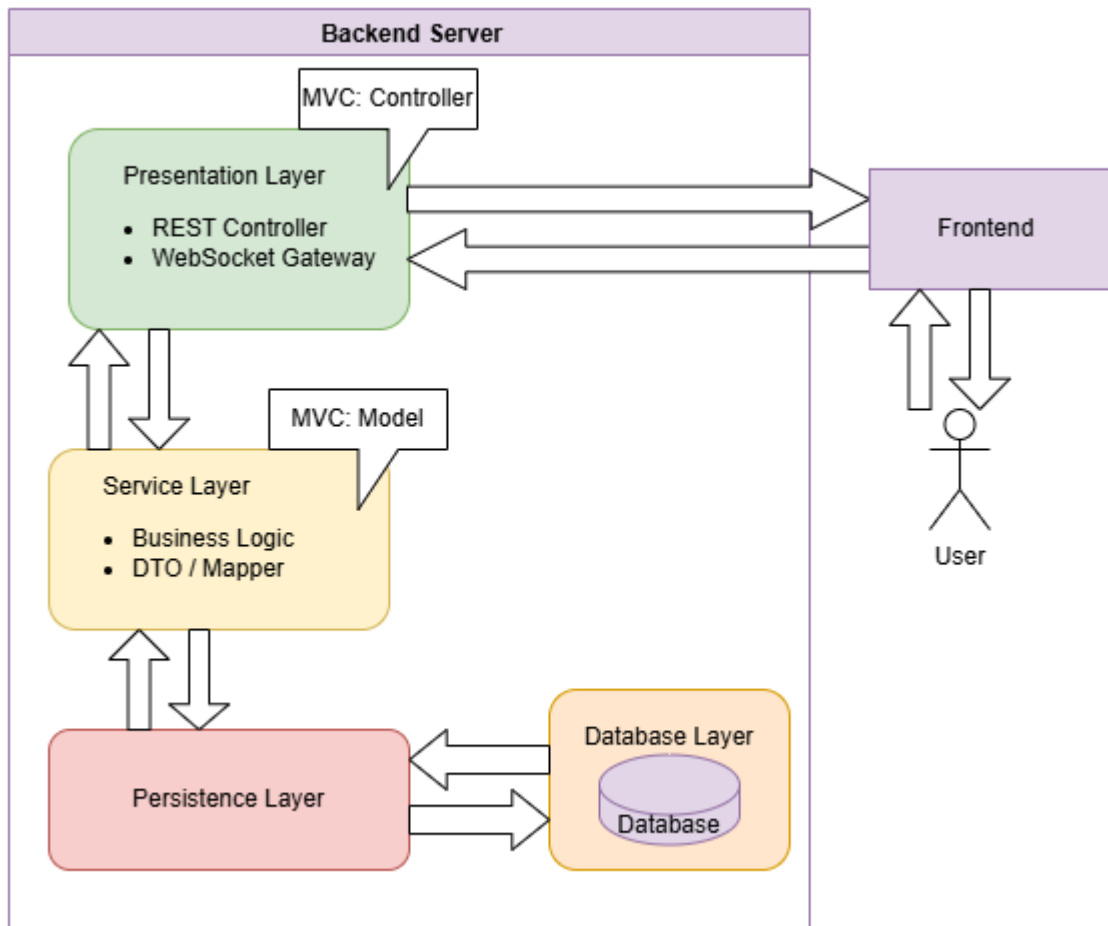
Server Side (Backend Server)

Server đóng vai trò trung tâm xử lý toàn bộ logic của hệ thống. Backend Server có các nhiệm vụ chính:

- Tiếp nhận và xử lý các yêu cầu từ Client thông qua REST Controller và WebSocket Gateway.
- Xử lý nghiệp vụ học tập, quản lý người dùng, bài học và tiến trình học tập.
- Tương tác với cơ sở dữ liệu để lưu trữ và truy xuất dữ liệu.

2.2.3.2. Kiến trúc Backend – Phân tầng kết hợp tư duy MVC

Trong hệ thống, mô hình MVC không được triển khai như một kiến trúc độc lập mà được vận dụng như một tư duy tổ chức và phân tách mã nguồn bên trong kiến trúc phân tầng của Backend. Việc áp dụng tư duy MVC giúp hệ thống đảm bảo nguyên tắc tách biệt trách nhiệm giữa các thành phần, đồng thời tăng khả năng mở rộng và bảo trì trong quá trình phát triển.



Hình 2.3. Kiến trúc phân tầng kết hợp tư duy MVC

Cụ thể, các thành phần của mô hình MVC được ánh xạ vào các tầng của Backend như sau:

Controller đóng vai trò tiếp nhận các yêu cầu từ phía Client thông qua các endpoint RESTful. Thành phần này chịu trách nhiệm kiểm tra và xử lý dữ liệu đầu vào, sau đó chuyển tiếp yêu cầu đến Service Layer để thực hiện các nghiệp vụ tương ứng. Trong hệ thống, Controller tương ứng với thành phần Controller trong mô hình MVC.

Model được thể hiện thông qua các Entity, DTO và các lớp xử lý nghiệp vụ trong Service Layer. Trong đó, Entity đại diện cho các đối tượng dữ liệu được ánh xạ với các bảng trong cơ sở dữ liệu thông qua JPA/Hibernate; DTO được sử dụng để truyền dữ liệu giữa các tầng nhằm tránh việc lộ trực tiếp cấu trúc Entity ra bên ngoài; Mapper đảm nhiệm việc chuyển đổi giữa Entity và DTO, góp phần đảm bảo tính tách biệt giữa dữ liệu lưu trữ và dữ liệu trao đổi với Client.

Thành phần View trong mô hình MVC không được triển khai trong Backend

mà được đặt tại phía Frontend. Do đó, hệ thống áp dụng mô hình MVC phân tán [3], trong đó Backend cung cấp dữ liệu và kết quả xử lý, còn Frontend chịu trách nhiệm hiển thị giao diện và tương tác với người dùng.

Việc kết hợp kiến trúc phân tầng với tư duy mô hình MVC giúp Backend có cấu trúc rõ ràng, đảm bảo tính nhất quán trong tổ chức mã nguồn, đồng thời phù hợp với các hệ thống theo mô hình Client–Server.

2.2.4. Công nghệ xây dựng hệ thống

Các công nghệ được lựa chọn không chỉ đảm bảo tính hiện đại mà còn được phân bổ chính xác vào các khối chức năng trong mô hình kiến trúc Client-Server tích hợp AI.

2.2.4.1. Khối Database và Storage

Đảm nhiệm vai trò lưu trữ bền vững và tăng tốc độ truy xuất dữ liệu cho hệ thống.

Cơ sở dữ liệu PostgreSQL

Giới thiệu chung

PostgreSQL là một hệ quản trị cơ sở dữ liệu quan hệ – đối tượng (ORDBMS) mã nguồn mở, hoạt động theo mô hình client–server. PostgreSQL được phát triển với mục tiêu cung cấp một hệ quản trị cơ sở dữ liệu mạnh mẽ, ổn định, tuân thủ chặt chẽ các tiêu chuẩn SQL và hỗ trợ nhiều kiểu dữ liệu nâng cao.

PostgreSQL được sử dụng rộng rãi trong các hệ thống lớn và có thể được cài đặt trên nhiều hệ điều hành khác nhau như Windows, Linux và macOS. Hệ quản trị này đặc biệt phù hợp cho các ứng dụng yêu cầu tính toàn vẹn dữ liệu cao, khả năng mở rộng tốt và xử lý các nghiệp vụ phức tạp.

Với PostgreSQL, người dùng có thể dễ dàng tạo và kết nối cơ sở dữ liệu với các website và ứng dụng được phát triển bằng nhiều ngôn ngữ lập trình khác nhau như Java, C#, Python, NodeJS, PHP,... thông qua các thư viện và driver hỗ trợ phong phú.

Ưu điểm của PostgreSQL

- Mã nguồn mở: PostgreSQL là hệ quản trị cơ sở dữ liệu mã nguồn mở hoàn toàn, cho phép người dùng tự do sử dụng, chỉnh sửa và phân phối mà không phải trả chi phí bản quyền.

- Tuân thủ chuẩn SQL cao: PostgreSQL hỗ trợ đầy đủ và chặt chẽ các tiêu chuẩn SQL, giúp đảm bảo tính nhất quán và khả năng tương thích trong quá trình phát triển hệ thống.

- Hiệu suất và độ ổn định cao: PostgreSQL được thiết kế để xử lý khối lượng dữ liệu lớn và các truy vấn phức tạp, hoạt động ổn định trong các hệ thống có tải cao và yêu cầu tính sẵn sàng cao.

- Khả năng mở rộng mạnh mẽ: PostgreSQL hỗ trợ nhiều cơ chế mở rộng như stored procedure, trigger, custom data type, extension và cơ chế nhân bản (replication), đáp ứng tốt nhu cầu phát triển lâu dài của hệ thống.

- Hỗ trợ đa nền tảng: PostgreSQL có thể chạy trên nhiều hệ điều hành khác nhau như Windows, Linux và macOS, thuận tiện cho việc triển khai và vận hành.

Nhược điểm của PostgreSQL

- Độ phức tạp cao hơn: PostgreSQL có nhiều tính năng nâng cao, do đó việc cấu hình và quản trị hệ thống có thể phức tạp hơn đối với người mới bắt đầu.

- Tốc độ với truy vấn đơn giản: Trong một số trường hợp truy vấn đơn giản hoặc hệ thống nhỏ, hiệu năng của PostgreSQL có thể không vượt trội so với các hệ quản trị cơ sở dữ liệu nhẹ hơn.

- Yêu cầu tài nguyên hệ thống: PostgreSQL thường yêu cầu tài nguyên phần cứng cao hơn (RAM, CPU) để khai thác tối đa hiệu năng, đặc biệt đối với các hệ thống có quy mô lớn.

Redis Cache

Redis là một hệ quản trị cơ sở dữ liệu NoSQL lưu trữ dữ liệu trong bộ nhớ, được sử dụng như một hệ thống cache nhằm tăng tốc độ truy xuất dữ liệu và giảm tải cho hệ thống backend.

Trong hệ thống học tiếng Anh, Redis được sử dụng để lưu trữ tạm thời các dữ liệu có tần suất truy cập cao hoặc các kết quả xử lý tốn nhiều tài nguyên, đặc biệt là các kết quả sinh ra từ AI. Việc sử dụng Redis giúp hệ thống phản hồi nhanh hơn, cải thiện trải nghiệm người dùng và nâng cao

Redis hỗ trợ nhiều cấu trúc dữ liệu như String, List, Set và Hash, cho phép hệ thống linh hoạt trong việc lưu trữ và quản lý dữ liệu. Ngoài ra, Redis còn cung cấp cơ chế thời gian sống (TTL – Time To Live) cho dữ liệu cache, giúp tự động loại bỏ

các dữ liệu không còn cần thiết, đảm bảo bộ nhớ được sử dụng hiệu quả.

Nhờ khả năng xử lý nhanh và độ trễ thấp, Redis đóng vai trò quan trọng trong việc tối ưu hiệu năng cho các chức năng yêu cầu phản hồi gần thời gian thực, đặc biệt là các tính năng AI hỗ trợ học tập trong hệ thống.

2.2.4.2. Khối Backend Server

Thành phần trung tâm chịu trách nhiệm điều phối logic toàn hệ thống và kết nối giữa giao diện với dữ liệu.

Giới thiệu ngôn ngữ lập trình Java

Java là một ngôn ngữ lập trình hướng đối tượng, có cú pháp rõ ràng, dễ học và dễ bảo trì. Java được phát triển với mục tiêu cho phép chương trình có thể chạy trên nhiều nền tảng khác nhau mà không cần thay đổi mã nguồn, thông qua cơ chế “viết một lần, chạy mọi nơi” (Write Once, Run Anywhere).

Hiện nay, Java được sử dụng rộng rãi trong nhiều lĩnh vực như phát triển ứng dụng web, ứng dụng di động, phần mềm máy tính cá nhân, hệ thống máy chủ và các hệ thống nhúng. Nhờ tính ổn định, bảo mật cao và cộng đồng phát triển lớn, Java là một trong những ngôn ngữ lập trình phổ biến nhất trong phát triển phần mềm doanh nghiệp.

JVM và quá trình hoạt động của một ứng dụng Java

JVM (Java Virtual Machine – Máy ảo Java) là một môi trường thực thi cho các chương trình Java. JVM đóng vai trò trung gian giữa mã chương trình Java và hệ điều hành, giúp các ứng dụng Java có thể hoạt động trên nhiều nền tảng khác nhau mà không phụ thuộc vào hệ điều hành cụ thể.

Quá trình hoạt động của một ứng dụng Java được mô tả như sau:

- Chương trình được viết bằng ngôn ngữ Java thông qua môi trường phát triển tích hợp (IDE) hoặc trình soạn thảo văn bản.
- Mã nguồn Java được biên dịch thành bytecode bởi trình biên dịch Java (javac). Bytecode là dạng mã trung gian, không phụ thuộc vào nền tảng và có định dạng tệp .class.
- Các tệp bytecode cùng với các tài nguyên liên quan (hình ảnh, âm thanh, tệp cấu hình, ...) được đóng gói thành tệp JAR (Java Archive) hoặc WAR (Web Archive) tùy theo loại ứng dụng.

- Bytecode được nạp và thực thi bởi JVM. JVM chuyển bytecode thành mã máy phù hợp với hệ điều hành đang chạy.
- Ứng dụng Java được thực thi trên máy tính hoặc thiết bị cụ thể thông qua JVM.

Nhờ cơ chế này, Java đảm bảo tính đa nền tảng và khả năng tương thích cao trong quá trình triển khai ứng dụng.

Giới thiệu framework Spring

Spring Framework là một framework phát triển ứng dụng Java phía máy chủ (server-side) mạnh mẽ và phổ biến, được sử dụng rộng rãi trong việc xây dựng các ứng dụng web và ứng dụng doanh nghiệp. Spring giúp đơn giản hóa quá trình phát triển, nâng cao khả năng mở rộng, tái sử dụng mã nguồn và dễ dàng kiểm thử hệ thống.

Spring được thiết kế theo kiến trúc mô-đun, cho phép lập trình viên chỉ sử dụng những thành phần cần thiết cho ứng dụng. Framework này được xây dựng dựa trên hai nguyên tắc thiết kế cốt lõi:

- Dependency Injection (DI): Giúp giảm sự phụ thuộc chặt chẽ giữa các thành phần trong hệ thống, tăng tính linh hoạt và khả năng bảo trì.
- Aspect-Oriented Programming (AOP): Hỗ trợ tách các chức năng dùng chung như logging, bảo mật, xử lý giao dịch ra khỏi logic nghiệp vụ chính.

Các module chính của Spring được sử dụng trong hệ thống bao gồm:

- Spring Boot: Cung cấp tập hợp các thư viện và cấu hình sẵn, giúp xây dựng và triển khai ứng dụng Spring nhanh chóng.
- Spring Security: Hỗ trợ bảo mật ứng dụng thông qua cơ chế xác thực và phân quyền người dùng.
- Spring Data JPA: Cung cấp các công cụ truy cập và xử lý dữ liệu, giúp làm việc với cơ sở dữ liệu dễ dàng hơn.
- Spring REST (Spring Web): Hỗ trợ xây dựng và mô tả các dịch vụ web theo kiến trúc RESTful API.

2.2.4.3. Khối Frontend

Đóng vai trò là lớp hiển thị và tiếp nhận tương tác từ Học viên và Quản trị viên thông qua trình duyệt web.

Giới thiệu ngôn ngữ lập trình TypeScript

TypeScript là một ngôn ngữ lập trình mã nguồn mở do Microsoft phát triển, được xây dựng dựa trên JavaScript. TypeScript mở rộng JavaScript bằng cách bổ sung hệ thống kiểu dữ liệu tĩnh (static typing), giúp phát hiện lỗi ngay trong quá trình viết mã và nâng cao chất lượng phần mềm.

TypeScript được sử dụng phổ biến trong phát triển các ứng dụng web hiện đại, đặc biệt là các ứng dụng có quy mô lớn và yêu cầu cao về khả năng bảo trì. Mã nguồn TypeScript sẽ được biên dịch sang JavaScript trước khi thực thi, do đó có thể chạy trên tất cả các trình duyệt và nền tảng hỗ trợ JavaScript.

- Ưu điểm của TypeScript

- **Hỗ trợ kiểm tra kiểu dữ liệu:** TypeScript giúp phát hiện sớm các lỗi liên quan đến kiểu dữ liệu trong quá trình phát triển, từ đó giảm lỗi khi chạy chương trình.
- **Dễ bảo trì và mở rộng:** Với cú pháp rõ ràng và hỗ trợ lập trình hướng đối tượng, TypeScript phù hợp cho các dự án lớn và phức tạp.
- **Tương thích hoàn toàn với JavaScript:** TypeScript có thể sử dụng toàn bộ thư viện và mã nguồn JavaScript hiện có.
- **Hỗ trợ tốt từ công cụ phát triển:** TypeScript được hỗ trợ mạnh mẽ bởi các IDE, giúp tăng năng suất lập trình.

- Nhược điểm của TypeScript

- **Cần bước biên dịch:** Mã nguồn TypeScript phải được biên dịch sang JavaScript trước khi chạy, làm tăng thêm một bước trong quá trình phát triển.
- **Thời gian học ban đầu:** Đối với người mới, việc làm quen với hệ thống kiểu dữ liệu của TypeScript có thể mất thời gian hơn so với JavaScript thuần.

Giới thiệu thư viện ReactJS

ReactJS là một thư viện JavaScript mã nguồn mở do Facebook phát triển, được sử dụng để xây dựng giao diện người dùng (User Interface – UI) cho các ứng dụng web hiện đại, đặc biệt là các ứng dụng đơn trang (Single Page Application – SPA).

ReactJS cho phép xây dựng giao diện theo mô hình component, trong đó mỗi thành phần giao diện được tách riêng, dễ quản lý và tái sử dụng. ReactJS sử dụng cơ chế Virtual DOM giúp tối ưu hóa việc cập nhật giao diện, từ đó nâng cao hiệu năng của ứng dụng.

- Ưu điểm của ReactJS

- Dễ phát triển giao diện động: ReactJS giúp xây dựng các giao diện tương tác cao và cập nhật dữ liệu theo thời gian thực.
- Component-based: Việc chia giao diện thành các component độc lập giúp mã nguồn dễ bảo trì và tái sử dụng.
- Hiệu năng cao: Virtual DOM giúp giảm số lần thao tác trực tiếp với DOM thật, cải thiện hiệu suất ứng dụng.
- Hệ sinh thái phong phú: ReactJS có cộng đồng lớn và nhiều thư viện hỗ trợ như React Router, Redux, Zustand.
- Kết hợp tốt với TypeScript: ReactJS hoạt động hiệu quả khi kết hợp với TypeScript, giúp tăng độ an toàn và chất lượng mã nguồn.

- Nhược điểm của ReactJS

- Không phải framework hoàn chỉnh: ReactJS chỉ là thư viện xây dựng giao diện, cần kết hợp thêm các thư viện khác để xây dựng ứng dụng hoàn chỉnh.
- Đường cong học tập ban đầu: Việc làm quen với JSX, component và quản lý state có thể gây khó khăn cho người mới.

2.2.4.4. Khối AI Service

Ngôn ngữ lập trình Python

Python là một ngôn ngữ lập trình bậc cao, thông dịch, mã nguồn mở, được thiết kế với mục tiêu đơn giản, dễ đọc và dễ học. Python hỗ trợ nhiều mô hình lập trình khác nhau như lập trình hướng đối tượng, lập trình hàm và lập trình thủ tục, giúp lập trình viên dễ dàng xây dựng các ứng dụng đa dạng.

Python được sử dụng rộng rãi trong nhiều lĩnh vực như phát triển ứng dụng web, khoa học dữ liệu, trí tuệ nhân tạo, học máy, tự động hóa và xử lý dữ liệu. Nhờ cú pháp ngắn gọn và cộng đồng phát triển lớn, Python trở thành một trong những ngôn ngữ lập trình phổ biến nhất hiện nay.

Ứng dụng của Python trong hệ thống

Trong hệ thống web học tiếng Anh, Python được sử dụng chủ yếu để xây dựng các dịch vụ hỗ trợ trí tuệ nhân tạo và xử lý ngôn ngữ tự nhiên. Python giúp triển khai các chức năng như phân tích nội dung học tập, gợi ý lộ trình học và hỗ trợ trả lời câu hỏi cho người dùng.

FastAPI

FastAPI là một framework phát triển web hiện đại cho Python, được thiết kế để xây dựng các RESTful API với hiệu suất cao và dễ bảo trì. FastAPI dựa trên các tiêu chuẩn như ASGI, OpenAPI và JSON Schema, cho phép tự động sinh tài liệu API.

- Ưu điểm của FastAPI:

- Hiệu suất cao: FastAPI có tốc độ xử lý nhanh.
- Dễ sử dụng: Cú pháp rõ ràng, dễ học và dễ triển khai.
- Hỗ trợ bất đồng bộ (async/await): Phù hợp với các dịch vụ AI cần xử lý nhiều yêu cầu đồng thời.
- Tích hợp tốt với các thư viện AI/ML trong Python.

FastAPI được sử dụng trong hệ thống để xây dựng AI Service, cung cấp các API phục vụ việc xử lý và phân tích dữ liệu học tập.

ChromaDB

ChromaDB là một cơ sở dữ liệu vector (Vector Database), được thiết kế để lưu trữ và truy vấn các vector embedding, thường được sử dụng trong các bài toán tìm kiếm ngữ nghĩa và hệ thống hỏi – đáp.

- Chức năng chính của ChromaDB:

- Lưu trữ embedding của văn bản
- Thực hiện tìm kiếm tương đồng (similarity search)
- Hỗ trợ truy vấn dữ liệu dựa trên ngữ nghĩa thay vì từ khóa

- Ưu điểm của ChromaDB:

- Nhẹ và dễ triển khai
- Hiệu quả trong các hệ thống AI quy mô vừa và nhỏ
- Phù hợp với các ứng dụng học tập và chatbot

2.3. Phân tích hệ thống

2.3.1. Mục tiêu của hệ thống

Hệ thống được xây dựng nhằm hỗ trợ việc học tiếng Anh trực tuyến một cách hiệu quả, thuận tiện. Các mục tiêu chính của hệ thống bao gồm:

- Cung cấp nền tảng học tiếng Anh trực tuyến với nội dung phong phú, có cấu trúc rõ ràng.

- Hỗ trợ người học theo dõi tiến độ học tập và đánh giá kết quả học tập.
- Tăng cường tương tác giữa người học thông qua chức năng bình luận và đánh giá.
- Hỗ trợ quản trị viên trong việc quản lý nội dung, người dùng và hệ thống.
- Ứng dụng trí tuệ nhân tạo (AI) nhằm hỗ trợ giao tiếp, phân tích và sửa lỗi sai ngữ pháp cho người dùng.

2.3.2. Đối tượng sử dụng hệ thống

Hệ thống phục vụ hai nhóm đối tượng chính:

- Học viên (User): là người sử dụng hệ thống để học tiếng Anh, làm bài tập, theo dõi tiến độ học tập, bình luận và nhận thông báo.
- Quản trị viên (Admin): là người quản lý hệ thống, chịu trách nhiệm quản lý người dùng, nội dung khóa học, phân quyền và giám sát hoạt động của hệ thống.

2.3.3. Xác định và miêu tả các usecase sử dụng

a. Danh sách usecase cho Người dùng (User)

STT	Tên Usscase	Mô tả
1	Đăng ký tài khoản	Người dùng đăng ký tài khoản để sử dụng và truy cập vào hệ thống
2	Đăng nhập hệ thống	Người dùng đăng nhập vào hệ thống bằng tài khoản, mật khẩu đã đăng ký
3	Sửa thông tin cá nhân	Người dùng chỉnh sửa và bổ sung thông tin cá nhân
4	Xem danh sách bài học	Người dùng truy cập vào hệ thống, xem danh sách bài học
5	Xem danh sách đơn vị bài học	Người dùng truy cập vào hệ thống, xem danh sách đơn vị bài học
6	Học và luyện tập bài học	Người dùng truy cập hệ thống, chọn một trong các bài trong khóa học để học và luyện làm bài tập

7	Bình luận, đánh giá bài học	Người dùng bình luận và đánh giá chất lượng bài học trong khóa học.
8	Luyện nói tiếng anh với AI	Chatbot hỗ trợ luyện nói tiếng Anh thông qua hội thoại
9	Hỗ trợ viết tiếng Anh với AI	Chatbot trò chuyện giúp kiểm tra ngữ pháp, phát hiện lỗi sai và gợi ý viết lại câu tự nhiên hơn
10	Học tiếng anh bằng flashcard	Người dùng truy cập vào hệ thống, chọn mục Flashcard để học từ vựng tiếng anh
11	Hỗ trợ từ vựng với AI	Người dùng đang học kỹ năng viết, khi viết 1 từ sẽ gợi ý 1 từ đồng nghĩa, liên quan
12	Xem thống kê	Người dùng truy cập vào Dashboard để có thể xem thống kê chi tiết thông tin

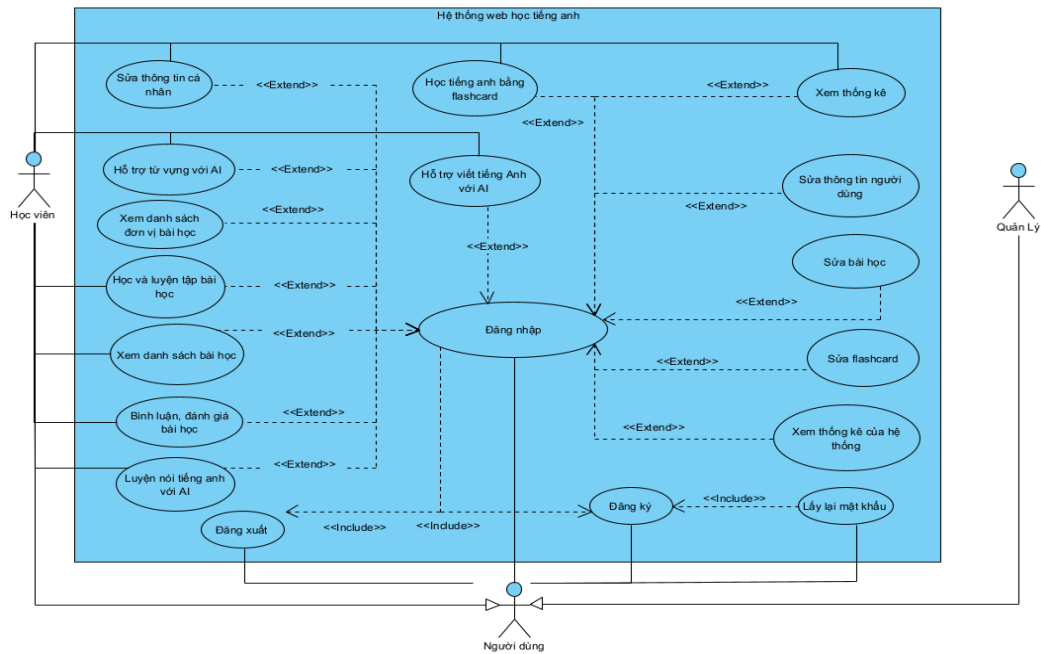
Bảng 2.1. Danh sách usecase cho Người dùng

b. Danh sách usecase cho Quản trị viên (Admin)

STT	Tên Usscase	Mô tả
1	Đăng nhập	Người quản trị đăng nhập hệ thống
2	Sửa thông tin người dùng	Người quản trị lấy danh sách người dùng và chỉnh sửa thông tin của người dùng
3	Sửa bài học	Người quản trị lấy danh sách các bài học, chỉnh sửa nội dung học
4	Sửa flashcard	Người quản trị lấy danh sách bộ thẻ học và sửa thông tin thẻ học
5	Xem thống kê	Người quản trị xem thống kê hoạt động người dùng toàn hệ thống

Bảng 2.2. Danh sách usecase cho Quản trị viên

2.3.4. Biểu đồ usecase tổng quan

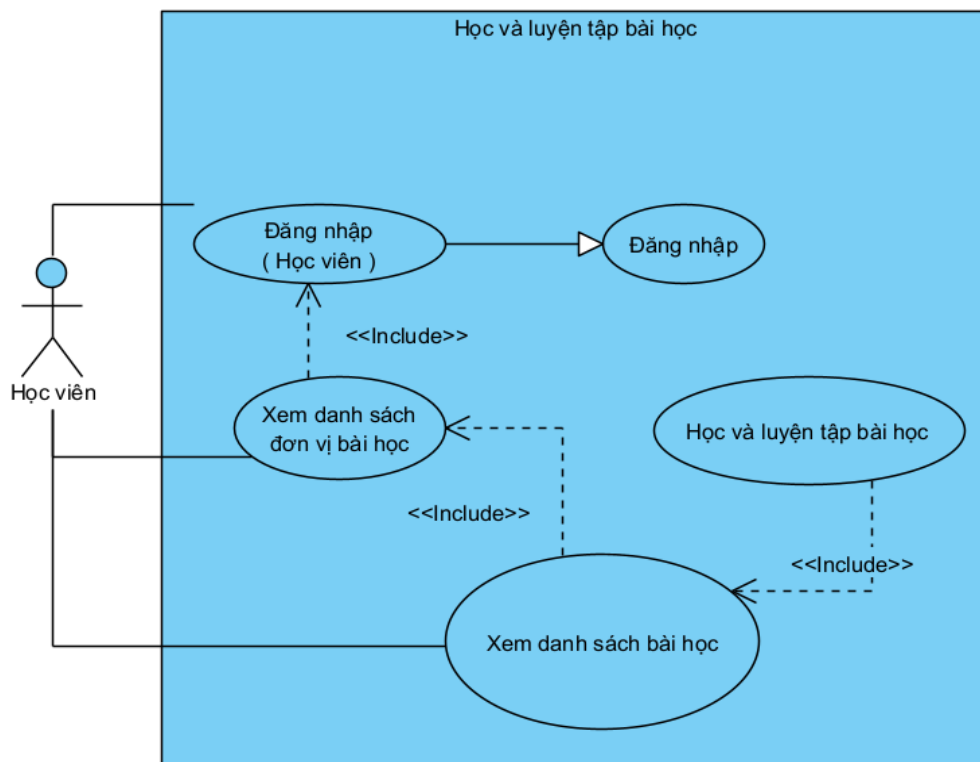


Hình 2.4. Usecase tổng quan

2.3.5. Biểu đồ Use case và kịch bản một số chức năng chính

2.3.5.1. Chức năng của người dùng

- Học và luyện tập bài học



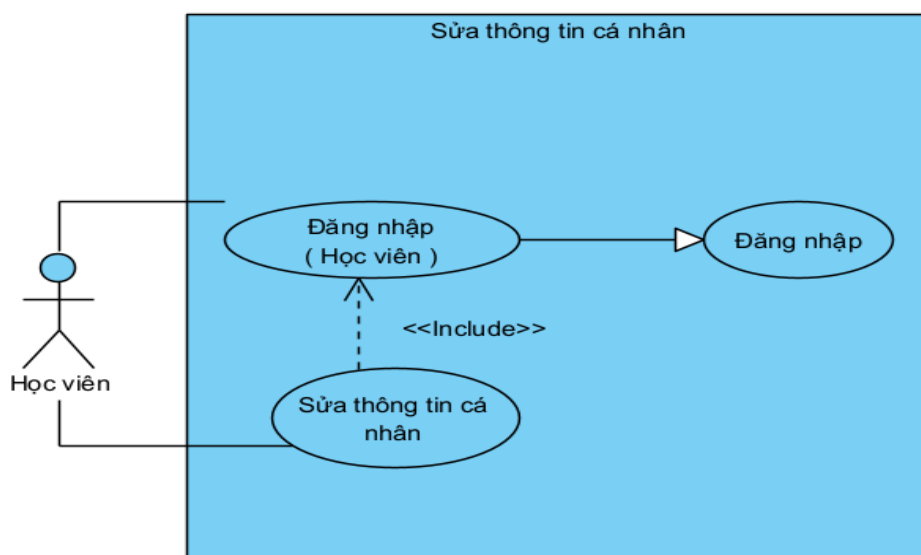
Hình 2.5. Biểu đồ usecase học và luyện tập bài học

Kịch bản chức năng học và luyện tập bài học

Tên use case	Học và luyện tập bài học
Tác nhân	Học viên
Điều kiện	Người dùng đã đăng nhập và đang ở giao diện bài học
Đảm bảo thành công	Người dùng hoàn thành bài học và lưu lại dữ liệu tiến độ
Kịch bản chính	1. Học viên truy cập vào bài học cụ thể trong đơn vị bài học 2. Hệ thống hiển thị nội dung bài học, bao gồm: văn bản, hình ảnh, audio/video, bài tập kèm theo 3. Học viên đọc, xem hoặc nghe nội dung bài học và thực hiện các bài tập 4. Hệ thống kiểm tra kết quả bài tập, đưa ra phản hồi ngay lập tức 5. Học viên hoàn thành bài học và nhấn Hoàn thành 6. Hệ thống lưu tiến độ học tập và kết quả bài tập vào cơ sở dữ liệu và thông báo học viên đã hoàn thành bài học
Ngoại lệ	6.1 Lỗi hệ thống khi lưu dữ liệu, thông báo thất bại và yêu cầu thử lại

Bảng 2.3. Kịch bản chức năng học và luyện tập bài học

- Sửa thông tin cá nhân



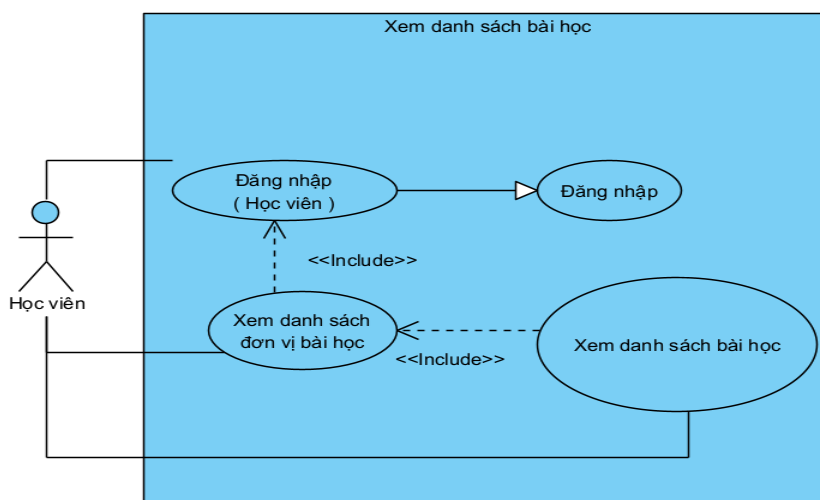
Hình 2.6. Biểu đồ sửa thông tin cá nhân

Kịch bản sửa thông tin cá nhân

Tên use case	Sửa thông tin cá nhân
Tác nhân	Học viên
Điều kiện	Học viên đã đăng nhập thành công vào hệ thống và đang ở giao diện trang cá nhân
Đảm bảo thành công	Người dùng cập nhật thông tin cá nhân thành công và lưu vào trong CSDL
Kịch bản chính	1. Học viên truy cập vào trang cá nhân. 2. Hệ thống hiển thị thông tin cá nhân hiện tại của học viên. 3. Học viên chọn chức năng “Sửa thông tin cá nhân”. 4. Hệ thống hiển thị biểu mẫu chỉnh sửa thông tin. 5. Học viên cập nhật các thông tin cần thay đổi và chọn “Lưu” để xác nhận cập nhật 6. Hệ thống kiểm tra tính hợp lệ của dữ liệu nhập vào, cập nhật thông tin mới vào cơ sở dữ liệu và thông báo cập nhật thông tin cá nhân thành công cho học viên.
Ngoại lệ	6.1 Nếu thông tin nhập vào không hợp lệ, hệ thống hiển thị thông báo lỗi và yêu cầu học viên nhập lại. 6.2 Nếu xảy ra lỗi trong quá trình lưu dữ liệu vào cơ sở dữ liệu, hệ thống thông báo cập nhật thất bại và giữ nguyên thông tin cũ.

Bảng 2.4. Kích bản chức năng sửa thông tin cá nhân

- Xem danh sách bài học



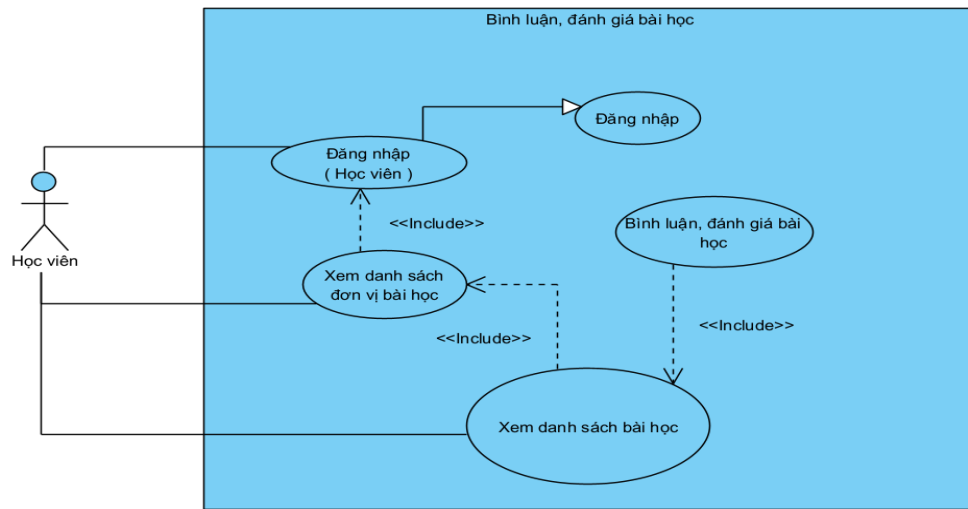
Hình 2.7. Biểu đồ xem danh sách bài học

Kịch bản xem danh sách bài học

Tên use case	Xem danh sách bài học
Tác nhân	Học viên
Điều kiện	Học viên đã đăng nhập thành công vào hệ thống
Đảm bảo thành công	Hệ thống hiển thị danh sách các bài học hiện có để học viên có thể xem và lựa chọn.
Kịch bản chính	1. Học viên chọn một khóa học từ danh sách khóa học. 2. Hệ thống hiển thị danh sách các level thuộc khóa học đã chọn. 3. Học viên chọn một level. 4. Hệ thống truy xuất danh sách các đơn vị bài học trong CSDL và hiển thị danh sách các đơn vị bài học thuộc level đã chọn. 5. Học viên chọn một đơn vị bài học. 6. Hệ thống truy xuất danh sách các bài học trong đơn vị bài học đã chọn và hiển thị danh sách các bài học thuộc đơn vị bài học đã chọn.
Ngoại lệ	4.1 Nếu hệ thống không tìm thấy đơn vị bài học nào, hệ thống hiển thị thông báo “Chưa có đơn vị bài học nào” cho học viên. 6.1 Nếu hệ thống không tìm thấy bài học nào, hệ thống hiển thị thông báo “Chưa có bài học nào” cho học viên.

Bảng 2.5. Kịch bản chức năng xem danh sách bài học

- Bình luận, đánh giá bài học



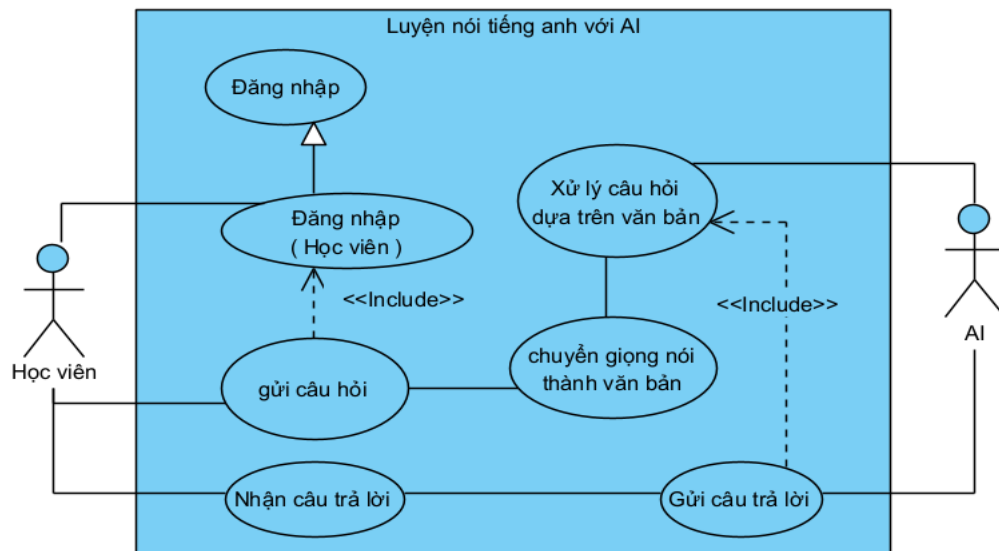
Hình 2.8. Biểu đồ bình luận và đánh giá bài học

Kịch bản bình luận, đánh giá bài học

Tên use case	Bình luận, đánh giá bài học
Tác nhân	Học viên
Điều kiện	Học viên đã đăng nhập thành công vào hệ thống và đang xem nội dung của một bài học cụ thể.
Đảm bảo thành công	Bình luận và đánh giá của học viên được lưu trữ thành công trong hệ thống và hiển thị cho các học viên khác.
Kịch bản chính	1. Học viên truy cập vào một bài học 2. Hệ thống hiển thị nội dung bài học cùng khu vực bình luận và đánh giá. 3. Học viên nhập nội dung bình luận và chọn mức đánh giá (số sao), sau đó chọn chức năng Gửi bình luận. 4. Hệ thống kiểm tra tính hợp lệ của nội dung bình luận/ mức đánh giá và lưu bình luận/ đánh giá vào cơ sở dữ liệu, sau đó hiển thị bình luận/ đánh giá vừa được gửi kèm thông báo thành công.
Ngoại lệ	3.1 Nếu học viên chưa chọn mức đánh giá, hệ thống yêu cầu học viên chọn mức đánh giá trước khi gửi. 4.1 Nếu xảy ra lỗi trong quá trình lưu dữ liệu, hệ thống thông báo gửi bình luận/đánh giá thất bại và yêu cầu học viên thử lại sau.

Bảng 2.6. Kịch bản chức năng bình luận và đánh giá bài học

- Luyện nói tiếng anh với AI



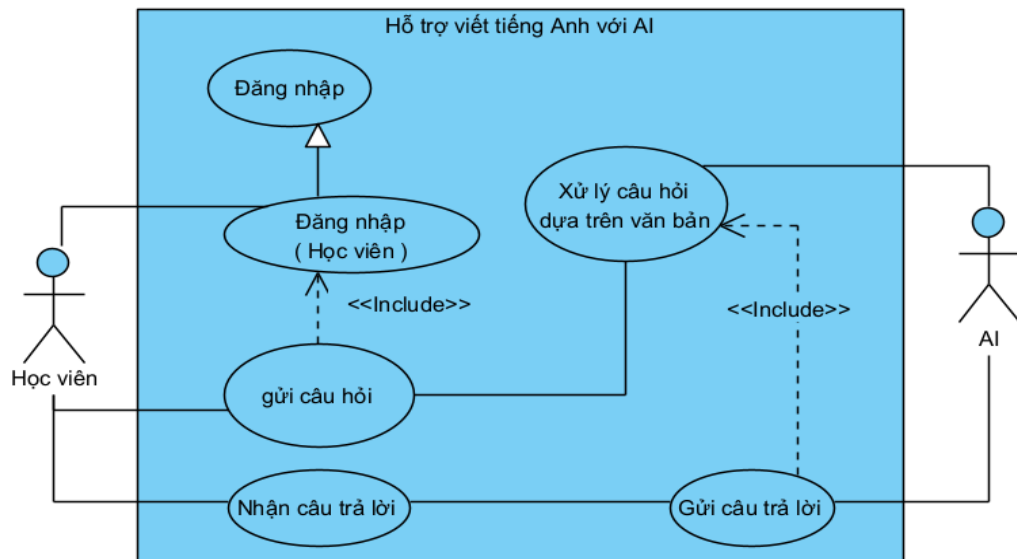
Hình 2.9. Biểu đồ luyện nói tiếng anh với AI

Kịch bản luyện nói tiếng anh với AI

Tên use case	Luyện nói tiếng anh với AI
Tác nhân	Học viên
Điều kiện	Học viên đã đăng nhập thành công vào hệ thống và thiết bị của học viên có micro hoạt động và đã cấp quyền truy cập micro cho hệ thống.
Đảm bảo thành công	Học viên hoàn thành một phiên luyện nói với AI.
Kịch bản chính	1. Học viên truy cập chức năng SpeakUp. 2. Hệ thống hiển thị giao diện luyện nói và hướng dẫn. 3. Học viên chọn chủ đề và bắt đầu nói tiếng Anh. 4. Hệ thống xử lý giọng nói, phân tích nội dung và phản hồi bằng chatbot AI. 5. Học viên nhận phản hồi và kết thúc phiên luyện nói.
Ngoại lệ	3.1 Nếu học viên chưa cấp quyền truy cập micro, hệ thống hiển thị thông báo yêu cầu cấp quyền. 4.1 Nếu xảy ra lỗi trong quá trình xử lý AI, hệ thống thông báo lỗi và yêu cầu học viên thử lại sau

Bảng 2.7. Kịch bản chức năng luyện nói tiếng anh với AI

- Hỗ trợ viết tiếng Anh với AI



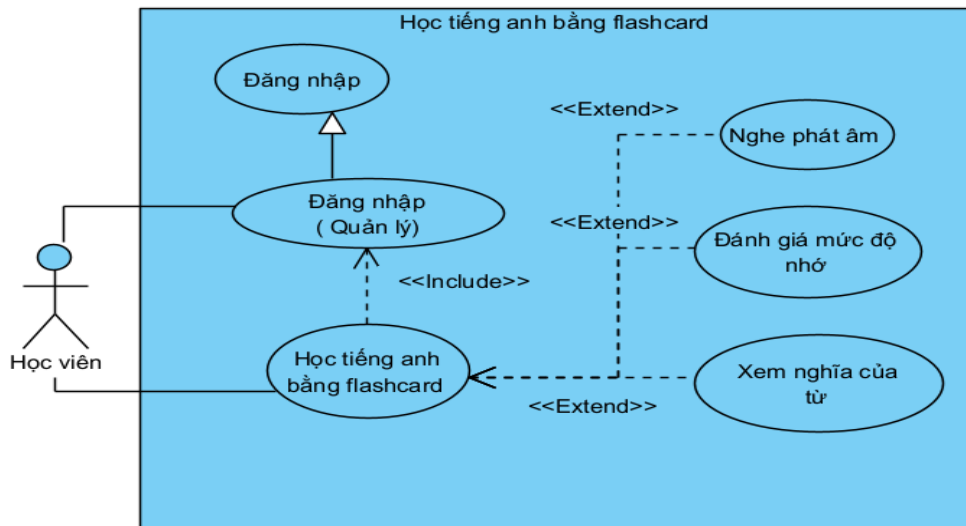
Hình 2.10. Biểu đồ hỗ trợ viết tiếng Anh với AI

Kịch bản hỗ trợ viết tiếng Anh với AI

Tên use case	Hỗ trợ viết tiếng Anh với AI
Tác nhân	Học viên
Điều kiện	Học viên đã đăng nhập thành công vào hệ thống và đang truy cập chức năng Chatbot.
Đảm bảo thành công	Học viên nhận được phản hồi từ AI về ngữ pháp và cách diễn đạt.
Kịch bản chính	1. Học viên truy cập vào chức năng "Chatbot". 2. Hệ thống hiển thị giao diện chatbot 3. Học viên nhập nội dung văn bản tiếng Anh cần hỏi. 4. Hệ thống tự động phân tích nội dung văn bản bằng AI. 5. Hệ thống phát hiện các lỗi ngữ pháp, cấu trúc câu và đưa ra các gợi ý sửa lỗi và viết lại câu tự nhiên hơn.
Ngoại lệ	4.1 Nếu xảy ra lỗi trong quá trình phân tích hoặc sinh gợi ý từ AI, hệ thống thông báo lỗi 5.1 Nếu xảy ra lỗi kết nối, hệ thống thông báo lỗi và yêu cầu học viên thử lại sau.

Bảng 2.8. Kịch bản chức năng hỗ trợ viết tiếng anh với AI

- Học tiếng anh bằng flashcard



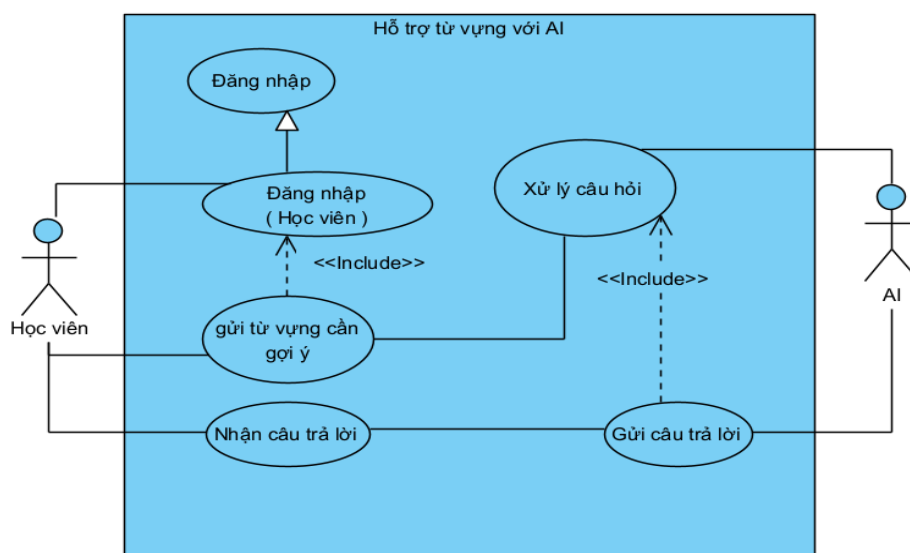
Hình 2.11. Biểu đồ học tiếng anh bằng flashcard

Kịch bản học tiếng anh bằng flashcard

Tên use case	Học tiếng anh bằng flashcard
Tác nhân	Học viên
Điều kiện	Học viên đã đăng nhập thành công vào hệ thống.
Đảm bảo thành công	Học viên hoàn thành phiên học flashcard và hệ thống ghi nhận kết quả học tập của học viên.
Kịch bản chính	1. Học viên truy cập vào chức năng Flashcard của hệ thống. 2. Hệ thống hiển thị danh sách các bộ flashcard theo khóa học hoặc chủ đề. 3. Học viên chọn một bộ flashcard để học. 4. Hệ thống hiển thị flashcard đầu tiên với từ vựng hoặc câu hỏi. 5. Học viên xem nội dung flashcard sau đó lật flashcard để xem đáp án hoặc nghĩa của từ. Đánh giá mức độ nhớ của từ lặp lại đến khi kết thúc bộ. 6. Hệ thống ghi nhận kết quả và hiển thị kết quả học tập của học viên.
Ngoại lệ	4.1 Nếu xảy ra lỗi khi tải dữ liệu flashcard, hệ thống thông báo lỗi và yêu cầu học viên thử lại sau.

Bảng 2.9. Kịch bản chức năng học tiếng anh bằng flashcard

- Hỗ trợ từ vựng với AI



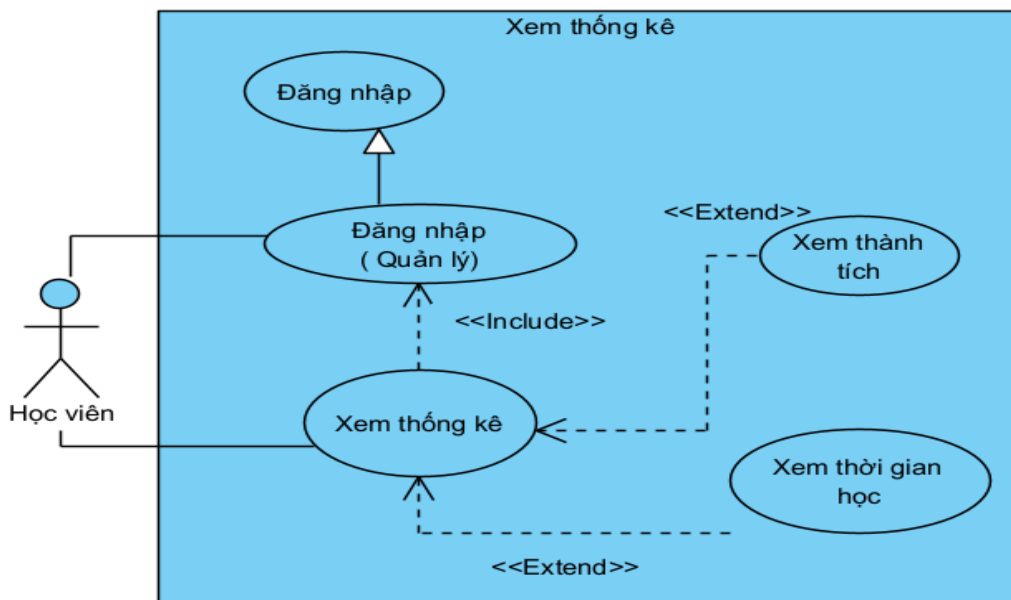
Hình 2.12. Biểu đồ hỗ trợ từ vựng với AI

Kịch bản hỗ trợ từ vựng với AI

Tên use case	Hỗ trợ từ vựng với AI
Tác nhân	Học viên
Điều kiện	Học viên đã đăng nhập vào hệ thống và đang truy cập chức năng viết tiếng Anh.
Đảm bảo thành công	Học viên nhận được phản hồi từ AI về từ vựng đồng thời có thể chỉnh sửa từ vựng theo các gợi ý của hệ thống.
Kịch bản chính	1. Học viên truy cập vào chức năng Viết tiếng Anh của hệ thống. 2. Hệ thống hiển thị giao diện soạn thảo văn bản. 3. Học viên nhập nội dung văn bản tiếng Anh và dùng con trỏ tại từ hoặc cụm từ cần phân tích. 4. Hệ thống tự động phân tích từ hoặc cụm từ được chọn bằng AI và hiển thị danh sách các từ hoặc cách diễn đạt gợi ý phù hợp ngữ cảnh. 5. Học viên lựa chọn áp dụng hoặc bỏ qua gợi ý sau đó xác nhận hoàn thành bài viết. 6. Hệ thống lưu bài viết và thông báo hoàn tất cho học viên.
Ngoại lệ	4.1 nếu xảy ra lỗi kết nối, thông báo lỗi và cho phép học viên tiếp tục viết mà không có hỗ trợ từ AI.

Bảng 2.10. Kịch bản chức năng hỗ trợ từ vựng với AI

- Xem thống kê



Hình 2.13. Biểu đồ xem thống kê

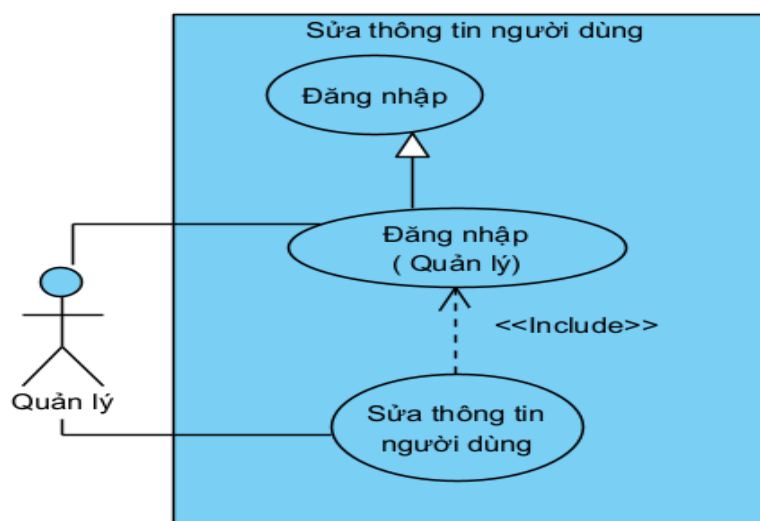
Kịch bản xem thống kê

Tên use case	Xem thống kê
Tác nhân	Học viên
Điều kiện	Học viên đã đăng nhập thành công vào hệ thống
Đảm bảo thành công	Hệ thống hiển thị đầy đủ và chính xác các thống kê học tập của học viên
Kịch bản chính	1. Học viên truy cập vào Dashboard của hệ thống. 2. Hệ thống tiếp nhận yêu cầu và truy xuất dữ liệu thống kê học tập của học viên. 3. Học viên chờ hệ thống xử lý và yêu cầu hiển thị thông tin tổng quan. 4. Hệ thống hiển thị các chỉ số học tập tổng quan như: số khóa học đã tham gia, tiến độ học tập, số bài đã hoàn thành. 5. Học viên lựa chọn xem chi tiết các thống kê học tập. 6. Hệ thống hiển thị các biểu đồ và số liệu chi tiết phục vụ phân tích. 7. Học viên theo dõi và đánh giá kết quả học tập của bản thân.
Ngoại lệ	3.1 Lỗi hệ thống khi tải dữ liệu, hệ thống thông báo và yêu cầu thử lại sau

Bảng 2.11. Kịch bản chức năng xem thống kê

2.3.5.2. Chức năng của admin

- Sửa thông tin người dùng



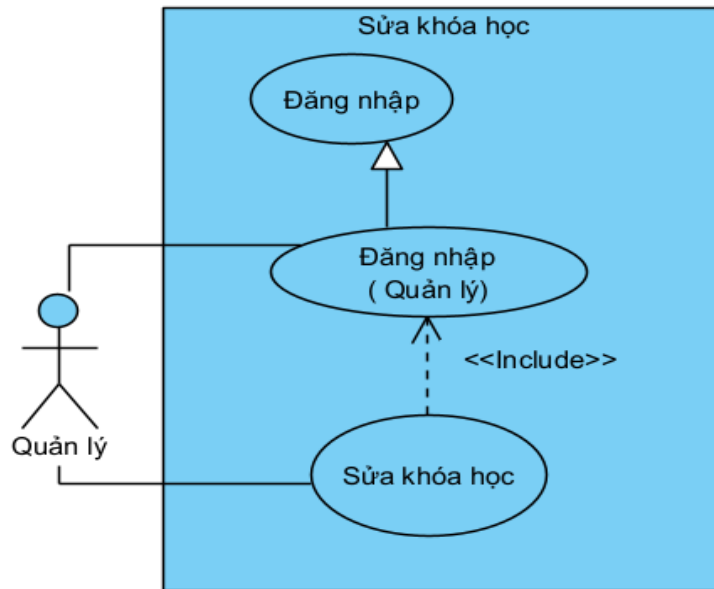
Hình 2.14. Biểu đồ sửa thông tin người dùng

Kịch bản sửa thông tin người dùng

Tên use case	Sửa thông tin người dùng
Tác nhân	Quản lý
Điều kiện	Quản lý đã đăng nhập thành công vào hệ thống với quyền quản trị
Đảm bảo thành công	Thông tin user được cập nhật chính xác trong cơ sở dữ liệu
Kịch bản chính	1. Quản lý truy cập vào chức năng Quản lý user trên hệ thống 2. Hệ thống hiển thị danh sách user hiện có kèm các tùy chọn thao tác (thêm, sửa, xóa, xem chi tiết) 3. Quản lý chọn user cần chỉnh sửa, cập nhật thông tin và lưu 4. Hệ thống kiểm tra dữ liệu nhập hợp lệ và thực hiện thao tác tương ứng sau đó thông báo thành công và cập nhật danh sách user
Ngoại lệ	3.1 Thông tin nhập không hợp lệ, hệ thống yêu cầu nhập lại 4.1 Lỗi hệ thống khi thao tác cơ sở dữ liệu, thông báo thất bại và yêu cầu thử lại

Bảng 2.12. Kịch bản chức năng quản lý sửa thông tin người dùng

- Sửa khóa học



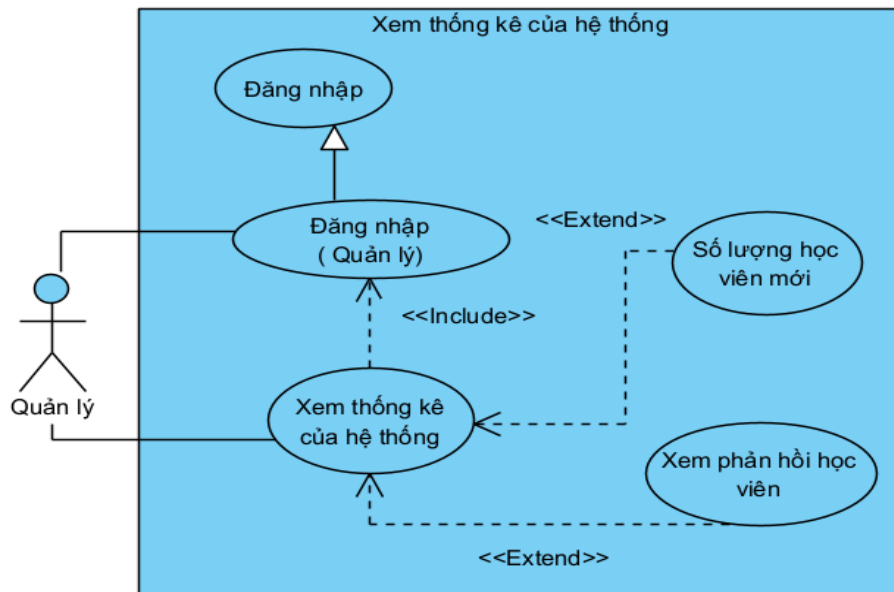
Hình 2.15. Biểu đồ sửa khóa học

Kịch bản sửa khóa học

Tên use case	Sửa khóa học
Tác nhân	Quản lý
Điều kiện	Quản lý đã đăng nhập thành công vào hệ thống với quyền quản trị
Đảm bảo thành công	Thông tin khóa học được cập nhật chính xác trong cơ sở dữ liệu
Kịch bản chính	1. Quản lý truy cập chức năng Quản lý khóa học trên hệ thống 2. Hệ thống hiển thị danh sách các khóa học hiện có 3. Quản lý chọn khóa học cần chỉnh sửa 4. Hệ thống hiển thị biểu mẫu thông tin khóa học hiện tại 5. Quản lý chỉnh sửa các thông tin cần thay đổi (tên khóa học, mô tả, level, số bài học...) và ấn nút “Lưu” 6. Hệ thống kiểm tra tính hợp lệ của dữ liệu nhập vào và cập nhật thông tin khóa học trong cơ sở dữ liệu 7. Quản lý nhận được thông báo thao tác thành công và hiển thị thông tin khóa học đã cập nhật
Ngoại lệ	6.1 Khóa học không tồn tại, hệ thống thông báo lỗi

Bảng 2.13. Kịch bản chức năng quản lý sửa khóa học

- Xem thống kê của hệ thống



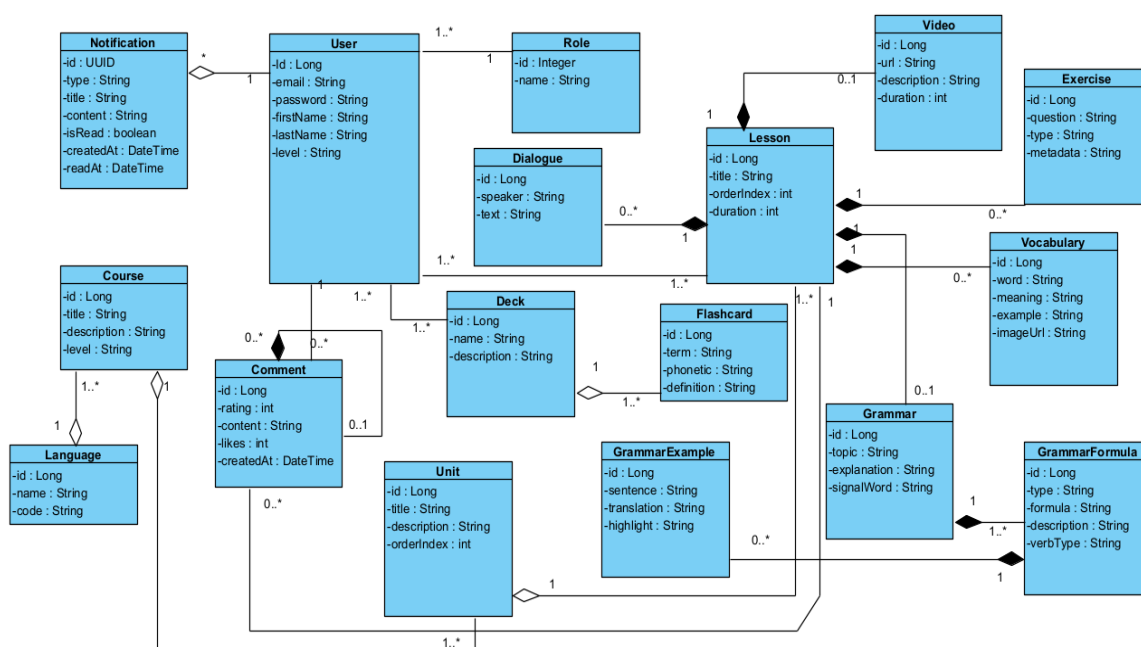
Hình 2.16. Biểu đồ xem thống kê của hệ thống

Kịch bản xem thống kê của hệ thống

Tên use case	Xem thống kê của hệ thống
Tác nhân	Quản lý
Điều kiện	Quản lý đã đăng nhập thành công vào hệ thống với quyền quản trị
Đảm bảo thành công	Hệ thống hiển thị đầy đủ các thống kê liên quan đến hoạt động của học viên và khóa học
Kịch bản chính	1. Quản lý truy cập vào chức năng Dashboard của hệ thống. 2. Hệ thống kiểm tra quyền truy cập và xác thực người dùng đối với chức năng Dashboard. 3. Quản lý xác nhận yêu cầu xem các thống kê tổng quan. 4. Hệ thống truy xuất dữ liệu thống kê liên quan từ cơ sở dữ liệu. 5. Quản lý lựa chọn loại thống kê cần theo dõi. 6. Hệ thống hiển thị các biểu đồ và số liệu chi tiết tương ứng với lựa chọn của quản lý.
Ngoại lệ	4.1 Lỗi truy xuất dữ liệu, hệ thống thông báo lỗi và yêu cầu thử lại

Bảng 2.14. Kịch bản chức năng quản lý xem thống kê hệ thống

2.3.6. Biểu đồ lớp phân tích



Hình 2.17. Biểu đồ lớp phân tích

2.3.6.1. Trích xuất danh từ

Danh từ trích xuất	Lớp được hình thành	Lập luận	Thuộc tính chính
Người dùng	User	Đối tượng tham gia vào toàn bộ quá trình học tập	email, mật khẩu, họ, tên, trình độ
Vai trò	Role	Quyền hạn và chức năng của người dùng	mã, tên chức năng
Khóa học	Course	Đơn vị nội dung học tập chính	tiêu đề, mô tả, cấp độ học
Đơn vị bài học	Unit	Thành phần cấu tạo của khóa học	tiêu đề, mô tả
Bài học	Lesson	Đơn vị học tập chi tiết	tiêu đề, mô tả
Thẻ học	Flashcard	Công cụ hỗ trợ ghi	từ, phát âm, nghĩa

		nhớ kiến thức	
Từ vựng	Vocabulary	Nội dung học tập	từ, nghĩa, ví dụ, ảnh minh họa
Ngữ pháp	Grammar	Nội dung học tập	chủ đề, mô tả, dấu hiệu nhận biết
Video	Video	Nội dung đa phương tiện phục vụ học tập	mô tả, thời lượng video
Đoạn hội thoại	Dialogue	Nội dung học tập	người nói, đoạn hội thoại
Bài tập	Exercise	Nội dung dùng để luyện tập	câu hỏi, dạng câu hỏi, dữ liệu mở rộng
Ngôn ngữ	Language	Ngôn ngữ của bài học	tên ngôn ngữ, kí hiệu
Thông báo	Notification	Thông báo cho người dùng	tiêu đề, nội dung
Bình luận	Comment	Nhận xét của người dùng	đánh giá, nội dung
Bộ thẻ học	Deck	Tập hợp bộ các flashcard để học tập	tên, mô tả
Công thức ngữ pháp	GrammarFormula	Cấu trúc / công thức hình thành ngữ pháp	công thức, mô tả, loại công thức(chủ động, ..), loại động từ (tobe, ..)
Ví dụ ngữ pháp	GrammarExample	Các câu ví dụ minh họa công thức ngữ pháp	câu ví dụ, câu dịch nghĩa, trọng tâm của câu

Bảng 2.15. Bảng trích xuất danh từ

2.3.6.2 Xác định quan hệ số lượng giữa các thực thể

- Một người dùng có thể học nhiều bài học, đồng thời một bài học cũng có thể được nhiều người dùng tham gia học. Do đó, giữa hai thực thể User và Lesson tồn tại mối quan hệ nhiều – nhiều (N – N).

- Một người dùng có thể nhận được nhiều thông báo, trong khi mỗi thông báo chỉ thuộc về một người dùng. Vì vậy, giữa User và Notification tồn tại mối quan hệ một – nhiều (1 – *).

- Mỗi người dùng được gán một quyền hạn xác định, và mỗi quyền hạn có thể được gán cho nhiều người dùng khác nhau. Do đó, giữa User và Role tồn tại mối quan hệ nhiều – một (* – 1).

- Một người dùng có thể học nhiều bộ thẻ, đồng thời một bộ thẻ cũng có thể được nhiều người dùng học. Vì vậy, giữa User và Deck tồn tại mối quan hệ nhiều – nhiều (N – N)

- Một người dùng có thể tạo nhiều bình luận, trong khi mỗi bình luận chỉ do một người dùng tạo ra. Do đó, giữa User và Comment tồn tại mối quan hệ một – nhiều (1 – *).

- Mỗi khóa học được giảng dạy theo một ngôn ngữ cụ thể, trong khi một ngôn ngữ có thể được sử dụng cho nhiều khóa học khác nhau. Vì vậy, giữa Language và Course tồn tại mối quan hệ một – nhiều (1 – *).

- Mỗi khóa học được chia thành nhiều đơn vị bài học, và mỗi đơn vị bài học chỉ thuộc về một khóa học. Do đó, giữa Course và Unit tồn tại mối quan hệ một – nhiều (1 – *).

- Mỗi đơn vị bài học bao gồm nhiều bài học, trong khi mỗi bài học chỉ thuộc về một đơn vị bài học. Vì vậy, giữa Unit và Lesson tồn tại mối quan hệ một – nhiều (1 – *).

- Mỗi bình luận có thể nhận được nhiều bình luận phản hồi, và mỗi bình luận phản hồi chỉ thuộc về một bình luận cha. Do đó, giữa Comment và Comment (reply) tồn tại mối quan hệ một – nhiều (1 – *).

- Mỗi bộ thẻ học bao gồm nhiều thẻ học, và mỗi thẻ học chỉ thuộc về một bộ thẻ. Vì vậy, giữa Deck và Flashcard tồn tại mối quan hệ một – nhiều (1 – *).

- Mỗi bài học có thể không có hoặc chỉ có tối đa một video học bài. Do đó, giữa

Lesson và Video tồn tại mỗi quan hệ một – không hoặc một (1 – 0..1).

- Mỗi bài học có thể không có hoặc có nhiều đoạn hội thoại phục vụ cho kỹ năng nghe – nói. Vì vậy, giữa Lesson và Dialogue tồn tại mỗi quan hệ một – nhiều (1 – 0..*).

- Mỗi bài học có thể không có hoặc chỉ có tối đa một chủ đề ngữ pháp. Do đó, giữa Lesson và Grammar tồn tại mỗi quan hệ một – không hoặc một (1 – 0..1).

- Mỗi bài học có thể không có hoặc có nhiều từ vựng. Vì vậy, giữa Lesson và Vocabulary tồn tại mỗi quan hệ một – nhiều (1 – 0..*).

- Mỗi bài học có thể không có hoặc có nhiều bài tập để thực hành. Do đó, giữa Lesson và Exercise tồn tại mỗi quan hệ một – nhiều (1 – 0..*).

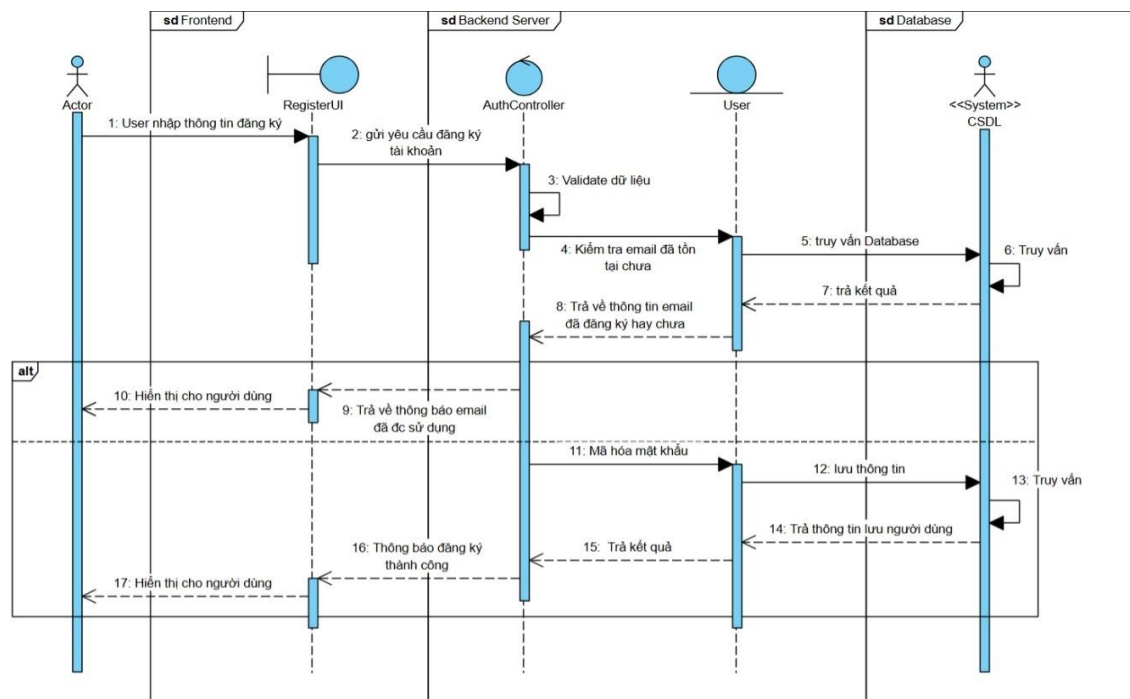
- Mỗi chủ đề ngữ pháp bao gồm nhiều công thức ngữ pháp. Vì vậy, giữa Grammar và GrammarFormula tồn tại mỗi quan hệ một – nhiều (1 – 1..*).

- Mỗi công thức ngữ pháp có thể không có hoặc có nhiều ví dụ ngữ pháp minh họa cho công thức đó. Do đó, giữa GrammarFormula và GrammarExample tồn tại mỗi quan hệ một – nhiều (1 – 0..*).

2.3.7. Biểu đồ tuần tự

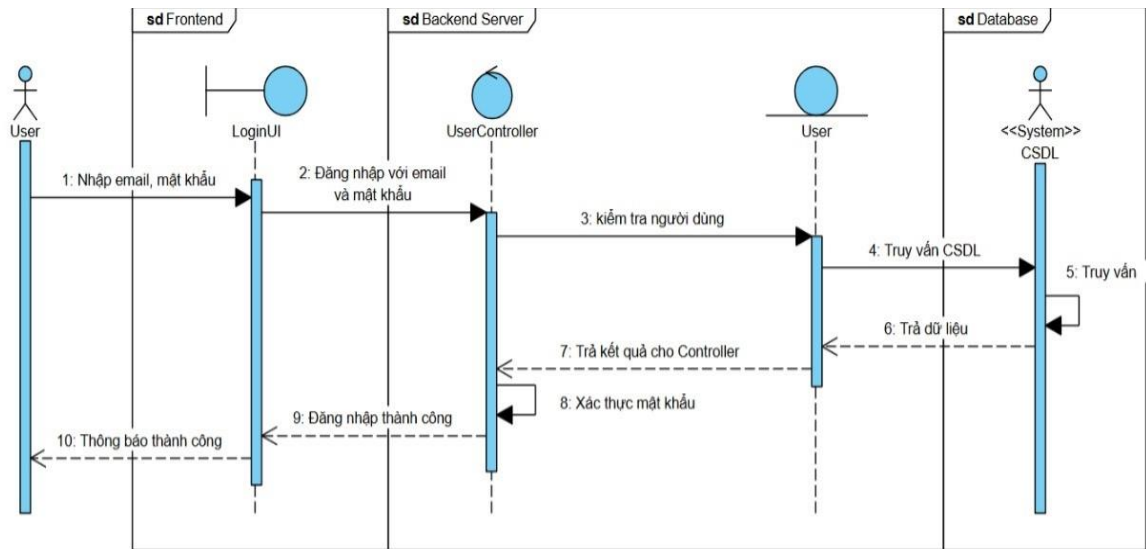
2.3.7.1. Biểu đồ tuần tự các chức năng của người dùng

- Người dùng đăng ký tài khoản mới



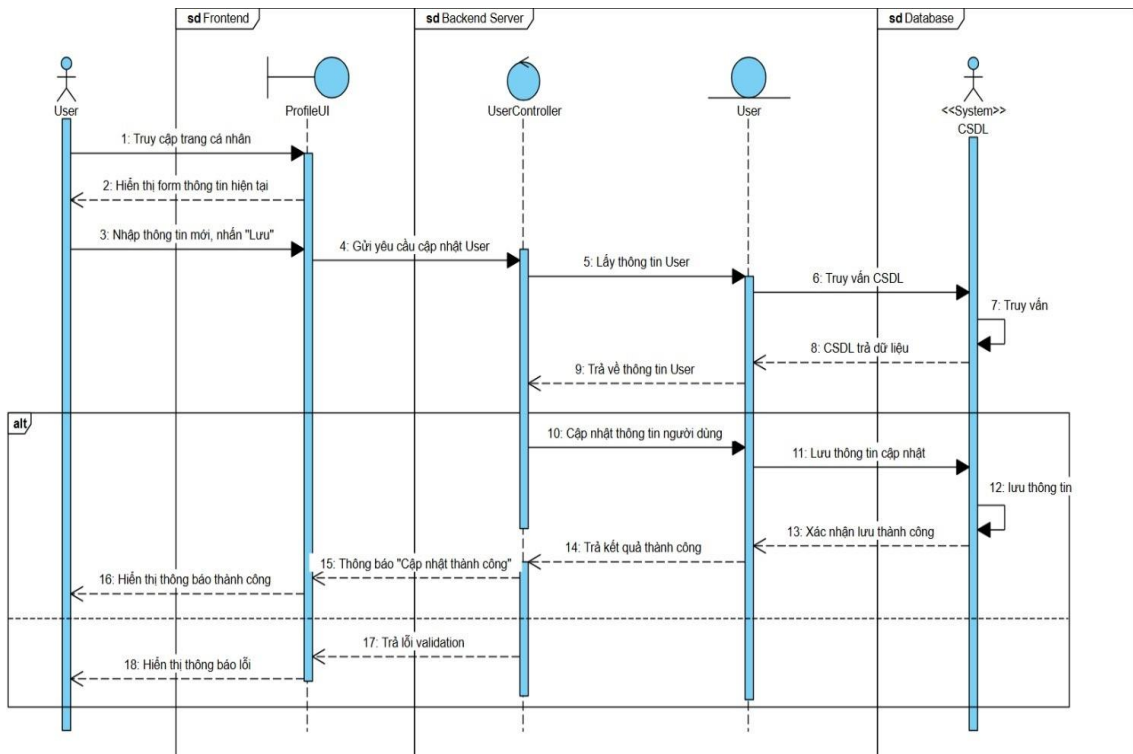
Hình 2.18. Biểu đồ tuần tự chức năng người dùng đăng ký hệ thống

- Người dùng đăng nhập hệ thống



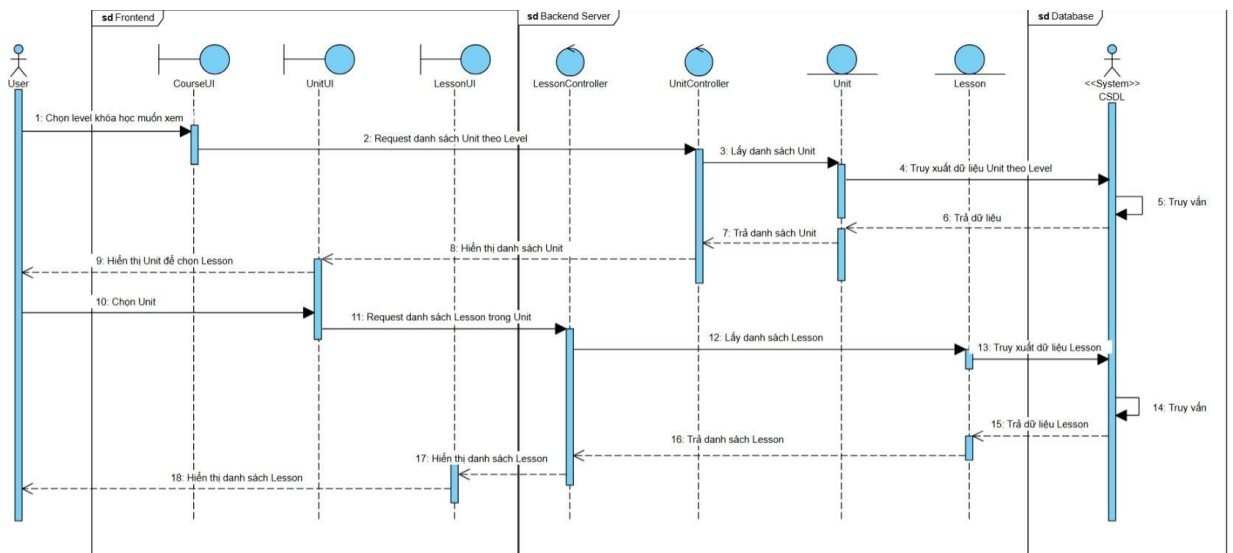
Hình 2.19. Biểu đồ tuần tự chức năng người dùng đăng nhập hệ thống

- Sửa thông tin cá nhân



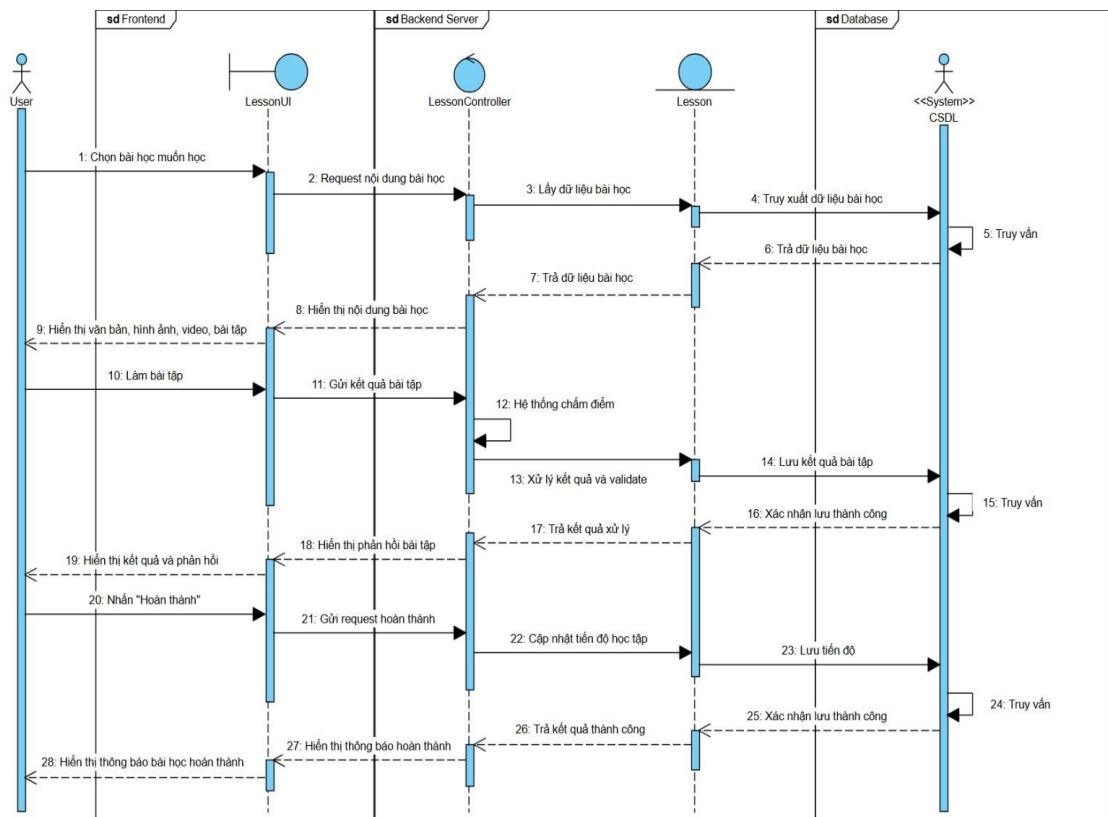
Hình 2.20. Biểu đồ tuần tự của sửa thông tin cá nhân

- Xem danh sách bài học



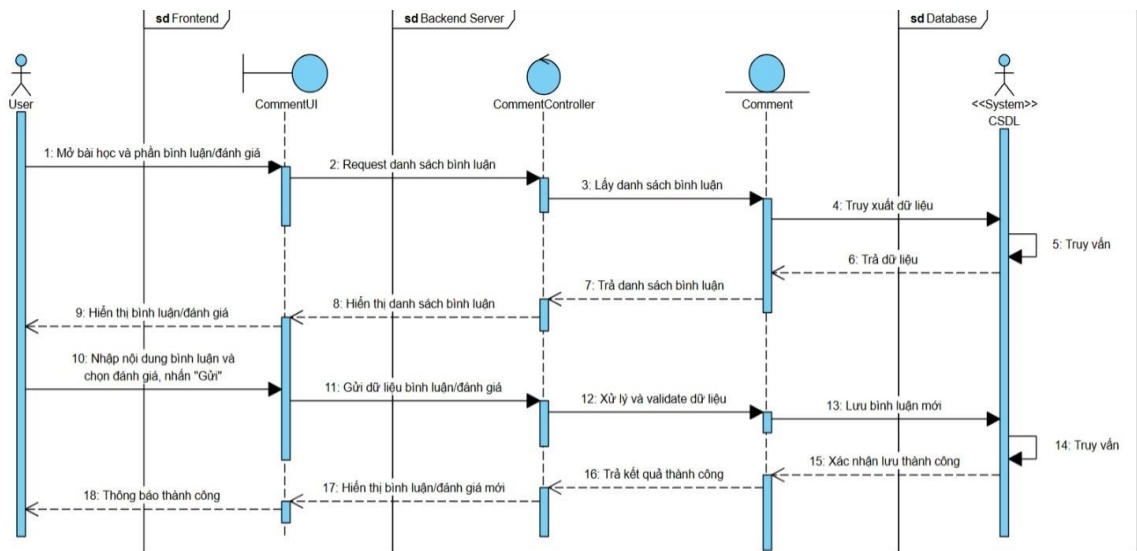
Hình 2.21. Biểu đồ tuần tự của xem danh sách bài học

- Học và luyện tập bài học



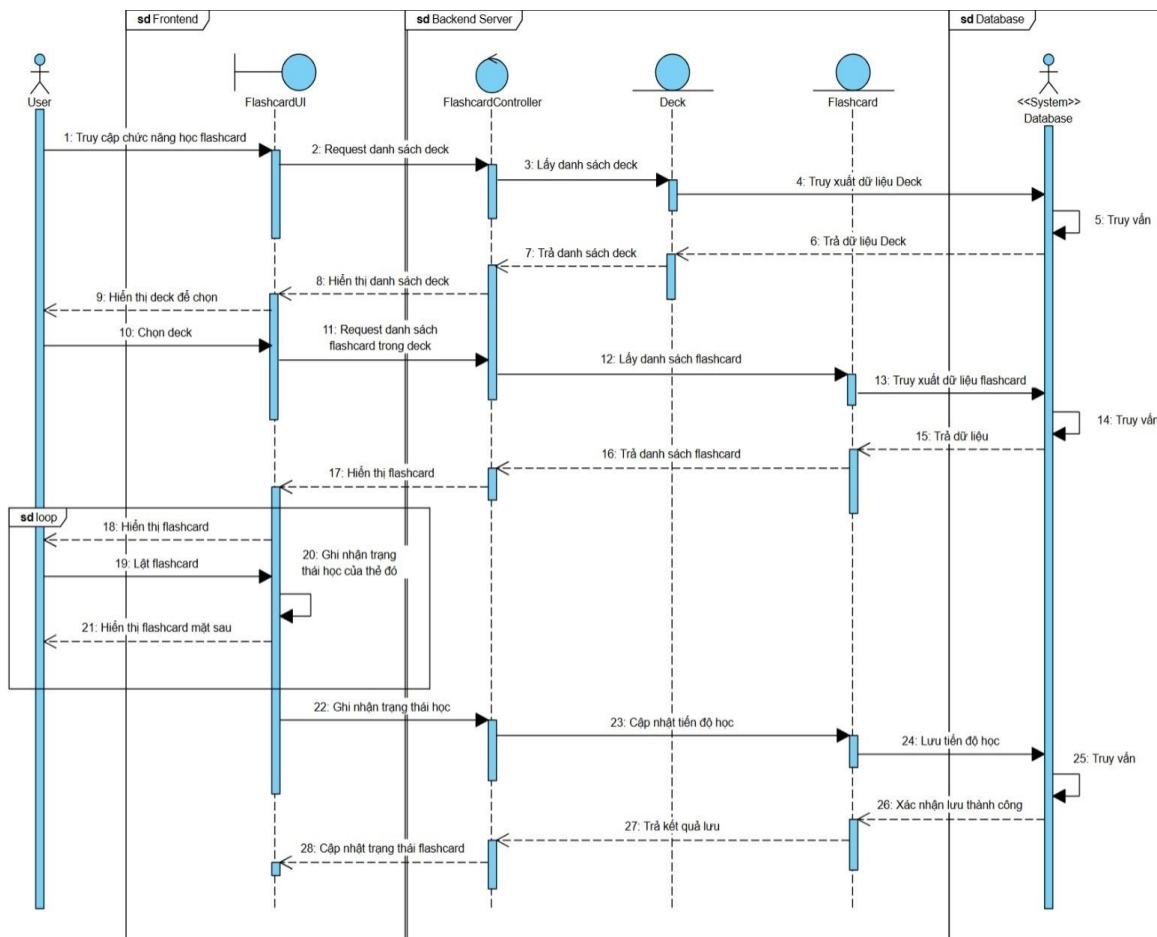
Hình 2.22. Biểu đồ tuần tự của học và luyện tập bài học

● Bình luận, đánh giá bài học



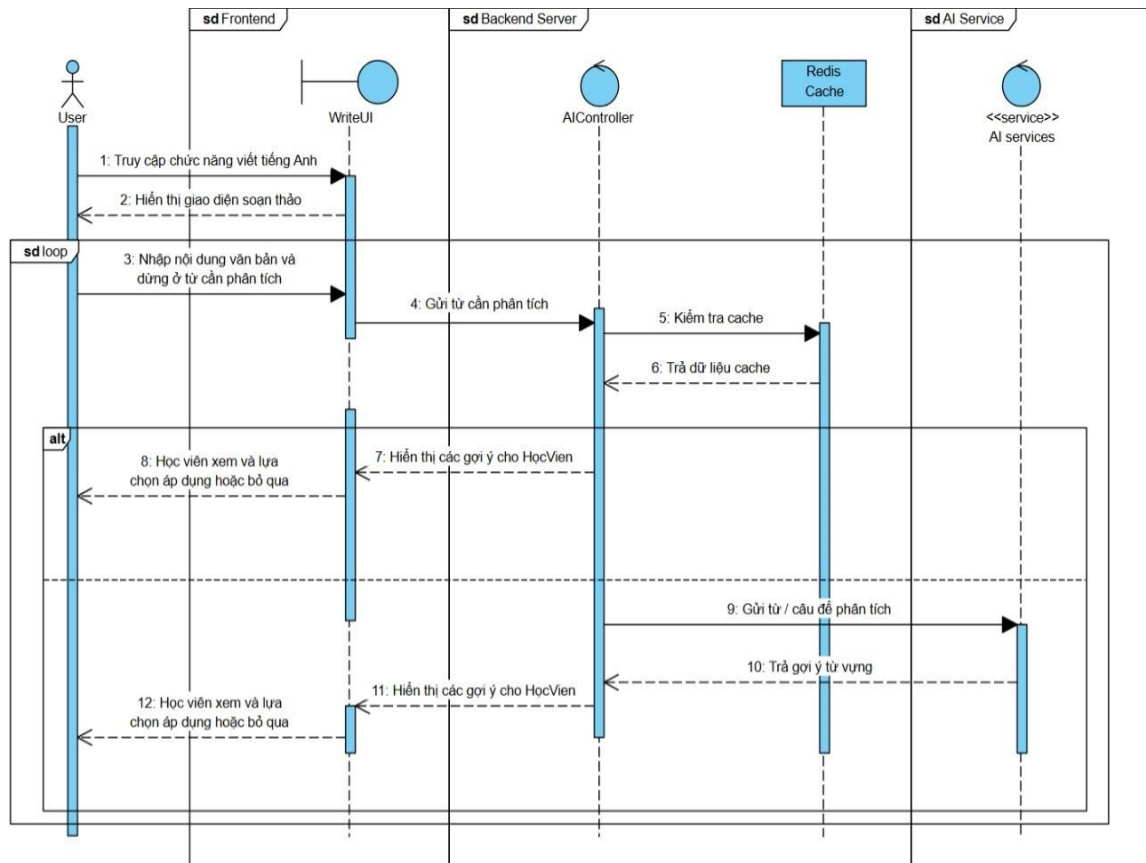
Hình 2.23. Biểu đồ tuần tự của bình luận và đánh giá bài học

● Học tiếng anh bằng flashcard



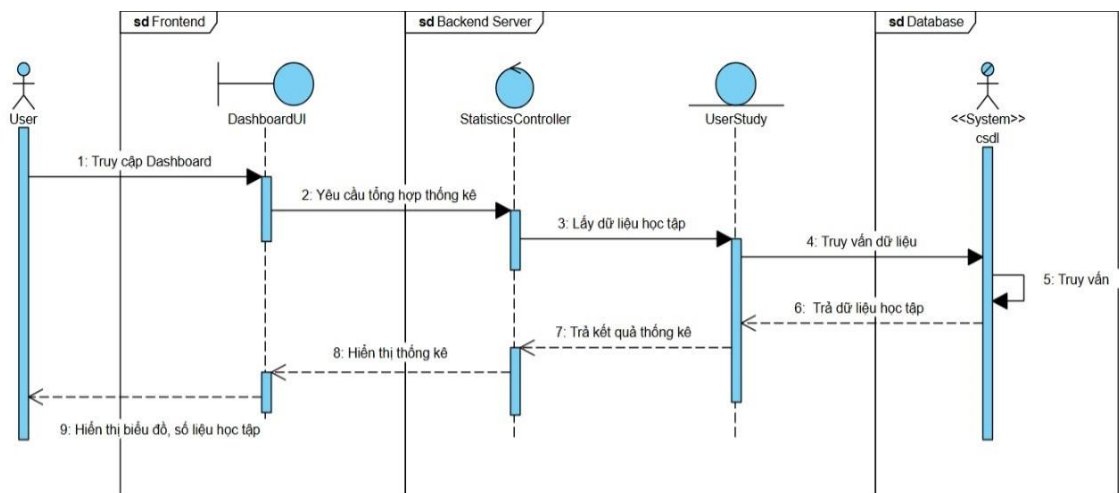
Hình 2.24. Biểu đồ tuần tự của học tiếng anh bằng flashcard

- Hỗ trợ từ vựng với AI



Hình 2.25. Biểu đồ tuần tự của hỗ trợ từ vựng với AI

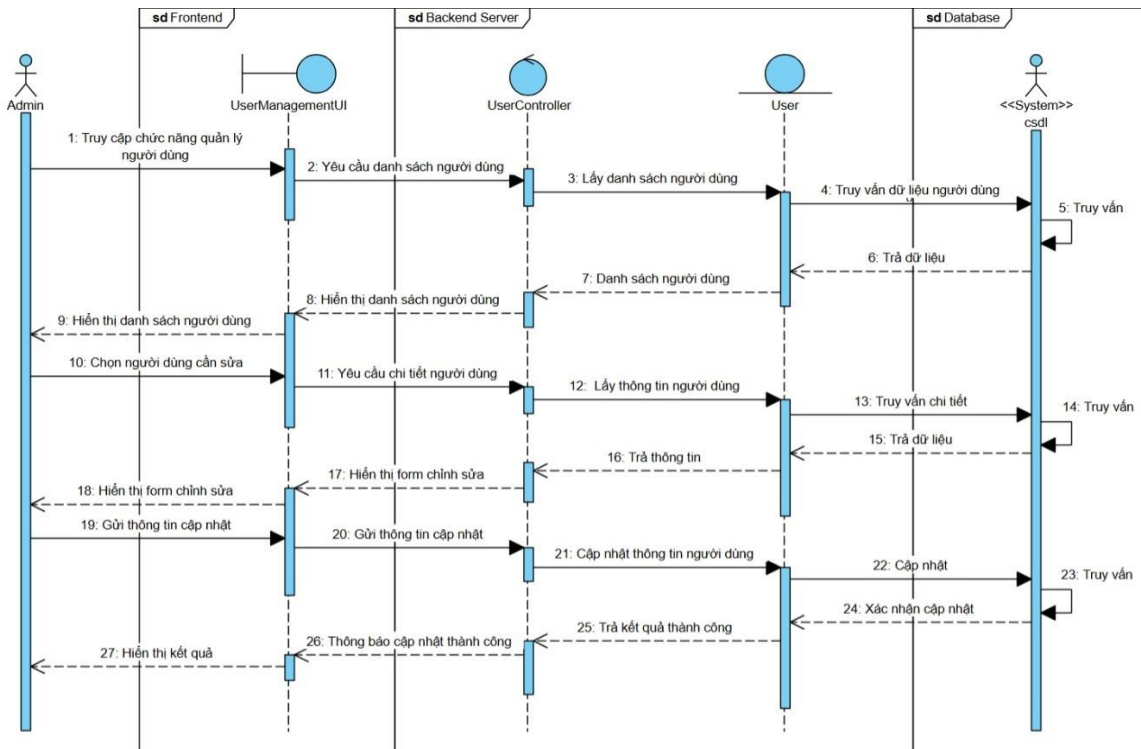
- Xem thống kê



Hình 2.26. Biểu đồ tuần tự của xem thống kê

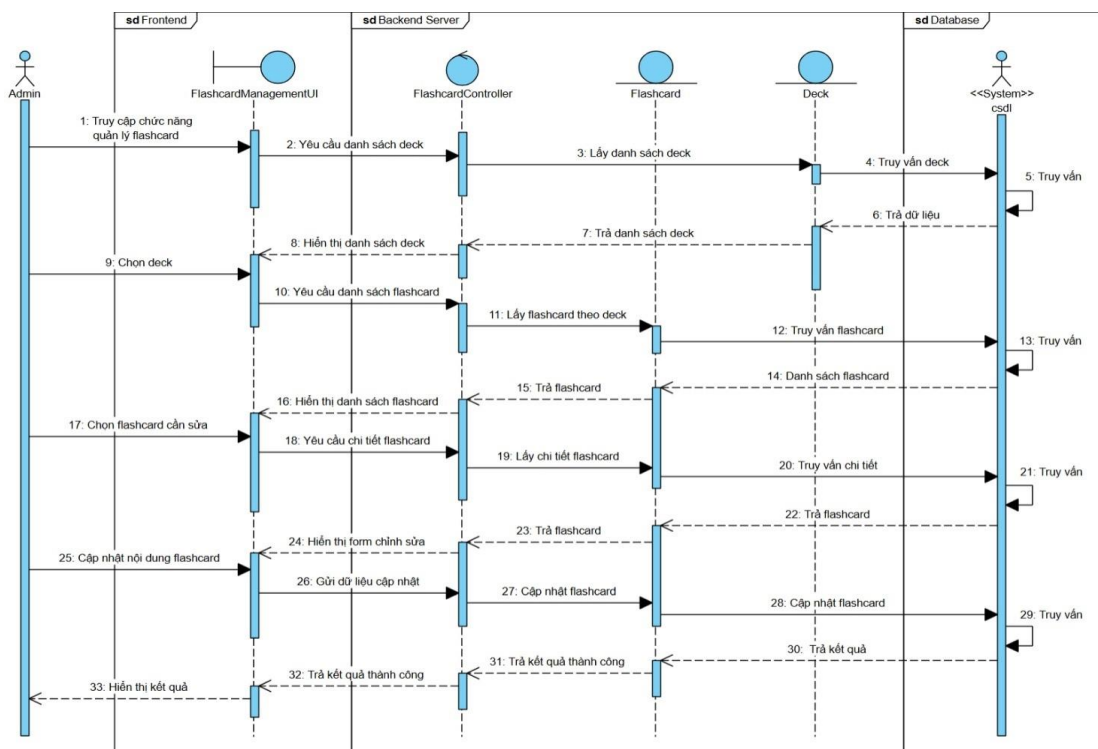
2.3.7.2. Biểu đồ tuần tự các chức năng của quản trị viên

● Sửa thông tin người dùng



Hình 2.27. Biểu đồ tuần tự của quản lý sửa thông tin người dùng

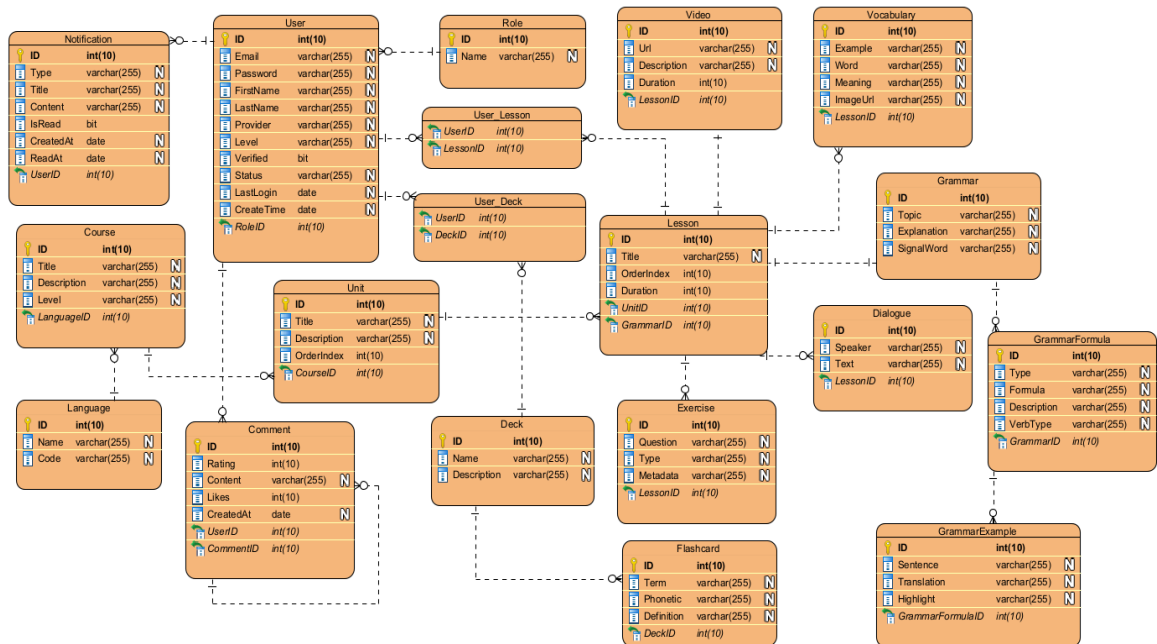
● Sửa flashcard



Hình 2.28. Biểu đồ tuần tự của quản lý sửa flashcard

2.5. Cơ sở dữ liệu

2.5.1. Lược đồ cơ sở dữ liệu



Hình 2.30. Lược đồ cơ sở dữ liệu

2.5.2. Xây dựng và mô tả các bảng trong cơ sở dữ liệu.

● Bảng Role

STT	Tên cột	Kiểu dữ liệu	Mô tả
1	id	Integer	PK - Mã vai trò
2	name	Varchar(255)	Tên vai trò

Bảng 2.16. Bảng Role

● Bảng User

STT	Tên cột	Kiểu dữ liệu	Mô tả
1	id	Integer	PK - Mã người dùng
2	email	Varchar(255)	Email đăng nhập (duy nhất)
3	password	Varchar(255)	Mật khẩu
4	firstName	Varchar(50)	Tên

5	lastName	Varchar(50)	Họ
6	provider	Varchar(50)	Nhà cung cấp đăng nhập
7	level	Varchar(50)	Trình độ người học
8	verified	Boolean	Trạng thái xác thực
9	lastLogin	Timestamp	Thời điểm đăng nhập gần nhất
10	status	Varchar(20)	Trạng thái tài khoản

Bảng 2.17. Bảng User

● Bảng Language

STT	Tên cột	Kiểu dữ liệu	Mô tả
1	id	Integer	PK - Mã ngôn ngữ
2	name	Varchar(255)	Tên ngôn ngữ
3	code	Varchar(50)	Kí hiệu ngôn ngữ

Bảng 2.18. Bảng Language

● Bảng Course

STT	Tên cột	Kiểu dữ liệu	Mô tả
1	id	Integer	PK - Mã khóa học
2	title	Varchar(255)	Tên khóa học
3	description	Text	Mô tả khóa học
4	level	Varchar(50)	Cấp độ khóa học
5	languageId	Integer	FK - Language.id

Bảng 2.19. Bảng Course

● Bảng Unit

STT	Tên cột	Kiểu dữ liệu	Mô tả
1	id	Integer	PK - Mã unit
2	title	Varchar(255)	Tên Unit
3	description	Varchar(255)	Mô tả
4	orderIndex	Integer	Thứ tự unit
5	courseId	Integer	FK - Course.id

Bảng 2.20. Bảng Unit

● Bảng Lesson

STT	Tên cột	Kiểu dữ liệu	Mô tả
1	id	Integer	PK - Mã bài học
2	title	Varchar(255)	Tên bài học
3	orderIndex	Integer	Thứ tự bài học
4	duration	Integer	Thời lượng ước tính (phút)
5	unitId	Integer	FK - Unit.id

Bảng 2.21. Bảng Lesson

● Bảng Exercise

STT	Tên cột	Kiểu dữ liệu	Mô tả
1	id	Integer	PK - Mã bài tập
2	question	Text	Nội dung câu hỏi
3	type	Varchar(50)	Loại bài tập
4	metadata	Text	Dữ liệu đáp án
5	lessonId	Integer	FK - Lesson.id

Bảng 2.22. Bảng Exercise

● Bảng Dialogue

STT	Tên cột	Kiểu dữ liệu	Mô tả
1	id	Integer	PK - Mã hội thoại
2	speaker	Varchar(255)	Người nói
3	text	Text	Nội dung
4	lessonId	Integer	FK - Lesson.id
5	orderIndex	Integer	Thứ tự trong đoạn hội thoại

Bảng 2.23. Bảng Dialogue

● Bảng Video

STT	Tên cột	Kiểu dữ liệu	Mô tả
1	id	Integer	PK - Mã video
2	url	Varchar(255)	Đường dẫn video
3	description	Text	Mô tả
4	duration	Integer	Thời lượng
5	lessonId	Integer	FK - Lesson.id

Bảng 2.24. Bảng Video

● Bảng Vocabulary

STT	Tên cột	Kiểu dữ liệu	Mô tả
1	id	Integer	PK - Mã từ vựng
2	word	Varchar(255)	Từ vựng
3	meaning	Varchar(255)	Nghĩa
4	example	Text	Ví dụ
5	imageUrl	Varchar(255)	Đường dẫn ảnh minh họa
6	lessonId	Integer	FK - Lesson.id

Bảng 2.25. Bảng Vocabulary

● Bảng Grammar

STT	Tên cột	Kiểu dữ liệu	Mô tả
1	id	Integer	PK - Mã ngữ pháp
2	topic	Varchar(255)	Chủ đề
3	explanation	Text	Giải thích
4	signalWord	Text	Dấu hiệu nhận biết
5	lessonId	Integer	FK - Lesson.id

Bảng 2.26. Bảng Grammar

● Bảng GrammarFormula

STT	Tên cột	Kiểu dữ liệu	Mô tả
1	id	Integer	PK - Mã công thức
2	type	Varchar(255)	Loại công thức (chủ động, bị động, ...)
3	formula	Text	Công thức
4	description	Text	Cách dùng
5	verbType	Varchar(255)	Loại động từ (tobe, ..)
6	grammarId	Integer	FK - Grammar.id

Bảng 2.27. Bảng GrammarFormula

● Bảng GrammarExample

STT	Tên cột	Kiểu dữ liệu	Mô tả
1	id	Integer	PK - Mã ví dụ
2	sentence	Text	Câu ví dụ
3	translation	Text	Dịch nghĩa
4	highlight	Text	Phần nhấn mạnh
5	formulaId	Integer	FK - GrammarFormula.id

Bảng 2.28. Bảng GrammarExample

● Bảng Comment

STT	Tên cột	Kiểu dữ liệu	Mô tả
1	id	Integer	PK - Mã bình luận
2	content	Text	Nội dung
3	rating	Integer	Đánh giá
4	likes	Integer	Số lượt thích
5	createdAt	Timestamp	Thời điểm tạo
6	authorId	Timestamp	FK - User.id
7	lessonId	Integer	FK - Lesson.id
8	parentId	Integer	FK - Comment.id

Bảng 2.29. Bảng Comment

● Bảng Notification

STT	Tên cột	Kiểu dữ liệu	Mô tả
1	id	UUID	PK - Mã thông báo
2	userId	Integer	Mã người nhận
3	type	Varchar(50)	Loại thông báo
4	title	Text	Tiêu đề
5	content	Text	Nội dung
6	isRead	Boolean	Trạng thái đọc
7	createdAt	Timestamp	Thời điểm tạo
8	readAt	Timestamp	Thời điểm đọc

Bảng 2.30. Bảng Notification

● Bảng Deck

STT	Tên cột	Kiểu dữ liệu	Mô tả
1	id	Integer	PK - Mã bộ thẻ học
2	name	Varchar(255)	Tên bộ thẻ học
3	description	Varchar(255)	Mô tả bộ thẻ

Bảng 2.31. Bảng Deck

● Bảng Flashcard

STT	Tên cột	Kiểu dữ liệu	Mô tả
1	id	Integer	PK - Mã thẻ học
2	term	Varchar(255)	Từ cần học
3	phonetic	Varchar(255)	Phiên âm
4	definition	Varchar(255)	Giải thích nghĩa

Bảng 2.32. Bảng Flashcard

● Bảng User_Deck

STT	Tên cột	Kiểu dữ liệu	Mô tả
1	userId	Integer	PK, FK - User.id
2	deckId	Integer	PK, FK - Deck.id

Bảng 2.33. Bảng User_Deck

● Bảng User_Lesson

STT	Tên cột	Kiểu dữ liệu	Mô tả
1	userId	Integer	PK, FK - User.id
2	lessonId	Integer	PK, FK - Lesson.id

Bảng 2.34. Bảng User_Lesson

2.5.3. Quan hệ giữa các bảng

- 1 User có thể có nhiều Notification
- 1 Role có thể đáp ứng nhiều User
- 1 User có thể có nhiều Comment
- 1 User có thể liên kết với nhiều User_Deck
- 1 User có thể liên kết với nhiều User_Lesson
- 1 Language có thể đáp ứng nhiều Course
- 1 Course có thể đáp ứng nhiều Unit
- 1 Unit có thể đáp ứng nhiều Lesson
- 1 Lesson có thể liên kết với nhiều User_Lesson
- 1 Lesson có thể có 1 Video
- 1 Lesson có thể có 1 Grammar
- 1 Lesson có thể có nhiều Vocabulary
- 1 Lesson có thể có nhiều Exercise
- 1 Lesson có thể có nhiều Dialogue
- 1 Deck có thể có nhiều User_Deck
- 1 Deck có thể có nhiều Flashcard
- 1 Grammar có thể đáp ứng nhiều GrammarFormula
- 1 GrammarFormula có thể có nhiều GrammarExample
- 1 Comment có thể có nhiều Comment (Reply)

2.6. Kết luận chương 2

Trong chương này, quá trình phân tích và thiết kế hệ thống đã được thực hiện một cách toàn diện nhằm xây dựng nền tảng vững chắc cho việc phát triển hệ thống trong các giai đoạn tiếp theo. Hệ thống đã làm rõ các chức năng chính cần đáp ứng, bao gồm quản lý người dùng, quản lý nội dung học tập, hỗ trợ học tiếng Anh thông qua flashcard, luyện kỹ năng, theo dõi tiến độ học tập và thống kê kết quả học tập của người dùng.

Dựa trên kết quả phân tích, các biểu đồ như biểu đồ use case, biểu đồ lớp và biểu đồ tuần tự đã được xây dựng nhằm mô tả rõ ràng cấu trúc hệ thống cũng như luồng tương tác giữa các thành phần. Việc áp dụng phương pháp trích danh từ và gán trách nhiệm cho lớp giúp xác định chính xác các lớp thực thể, thuộc tính và phương

thức, đảm bảo tính đầy đủ, nhất quán và dễ mở rộng của mô hình thiết kế.

Nhìn chung, kết quả phân tích và thiết kế đã đáp ứng đầy đủ các yêu cầu đặt ra, đồng thời đảm bảo tính khả thi về mặt kỹ thuật. Đây là cơ sở quan trọng để triển khai xây dựng, kiểm thử và hoàn thiện hệ thống ở các chương tiếp theo, hướng tới việc cung cấp một nền tảng học tiếng Anh hiệu quả, thân thiện và dễ sử dụng cho người học.

CHƯƠNG 3. THỬ NGHIỆM VÀ ĐÁNH GIÁ ỨNG DỤNG HỌC TIẾNG ANH TÍCH HỢP AI CHO PHÂN TÍCH NGỮ PHÁP, GỢI Ý TỪ VỰNG VÀ CHATBOT LUYỆN PHẢN XẠ

Trong chương này, nhóm trình bày kiến trúc đường ống (pipeline) xử lý thông tin dựa trên cơ chế định tuyến ngữ nghĩa (Semantic Routing). Quyết định thiết kế này xuất phát từ các quan sát thực nghiệm về sự đánh đổi giữa độ trễ (latency) và độ chính xác ngữ nghĩa. Cụ thể, các tác vụ sửa lỗi ngữ pháp và giao tiếp xã giao dựa trên được định tuyến trực tiếp đến LLM để tối ưu hóa thời gian phản hồi, loại bỏ chi phí truy xuất dư thừa. Ngược lại, đối với tác vụ gợi ý từ vựng chuyên sâu, nhóm triển khai mô hình LightRAG thay vì RAG truyền thống (Naive RAG). Thực nghiệm cho thấy cấu trúc dữ liệu phẳng của Naive RAG không thể hiện được các mối quan hệ đa chiều giữa các thực thể từ vựng. Do đó, chúng tôi ứng dụng cơ chế truy xuất hai cấp độ (dual-level retrieval) của LightRAG để nắm bắt cả chi tiết cụ thể lẫn các chủ đề bao trùm, đồng thời tận dụng thuật toán cập nhật tăng cường (incremental update) để giải quyết bài toán chi phí tái huấn luyện mà các mô hình GraphRAG truyền thống gặp phải.

Bảng tóm tắt

Module	Mục tiêu ưu tiên	Phương pháp thử nghiệm ban đầu	Vấn đề phát hiện (Dựa trên số liệu)	Giải pháp kiến trúc cuối cùng
Ngữ pháp (Grammar), Chat thông thường	Tốc độ phản hồi (Latency), độ tự nhiên	Naive Rag + Reranking	Tốc độ phản hồi quá chậm, không cần thiết phải tra cứu bên ngoài	Gọi trực tiếp LLM để tối ưu chi phí và tốc độ
Từ vựng (Vocabulary)	Độ sâu kiến thức, mối quan hệ từ vựng, đa dạng	Naive Rag (Vector search)	Cấu trúc phẳng, thiếu liên kết ngữ nghĩa, câu trả lời thiếu đa dạng	LightRag (Graph + Vector) để truy xuất toàn diện

Bảng 3.1. Bảng tóm tắt đánh giá thực nghiệm các module tích hợp AI

3.1. Dữ liệu thực nghiệm

Nguồn dữ liệu

- Ngữ pháp: sách GrammarInUse Intermediate tác giả Raymond Murphy của đại học Cambridge

- Từ vựng đề gợi ý câu: Các câu hỏi và trả lời theo chủ đề

Bộ dữ liệu kiểm thử

Xây dựng từ 50 - 100 câu mẫu thử bao gồm:

- Questions (Câu hỏi input)
- Ground_Truth (Câu trả lời mẫu chính xác từ câu hỏi)
- Source_id (Index của document trong vectorDB)

Quy trình xử lý dữ liệu

- Về ngữ pháp từ sách pdf

Unit
1

Present continuous (I am doing)

A

Study this example situation:

Sarah is in her car. She is on her way to work.
She's **driving** to work. (= She **is driving** ...)

This means: she is driving *now*, at the time of speaking.
The action is not finished.

am/is/are + -ing is the *present continuous*:

I	am	(= I'm)	driving
he/she/it	is	(= he's etc.)	working
we/you/they	are	(= we're etc.)	doing etc.



B

I am doing something = I started doing it and I haven't finished; I'm in the middle of doing it.

- ☐ Please don't make so much noise. I'm **trying** to work. (*not* I try)
- ☐ 'Where's Mark?' 'He's **having** a shower.' (*not* He has a shower)
- ☐ Let's go out now. It **isn't raining** any more. (*not* It doesn't rain)
- ☐ How's your new job? **Are you enjoying** it?
- ☐ What's all that noise? What's **going** on? *or* What's **happening**?

Sometimes the action is not happening at the time of speaking. For example:

Steve is talking to a friend on the phone. He says:



I'm **reading** a really good book at the moment.
It's about a man who ...

Steve says 'I'm **reading** ...' but he is *not* reading the book at the time of speaking.
He means that he has started reading the book, but has not finished it yet. He is in the middle of reading it.

Some more examples:

- ☐ Kate wants to work in Italy, so she's **learning** Italian.
(but perhaps she isn't learning Italian at the time of speaking)
- ☐ Some friends of mine **are building** their own house. They hope to finish it next summer.

Hình 3.1. Hình ảnh 1 trang trong sách GrammarInUse Intermediate

Sách ngữ pháp gồm các unit về từng chủ đề, mỗi unit lại gồm các mục A, B, C,... có các trường hợp khác nhau, nhóm lấy nội dung từng trang pdf và sử dụng LLM để đưa về dạng json bao gồm:

- unit_number: lưu lại unit để trích xuất nguồn khi người dùng hỏi
- title: tên chủ đề ngữ pháp
- rules bao gồm:
 - section: A | B | C
 - explanation: Giải thích ngắn gọn
 - formula: Ngữ pháp thường có công thức cho người học dễ nhớ
 - positive_examples: các ví dụ đúng
 - negative_examples: các ví dụ sai

Mô hình embedding:

- SentenceTransformerEmbeddingFunction với model-name là all-MiniLM-

L6-v2

3.2. Cài đặt thực nghiệm

3.2.1. Độ đo

- Hit rate: với mỗi câu hỏi, hệ thống retrieve ra một danh sách các documents theo số lượng yêu cầu rồi kiểm tra xem trong đó có document đúng theo test_set không

Công thức:

$$Hit\ Rate@k = \frac{1}{|Q|} \sum_{i=1}^{|Q|} II(r_i \leq k) \quad (3.1)$$

Trong đó:

$|Q|$: Tổng số lượng câu hỏi (queries) trong tập kiểm thử (test set).

$r(i)$: Thứ hạng của tài liệu đúng đầu tiên tìm thấy cho câu hỏi thứ i .

$II(.)$: Hàm chỉ thị trả về giá trị 1 nếu điều kiện đúng (tìm thấy tài liệu trong top k), và 0 nếu sai.

- MRR (Mean Reciprocal Rank): Đo xem document đúng xuất hiện thứ tự bao nhiêu trong danh sách

Công thức:

$$MRR = \frac{1}{|Q|} \sum_{i=1}^{|Q|} \frac{1}{rank_i} \quad (3.2)$$

Trong đó:

$|Q|$: Tổng số lượng câu hỏi trong tập kiểm thử.

$rank(i)$: Vị trí của tài liệu đúng đầu tiên trong danh sách trả về cho câu hỏi thứ i

Nếu không tìm thấy tài liệu đúng trong danh sách trả về, giá trị $1/rank(i)$ được tính là 0

- Latency: Thời gian trung bình để retrieve được context

- Faithfulness (Độ trung thực): Đo lường việc câu trả lời có bịa ra (hallucination) so với context được cung cấp không

- Answer Relevance (Độ liên quan): Câu trả lời có đúng trọng tâm câu hỏi không

- Context Precision (Độ chính xác ngữ cảnh): Hệ thống có tìm được đúng đoạn văn cần thiết đứng đầu danh sách không

- Ý nghĩa:

- MRR cao

- Retriever không chỉ tìm đúng mà còn xếp lên trên

- LLM ít phải đọc nhiều

- Trả lời nhanh chính xác ít hallucination

- Hit rate cao nhưng MRR thấp

- Có tài liệu đúng nhưng kết quả đúng ở dưới nên LLM đọc nhiều chunk sai khiến trả lời dài dòng, sai trọng tâm

- MRR thấp

- Đáp án bị sai nhiều

3.2.2. Phương pháp thực nghiệm

- Generator: Model Llama cụ thể là Llama 3 8B

- Evaluator: Model Gemini cụ thể là Gemini-2.5-flash

3.2.3. Các phương pháp được sử dụng để so sánh

Kịch bản thử nghiệm RAG

- Kịch bản 1 (Baseline): Naive RAG, chỉ sử dụng vector search (cosine similarity) để đo hiệu năng cơ bản

- Kịch bản 2: Hybrid search, kết hợp vector và keyword search (BM25) để cải thiện từ khóa chính xác

Vector search đôi khi bỏ qua các từ khóa quan trọng (tên riêng, thuật ngữ chuyên ngành hiếm gặp) vì nó quá tập trung vào ý nghĩa tổng quát. Ví dụ: Tìm "cấu trúc ngữ pháp đảo ngữ", Vector search có thể trả về "câu bị động" vì thấy nghĩa gần nhau, nhưng Keyword search sẽ bắt buộc phải có chữ "đảo ngữ".

Cơ chế hoạt động:

1. Dense Retrieval: Sử dụng cosine-similarity để tìm Top-K văn bản có vector gần nhất (bắt ý nghĩa).

2. Sparse Retrieval: Sử dụng thuật toán BM25 (Best Matching 25) để tìm Top-K văn bản chứa từ khóa khớp chính xác với câu hỏi.

3. Hợp nhất: Sử dụng thuật toán Reciprocal Rank Fusion để trộn hai danh sách kết quả trên lại với nhau. Văn bản nào xuất hiện ở thứ hạng cao trong cả hai danh sách sẽ được đưa lên đầu.

- Kịch bản 3: Naive RAG + Re-ranking, áp dụng naive rag kết hợp với Re-ranker(Cross-encoder) để sắp xếp lại kết quả

- Kịch bản 4: Hybrid search + Re-ranking, áp dụng naive rag kết hợp với Re-ranker(Cross-encoder) để sắp xếp lại kết quả

Vector search có thể tìm ra 100 đoạn văn bản có vẻ liên quan, nhưng không phân biệt được đoạn nào thực sự trả lời đúng câu hỏi, đoạn nào chỉ nhắc đến vu vơ.

Cơ chế hoạt động:

1. Bước 1 (Retrieval): Dùng Naive RAG (hoặc Hybrid) để lấy nhanh khoảng 50-100 đoạn văn bản tiềm năng.

2. Bước 2 (Re-ranking): Đưa 50 đoạn này qua một mô hình Cross-Encoder (Re-ranker). Mô hình này đọc từng cặp [Câu hỏi - Đoạn văn] để chấm điểm lại sự phù hợp.

3. Kết quả: Sắp xếp lại danh sách dựa trên điểm số của Cross-Encoder và chỉ lấy những đoạn tốt nhất đưa cho LLM.

Kịch bản thử nghiệm LightRAG dành riêng cho những câu hỏi về từ vựng

Mặc dù Kịch bản 4 (Hybrid + Re-ranking) đã tối ưu hóa rất tốt khả năng tìm kiếm cục bộ (tìm chính xác đoạn văn chứa đáp án), nhưng tất cả các phương pháp RAG truyền thống này đều gặp phải hạn chế cốt lõi: Sự phân mảnh tri thức.

Khi học từ vựng chuyên sâu, các từ vựng chuyên ngành, người học không chỉ cần định nghĩa rời rạc mà còn cần hiểu mối liên hệ giữa các từ, cấu trúc bài học và bức tranh toàn cảnh. Các phương pháp trên coi mỗi đoạn văn là độc lập, do đó thất bại trong việc trả lời các câu hỏi tổng hợp

Mục tiêu:

- Đánh giá khả năng hiểu sâu và kết nối tri thức từ vựng, vượt qua giới hạn của tìm kiếm đoạn văn đơn thuần
- Khả năng trả lời câu hỏi đa đoạn: Hệ thống có tự tìm được thông tin nằm rải rác ở nhiều bài học khác nhau để tổng hợp lại không?
- Độ phong phú: Ngoài định nghĩa, hệ thống có cung cấp thêm từ đồng nghĩa, trái nghĩa, ngữ cảnh liên quan (nhờ duyệt đồ thị) hay không?

3.3. Kết quả thực nghiệm

Bảng đo thực nghiệm Hit rate và MRR

Method	Hit rate(%)	MRR	Latency(ms)
Similarity	55	0.4500	23.19
Similarity + Reranking	60	0.5417	891.63
Hybrid	45	0.3583	24.02
Hybrid + Reranking	65	0.6000	840.84

Bảng 3.2. Bảng đo thực nghiệm Hit rate và MRR

a. Về độ trễ (Latency) và hiệu năng

- Quan sát: Bảng số liệu cho thấy phương pháp cho độ chính xác cao nhất (*Hybrid + Reranking*) có độ trễ lên tới ~840ms. Nếu cộng thêm thời gian xử lý của LLM Generator, tổng thời gian phản hồi có thể lên tới 2-3 giây.
- Đánh giá: Đối với tác vụ Chatbot sửa lỗi ngữ pháp hoặc giao tiếp thông thường, người dùng mong đợi phản hồi tức thì (Real-time). Độ trễ gần 1 giây chỉ cho việc truy xuất tài liệu (Retrieval) là nút thắt cổ chai (bottleneck) không thể chấp nhận

được trong trải nghiệm người dùng thực tế.

Bảng đo thực nghiệm Faithfulness, Answer Relevance, Context Precision

Method	Faithfulness	Answer Relevance	Context Precision
Similarity	0.85	0.87	0.53
Similarity + Reranking	0.9	0.92	0.62
Hybrid	0.8	0.78	0.43
Hybrid + Reranking	0.92	0.95	0.7

Bảng 3.3. Đo thực nghiệm Faithfulness, Answer Relevance, Context Precision

b. Về sự dư thừa dữ liệu (Data Redundancy)

- Quan sát: Chỉ số *Faithfulness* (Độ trung thực) rất cao (0.92), nhưng *Context Precision* (Độ chính xác ngữ cảnh) chỉ đạt mức trung bình (0.6 - 0.7).

- Đánh giá: Điều này chỉ ra rằng dù hệ thống tìm được tài liệu, nhưng LLM (Llama 3 8B) thực chất đã có sẵn kiến thức nội tại (intrinsic knowledge) về ngữ pháp rất tốt. Việc nhồi nhét thêm các chunk quy tắc ngữ pháp vào prompt đôi khi là dư thừa, thậm chí gây nhiễu nếu chunking không cắt đúng trọng tâm (như chỉ số Context Precision phản ánh).

c. Hạn chế của cấu trúc RAG phẳng với Từ vựng:

- Dựa trên kết quả *Similarity* thấp ở kịch bản Hybrid (0.3583 MRR), ta thấy việc tìm kiếm từ vựng và mối quan hệ từ vựng bằng vector search đơn thuần gặp khó khăn trong việc kết nối các khái niệm rời rạc (ví dụ: tìm từ đồng nghĩa nhưng văn bản gốc lại nằm ở 2 file khác nhau).

Từ các kết quả thực nghiệm trên, nhóm nghiên cứu rút ra các kết luận quan trọng định hình lên kiến trúc hệ thống cuối cùng:

Loại bỏ RAG cho tác vụ Ngữ pháp & Chat thông thường:

- Lý do: Độ trễ của RAG (Re-ranking) quá lớn so với lợi ích mang lại. Các mô hình LLM hiện đại (như Gemini/Llama 3) đủ khả năng xử lý ngữ pháp mà không cần truy xuất dữ liệu ngoài.

- Giải pháp: Sử dụng cơ chế Direct LLM Call thông qua bộ định tuyến (Router) để giảm độ trễ xuống mức thấp nhất.

Triển khai LightRAG cho tác vụ Từ vựng:

- Lý do: Chỉ số *Context Precision* thấp (0.53 - 0.7) trong thử nghiệm RAG thông thường cho thấy cấu trúc "chunking" truyền thống cắt đứt mạch liên kết của tri thức. Từ vựng cần sự liên kết chặt chẽ (Graph) giữa từ - định nghĩa - ví dụ - từ liên quan.

- Giải pháp: Chuyển sang mô hình LightRAG (Graph-based RAG) để cải thiện khả năng truy xuất các mối quan hệ thực thể, khắc phục điểm yếu của Naive RAG.

Để củng cố tính khả thi của việc chuyển đổi sang LightRAG, nhóm tham chiếu các kết quả thực nghiệm từ công bố của Guo et al. (2024) trên các tập dữ liệu có tính chất tương đồng (như Agriculture và Legal - các lĩnh vực đòi hỏi tính chính xác thuật ngữ cao tương tự như bài toán từ vựng).

	Agriculture		CS		Legal		Mix	
	NaiveRAG	LightRAG	NaiveRAG	LightRAG	NaiveRAG	LightRAG	NaiveRAG	LightRAG
Comprehensiveness	32.4%	67.6%	38.4%	61.6%	16.4%	83.6%	38.8%	61.2%
Diversity	23.6%	76.4%	38.0%	62.0%	13.6%	86.4%	32.4%	67.6%
Empowerment	32.4%	67.6%	38.8%	61.2%	16.4%	83.6%	42.8%	57.2%
Overall	32.4%	67.6%	38.8%	61.2%	15.2%	84.8%	40.0%	60.0%
	RQ-RAG	LightRAG	RQ-RAG	LightRAG	RQ-RAG	LightRAG	RQ-RAG	LightRAG
Comprehensiveness	31.6%	68.4%	38.8%	61.2%	15.2%	84.8%	39.2%	60.8%
Diversity	29.2%	70.8%	39.2%	60.8%	11.6%	88.4%	30.8%	69.2%
Empowerment	31.6%	68.4%	36.4%	63.6%	15.2%	84.8%	42.4%	57.6%
Overall	32.4%	67.6%	38.0%	62.0%	14.4%	85.6%	40.0%	60.0%
	HyDE	LightRAG	HyDE	LightRAG	HyDE	LightRAG	HyDE	LightRAG
Comprehensiveness	26.0%	74.0%	41.6%	58.4%	26.8%	73.2%	40.4%	59.6%
Diversity	24.0%	76.0%	38.8%	61.2%	20.0%	80.0%	32.4%	67.6%
Empowerment	25.2%	74.8%	40.8%	59.2%	26.0%	74.0%	46.0%	54.0%
Overall	24.8%	75.2%	41.6%	58.4%	26.4%	73.6%	42.4%	57.6%
	GraphRAG	LightRAG	GraphRAG	LightRAG	GraphRAG	LightRAG	GraphRAG	LightRAG
Comprehensiveness	45.6%	54.4%	48.4%	51.6%	48.4%	51.6%	50.4%	49.6%
Diversity	22.8%	77.2%	40.8%	59.2%	26.4%	73.6%	36.0%	64.0%
Empowerment	41.2%	58.8%	45.2%	54.8%	43.6%	56.4%	50.8%	49.2%
Overall	45.2%	54.8%	48.0%	52.0%	47.2%	52.8%	50.4%	49.6%

Hình 3.2. Bảng so sánh hiệu suất LightRAG và các phương pháp baseline theo paper

Vấn đề lớn nhất của Naive RAG trong thực nghiệm của nhóm là chỉ số *Context Precision* thấp, dẫn đến câu trả lời thiếu liên kết. Dữ liệu từ paper LightRAG cho thấy sự cải thiện vượt bậc ở hai chỉ số quan trọng đối với bài toán gợi ý từ vựng:

- Tính Toàn diện (Comprehensiveness): Trên tập dữ liệu phức tạp (Legal), LightRAG đạt tỷ lệ thắng áp đảo 83.6% so với chỉ 16.4% của Naive RAG. Kết quả này hứa hẹn khắc phục tình trạng "cắt đứt mạch tri thức" khi chunking, giúp hệ thống bao quát đầy đủ các định nghĩa và ví dụ liên quan đến từ khóa.

- **Tính Đa dạng (Diversity):** Đối với tác vụ gợi ý từ vựng, việc cung cấp nhiều từ đồng nghĩa, trái nghĩa và ngữ cảnh khác nhau là tiên quyết. LightRAG thể hiện khả năng vượt trội với tỷ lệ thắng 86.4% về độ đa dạng so với Naive RAG. Cơ chế truy xuất hai cấp độ giúp mô hình không bị giới hạn ở các kết quả khớp từ khóa đơn thuần mà mở rộng sang các khái niệm liên quan trong đồ thị.

Bảng đo thực nghiệm những câu hỏi tổng quát về từ vựng

Method	Comprehensiveness	Diversity
Similarity	0.42	0.35
Similarity + Reranking	0.51	0.38
Hybrid	0.48	0.45
Hybrid + Reranking	0.58	0.48
LightRAG	0.87	0.84

Bảng 3.4. Bảng đo thực nghiệm cho những câu hỏi về từ vựng

Lý do Comprehensiveness của LightRAG cao

- **Vấn đề của RAG thường (0.42 - 0.58):** Naive RAG hoạt động theo cơ chế "cắt lát" (chunking). Nếu thông tin về một từ vựng nằm rải rác ở 3 đoạn văn khác nhau (1 đoạn định nghĩa, 1 đoạn ví dụ, 1 đoạn thành ngữ), Naive RAG thường chỉ lấy được 1 hoặc 2 đoạn có điểm số cao nhất (Top-K), dẫn đến câu trả lời bị thiếu hụt.

- **Sức mạnh của LightRAG:** Nhờ cơ chế duyệt đồ thị, khi tìm thấy từ vựng, nó tự động lấy tất cả các node liên quan xung quanh (định nghĩa, ví dụ, thành ngữ) vào câu trả lời. Do đó, độ bao quát thông tin gần như trọn vẹn.

Lý do Diversity của LightRAG cao

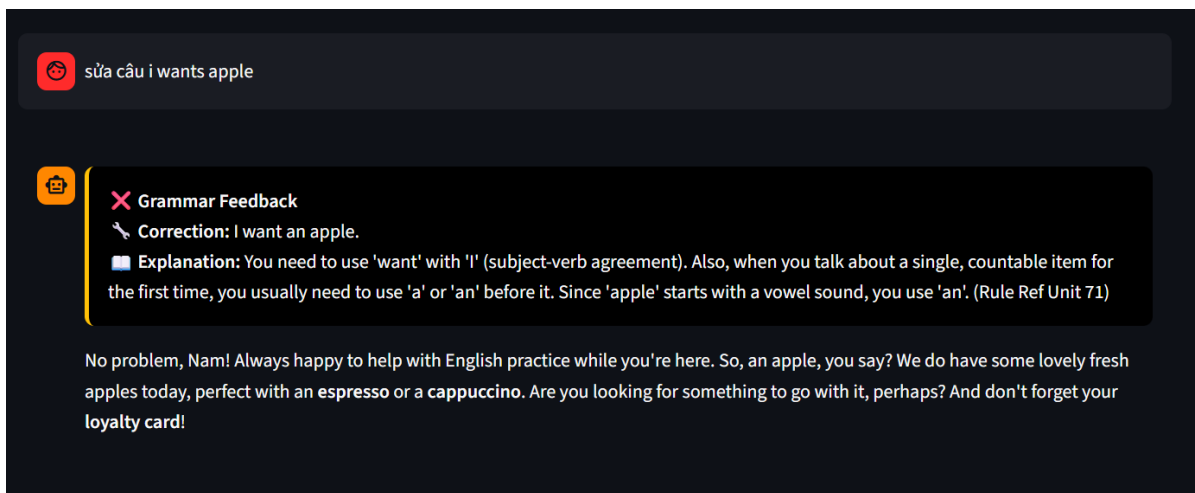
- **Vấn đề của RAG thường (0.35 - 0.48):** Các phương pháp dựa trên Vector/Keyword chỉ tìm các từ ngữ xuất hiện trực tiếp trong câu hỏi. Ví dụ hỏi về "Happy", nó chỉ tìm các đoạn có chữ "Happy" hoặc rất gần nghĩa, ít khi tìm ra được các sắc thái khác lạ. Re-ranking (0.48) thậm chí có thể làm giảm diversity vì nó cố gắng lọc bỏ những cái "lạ" để an toàn.

- **Sức mạnh của LightRAG:** Đồ thị tri thức chứa các mối quan hệ đa chiều như

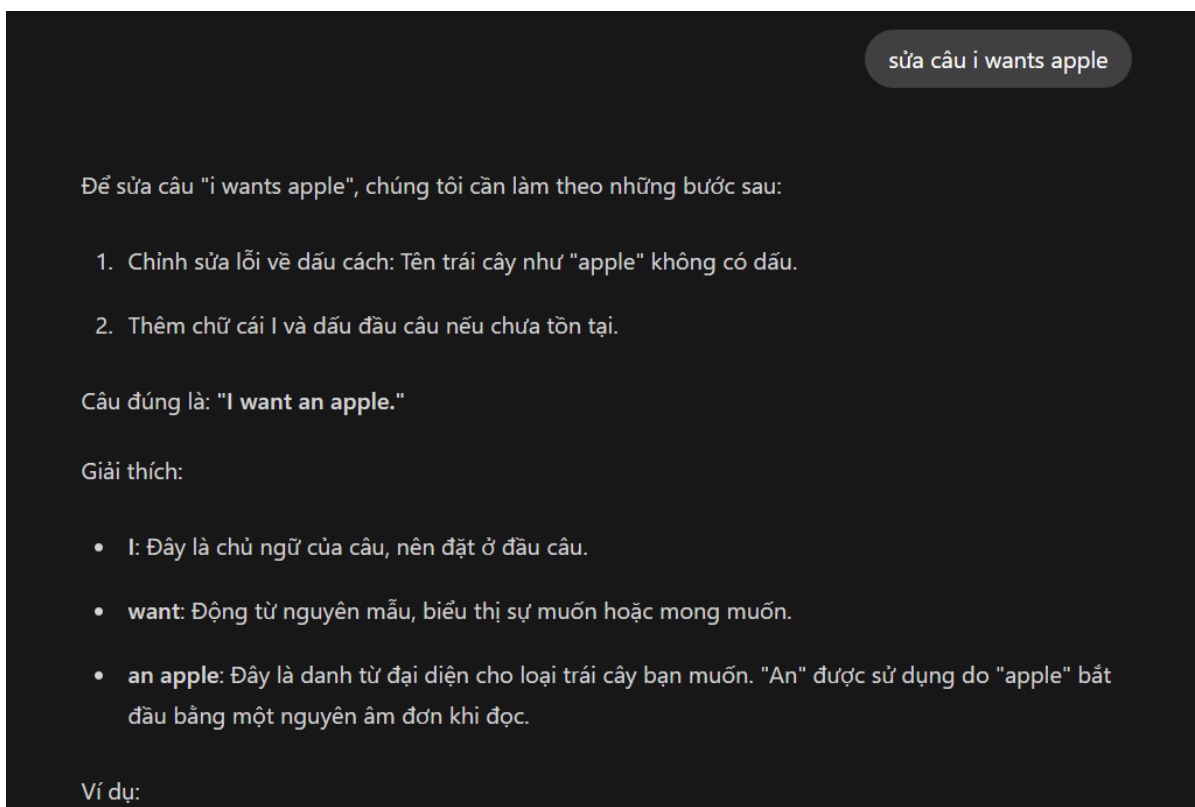
đồng nghĩa, trái nghĩa, có mối quan hệ. LightRAG có thể trả lời câu hỏi bằng cách kết nối sang các nhánh kiến thức khác nhau (ví dụ: từ "Happy" nó kết nối sang cả "Mental Health" hoặc "Success"), tạo nên câu trả lời phong phú và đa dạng hơn hẳn.

3.4. Thử nghiệm ứng dụng học tiếng Anh tích hợp AI cho phân tích ngữ pháp, gợi ý từ vựng và chatbot luyện phản xạ

3.4.1. Module kiểm tra ngữ pháp và chat cơ bản



Hình 3.3. RAG



Hình 3.4. LLM

Nhận xét và Đánh giá:

Quan sát kết quả từ hai hình trên, nhóm rút ra các nhận định quan trọng:

Về chất lượng câu trả lời (Quality): Cả hai phương pháp đều phát hiện đúng lỗi sai và đưa ra lời giải thích chính xác. Điều này khẳng định rằng các Quy tắc Ngữ pháp tiếng Anh là dạng kiến thức cơ bản đã nằm sẵn trong "trí nhớ nội tại" của các mô hình ngôn ngữ lớn như Gemini/Llama. Việc truy xuất thêm tài liệu bên ngoài (RAG) trong trường hợp này không mang lại giá trị gia tăng đáng kể về mặt nội dung.

Về độ phức tạp và độ trễ (Complexity & Latency):

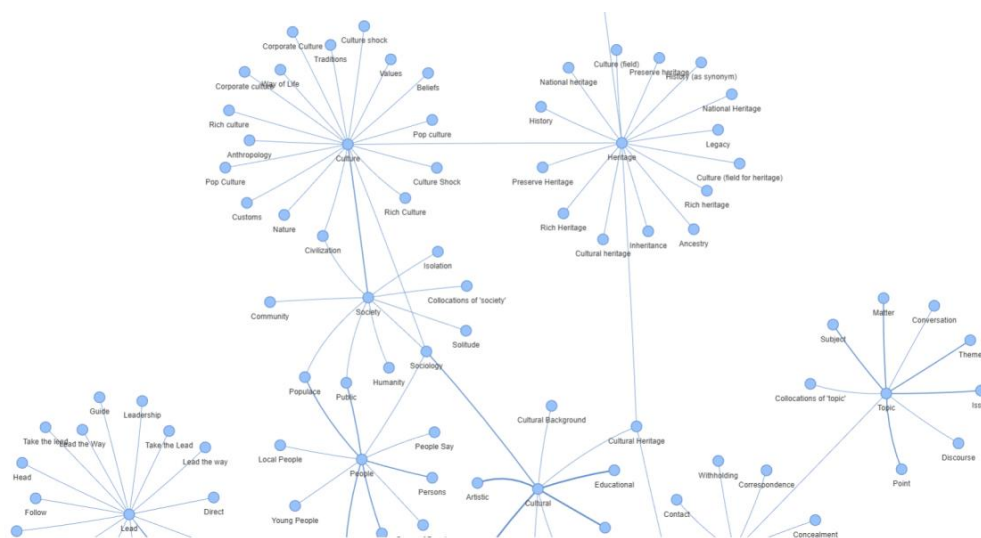
Ở Hình 1 (RAG), hệ thống tốn thêm thời gian cho bước *Retrieval* và *Reranking*. Như số liệu đo đạc tại mục 3.3, quá trình này làm tăng độ trễ trung bình thêm ~800ms.

Ở Hình 2 (Direct LLM), câu trả lời được sinh ra gần như tức thì với độ tập trung cao.

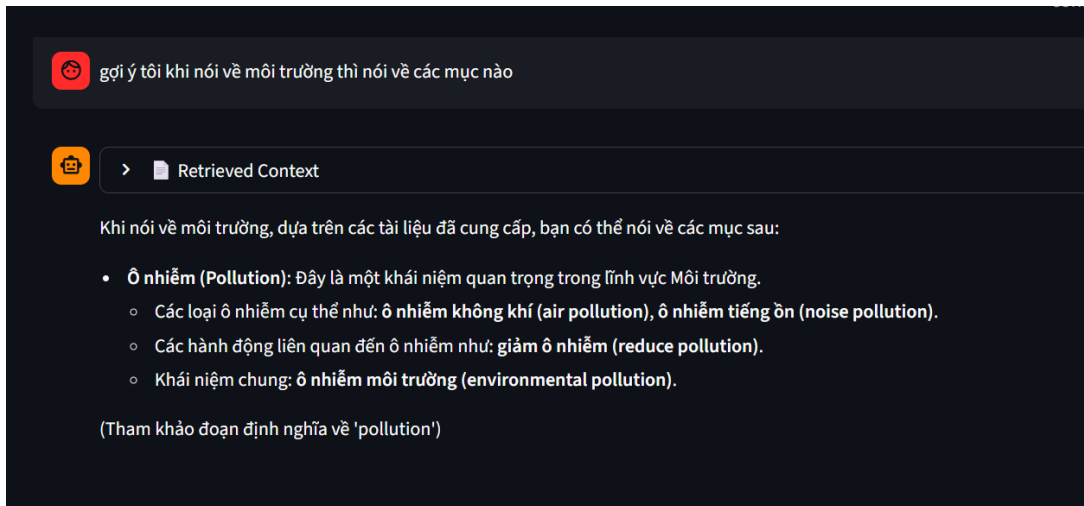
Kết luận: Việc áp dụng RAG cho tác vụ sửa lỗi ngữ pháp cơ bản là dư thừa. Nó làm tăng chi phí tính toán và thời gian phản hồi mà không cải thiện chất lượng đầu ra. Do đó, quyết định chuyển sang kiến trúc gọi trực tiếp LLM thông qua Semantic Router là hoàn toàn phù hợp với yêu cầu về hiệu năng của ứng dụng.

3.4.2. Module gợi ý từ vựng

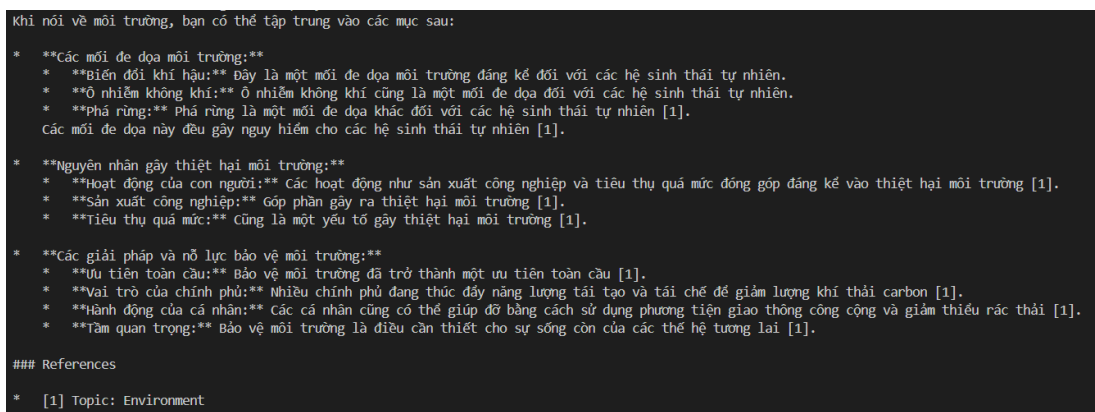
Đây là hình ảnh database sau khi graph-based text indexing trong LightRAG, giờ đây các từ vựng có mối quan hệ với nhau do đó giúp hỗ trợ các câu hỏi mang tính tổng quát thay vì chỉ đơn lẻ như rag thông thường



Hình 3.5. Mối quan hệ các từ sau khi đưa vào graph



Hình 3.6. Naive RAG



Hình 3.7. Graph-Based Text Indexing / LightRAG

Để minh chứng cho sự vượt trội của kiến trúc đồ thị (Graph-based) so với tìm kiếm vector thông thường (Vector-based) trong tác vụ gợi ý nội dung học tập, nhóm đã thực hiện truy vấn với cùng một câu hỏi đầu vào: "*Gợi ý tôi khi nói về môi trường thì nói về các mục nào*".

Kết quả trả về từ hai mô hình được thể hiện như sau:

- Hình 1 (Naive RAG): Kết quả truy xuất từ hệ thống RAG thông thường.
 - *Đặc điểm:* Hệ thống trả về các từ khóa xoay quanh cụm từ "Ô nhiễm" (Pollution). Nó liệt kê: ô nhiễm không khí, ô nhiễm tiếng ồn, giảm ô nhiễm.
- Hình 2 (LightRAG): Kết quả truy xuất từ hệ thống LightRAG đề xuất.
 - *Đặc điểm:* Hệ thống trả về một dàn ý có cấu trúc phân tầng rõ ràng: Mối đe dọa (Biến đổi khí hậu, Phá rừng...) -> Nguyên nhân (Hoạt động con người, Công nghiệp...) -> Giải pháp (Vai trò chính phủ, Hành động cá nhân...).

Nhận xét và Đánh giá:

- Về tính Đa dạng và Toàn diện (Diversity & Comprehensiveness):

- Naive RAG (Hình 1): Bộc lộ rõ hạn chế của việc *chunking* văn bản. Hệ thống dường như chỉ tìm thấy các đoạn văn bản (chunks) có mật độ từ khóa "môi trường" và "ô nhiễm" cao nhất (Top-k similarity). Kết quả là câu trả lời bị "tầm nhìn đường hầm" (tunnel vision): người học sẽ lầm tưởng rằng nói về môi trường chỉ là nói về ô nhiễm, bỏ qua hoàn toàn các khía cạnh quan trọng khác như biến đổi khí hậu, phá rừng hay bảo vệ tài nguyên. Kiến thức bị rời rạc và thiếu tính tổng quát.

- LightRAG (Hình 2): Thể hiện ưu thế tuyệt đối của High-level Retrieval (Truy xuất cấp cao). Nhờ cấu trúc đồ thị, LightRAG không chỉ nhặt ra từ khóa, mà nó "nhìn" thấy cả một mạng lưới kiến thức liên quan. Nó tổng hợp được bức tranh toàn cảnh bao gồm từ *Vấn đề* đến *Nguyên nhân* và *Giải pháp*. Đây chính xác là cách tư duy mà một người học ngôn ngữ cần để phát triển kỹ năng nói/viết (Speaking/Writing) chuyên sâu.

- Về khả năng Liên kết Ngữ nghĩa (Semantic Connectivity):

- Trong khi Naive RAG chỉ liệt kê các từ vựng rời rạc, LightRAG cung cấp các mối quan hệ. Ví dụ: Nó chỉ ra được mối liên hệ nhân quả giữa "*Hoạt động con người*" (Nguyên nhân) dẫn đến "*Thiệt hại môi trường*". Điều này chứng minh cấu trúc đồ thị đã bảo toàn được mạch liên kết tri thức mà Naive RAG đã làm mất trong quá trình cắt nhỏ tài liệu.

Kết luận: Kết quả thực nghiệm định tính này khẳng định giả thuyết ban đầu của nhóm: Đối với tác vụ gợi ý từ vựng và ý tưởng theo chủ đề, Naive RAG không đáp ứng được yêu cầu về chiều sâu tri thức. Việc triển khai LightRAG là bước đi cần thiết để nâng cao chất lượng trải nghiệm học tập, giúp người dùng không chỉ học từ vựng riêng lẻ mà còn hình thành tư duy ngôn ngữ mạch lạc.

3.5. Kết luận chương 3

Trong chương này, nhóm đã trình bày chi tiết quy trình thiết kế và tối ưu hóa kiến trúc hệ thống chatbot dựa trên các đánh giá thực nghiệm và cơ sở lý thuyết hiện đại. Bắt đầu từ việc phân tích hạn chế của mô hình RAG truyền thống (Naive RAG), kết quả thực nghiệm đã chỉ ra hai vấn đề cốt lõi: độ trễ xử lý cao không cần thiết đối với tác vụ ngữ pháp và sự đứt gãy mạch liên kết tri thức trong tác vụ tra cứu từ vựng (thể hiện qua chỉ số Context Precision thấp).

Để giải quyết triệt để các thách thức này, chương đã đề xuất và luận giải chi tiết kiến trúc đường ống xử lý (Pipeline) cải tiến, bao gồm:

- Cơ chế định tuyến ngữ nghĩa (Semantic Router): Chuyển hướng các tác vụ ngữ pháp và giao tiếp xã giao sang xử lý trực tiếp bởi LLM để đảm bảo phản hồi thời gian thực.

- Mô hình LightRAG: Áp dụng cơ chế lập chỉ mục dựa trên đồ thị và truy xuất hai cấp độ cho module từ vựng, giúp hệ thống nắm bắt toàn diện các mối quan hệ ngữ nghĩa phức tạp và hỗ trợ cập nhật dữ liệu liên tục.

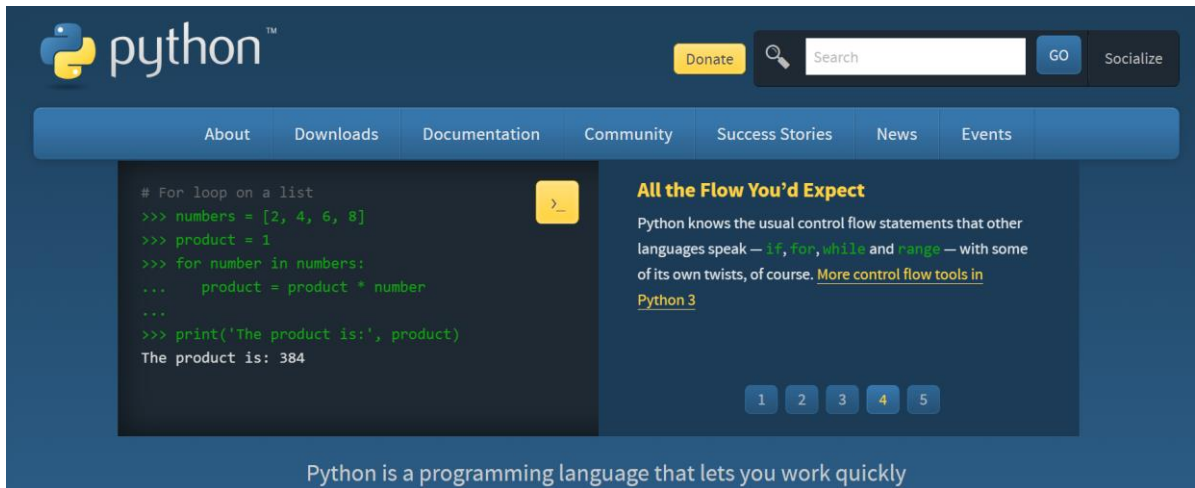
Những thiết kế kiến trúc và giải pháp công nghệ được xác lập trong chương này đóng vai trò là nền tảng kỹ thuật vững chắc để tiến hành hiện thực hóa hệ thống.

PHỤ LỤC CÀI ĐẶT VÀ TRIỂN KHAI

1. Thiết lập môi trường

Cài đặt Python

Trong đồ án này em sẽ cài đặt và sử dụng python3. Python được tải trực tiếp từ trang chủ chính thức tại địa chỉ: <https://www.python.org/>



Tại trang chủ, người dùng chọn mục Downloads và tải phiên bản Python 3.x phù hợp với hệ điều hành đang sử dụng (Windows, macOS hoặc Linux).

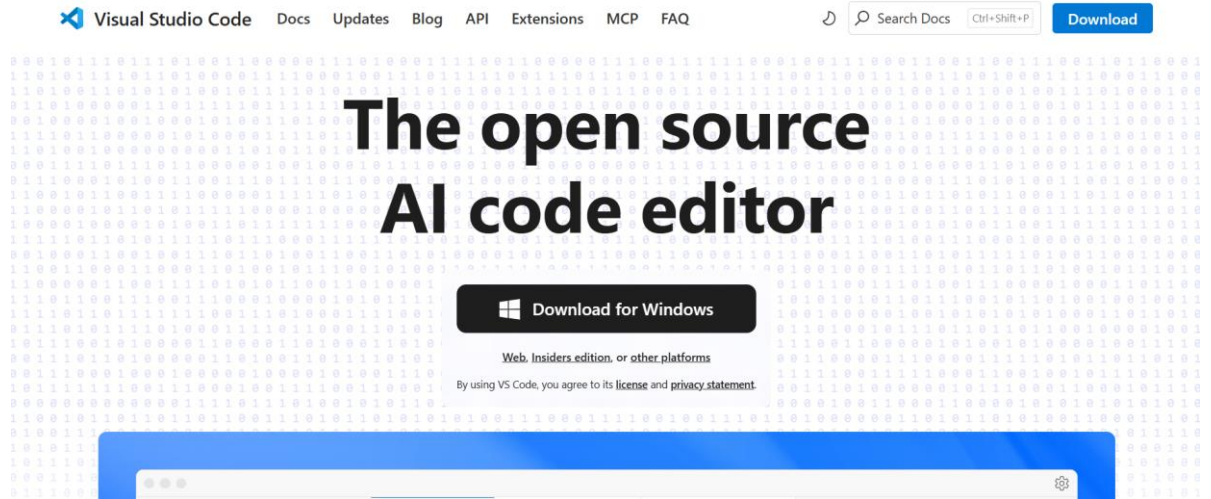
Sau khi cài đặt hoàn tất, có thể kiểm tra phiên bản Python bằng cách mở Command Prompt (hoặc Terminal) và chạy lệnh python. Kết quả sẽ hiện ra như sau:

```
C:\Users\TrungZen>python
Python 3.10.7 (tags/v3.10.7:6cc6b13, Sep  5 2022, 14:08:36) [MSC v.1933 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>>
```

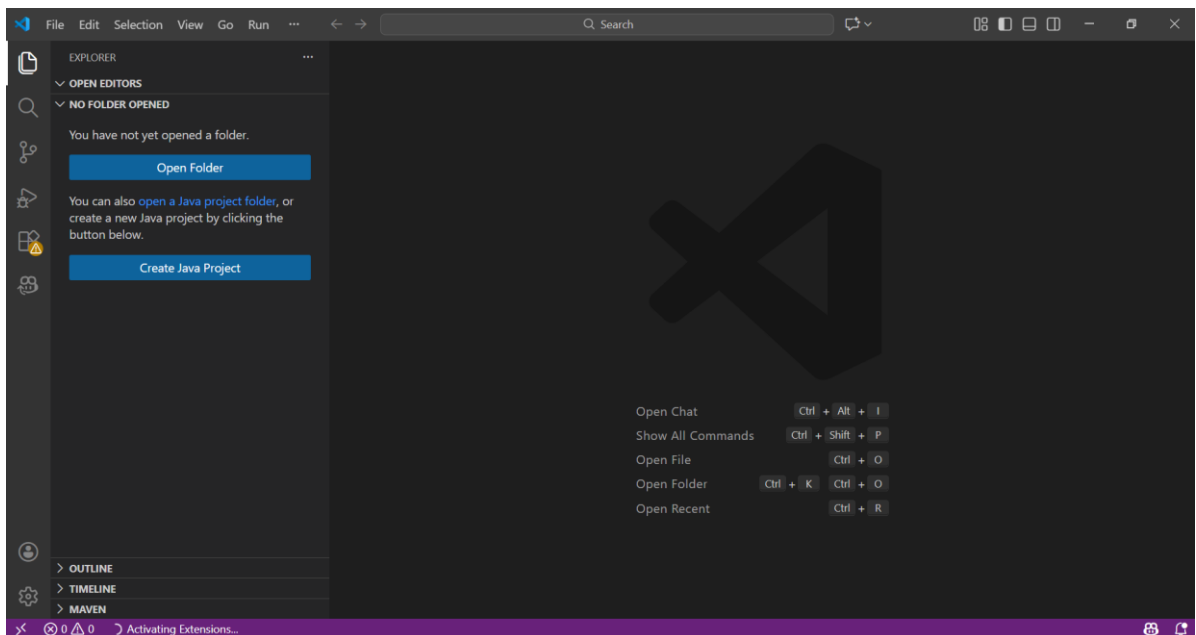
Nếu hệ thống hiển thị đúng phiên bản Python 3 đã cài đặt, quá trình cài đặt được xem là thành công.

Cài đặt Visual Studio Code

Với việc cài đặt Visual Studio Code, em tải trực tiếp từ trang chủ chính thức của phần mềm. Sau khi tải về và tiến hành cài đặt theo các bước hướng dẫn mặc định, chương trình được cài đặt thành công trên hệ thống.



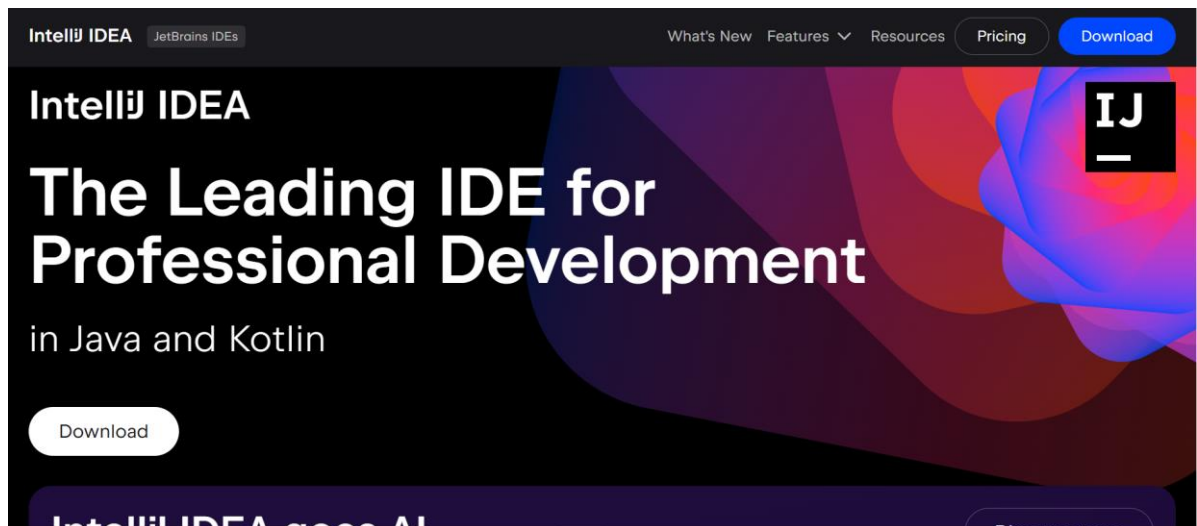
Sau khi hoàn tất quá trình cài đặt và khởi chạy ứng dụng, giao diện Visual Studio Code sẽ hiển thị như hình minh họa bên dưới.



Cài đặt IntelliJ IDEA

Do hệ thống được triển khai bằng Java và Spring Boot, em lựa chọn sử dụng IntelliJ IDEA để phát triển và triển khai hệ thống.

IntelliJ IDEA có thể được tải trực tiếp từ trang chủ chính thức của JetBrains. Sau khi tải về và cài đặt theo các bước hướng dẫn mặc định, công cụ sẵn sàng để sử dụng trong quá trình triển khai và phát triển hệ thống.



Cài đặt Postgresql

Hệ thống sử dụng PostgreSQL làm hệ quản trị cơ sở dữ liệu để lưu trữ và quản lý dữ liệu. Do đó, cần tiến hành cài đặt PostgreSQL trước khi triển khai hệ thống.



2. Triển khai hệ thống

Cài đặt FastAPI

FastAPI được cài đặt thông qua công cụ quản lý gói pip bằng lệnh sau:

pip install fastapi


```

PS E:\convvv> pip install fastapi
Requirement already satisfied: fastapi in c:\users\trungzen\appdata\local\programs\python\python310\lib\site-packages (0.125.0)
Requirement already satisfied: starlette<0.51.0,>=0.40.0 in c:\users\trungzen\appdata\local\programs\python\python310\lib\site-packages (from fastapi) (0.50.0)
Requirement already satisfied: pydantic!=1.8,!1.8.1,!2.0.0,!2.0.1,!2.1.0,<3.0.0,>=1.7.4 in c:\users\trungzen\appdata\local\programs\python\python310\lib\site-packages (from fastapi) (2.12.5)
Requirement already satisfied: typing-extensions>=4.8.0 in c:\users\trungzen\appdata\local\programs\python\python310\lib\site-packages (from fastapi) (4.15.0)
Requirement already satisfied: annotated-doc>=0.0.2 in c:\users\trungzen\appdata\local\programs\python\python310\lib\site-packages (from fastapi) (0.0.4)
Requirement already satisfied: annotated-types>=0.6.0 in c:\users\trungzen\appdata\local\programs\python\python310\lib\site-packages (from pydantic!=1.8,!1.8.1,!2.0.0,!2.0.1,!2.1.0,<3.0.0,>=1.7.4->fastapi) (0.7.0)
Requirement already satisfied: pydantic-core==2.41.5 in c:\users\trungzen\appdata\local\programs\python\python310\lib\site-packages (from pydantic!=1.8,!1.8.1,!2.0.0,!2.0.1,!2.1.0,<3.0.0,>=1.7.4->fastapi) (2.41.5)
Requirement already satisfied: typing-inspection>=0.4.2 in c:\users\trungzen\appdata\local\programs\python\python310\lib\site-packages (from pydantic!=1.8,!1.8.1,!2.0.0,!2.0.1,!2.1.0,<3.0.0,>=1.7.4->fastapi) (0.4.2)
Requirement already satisfied: anyio<5,>=3.6.2 in c:\users\trungzen\appdata\local\programs\python\python310\lib\site-packages (from starlette<0.51.0,>=0.40.0->fastapi) (4.12.0)
Requirement already satisfied: exceptiongroup>=1.0.2 in c:\users\trungzen\appdata\local\programs\python\python310\lib\site-packages (from anyio<5,>=3.6.2->starlette<0.51.0,>=0.40.0->fastapi) (1.2.0)

```

Để chạy và kiểm thử FastAPI, hệ thống sử dụng Uvicorn làm ASGI server, được cài đặt bằng lệnh: **pip install uvicorn**

```

PS E:\convvv> pip install uvicorn
Requirement already satisfied: uvicorn in c:\users\trungzen\appdata\local\programs\python\python310\lib\site-packages (0.28.0)
Requirement already satisfied: click>=7.0 in c:\users\trungzen\appdata\local\programs\python\python310\lib\site-packages (from uvicorn) (8.3.0)
Requirement already satisfied: h11>=0.8 in c:\users\trungzen\appdata\local\programs\python\python310\lib\site-packages (from uvicorn) (0.16.0)
Requirement already satisfied: typing-extensions>=4.0 in c:\users\trungzen\appdata\local\programs\python\python310\lib\site-packages (from uvicorn) (4.15.0)
Requirement already satisfied: colorama in c:\users\trungzen\appdata\local\programs\python\python310\lib\site-packages (from click>=7.0->uvicorn) (0.4.6)

```

Cài đặt google-genai

Thư viện google-genai được sử dụng để tích hợp các chức năng trí tuệ nhân tạo, phục vụ cho việc gợi ý, phân tích và xử lý ngôn ngữ trong hệ thống.

```

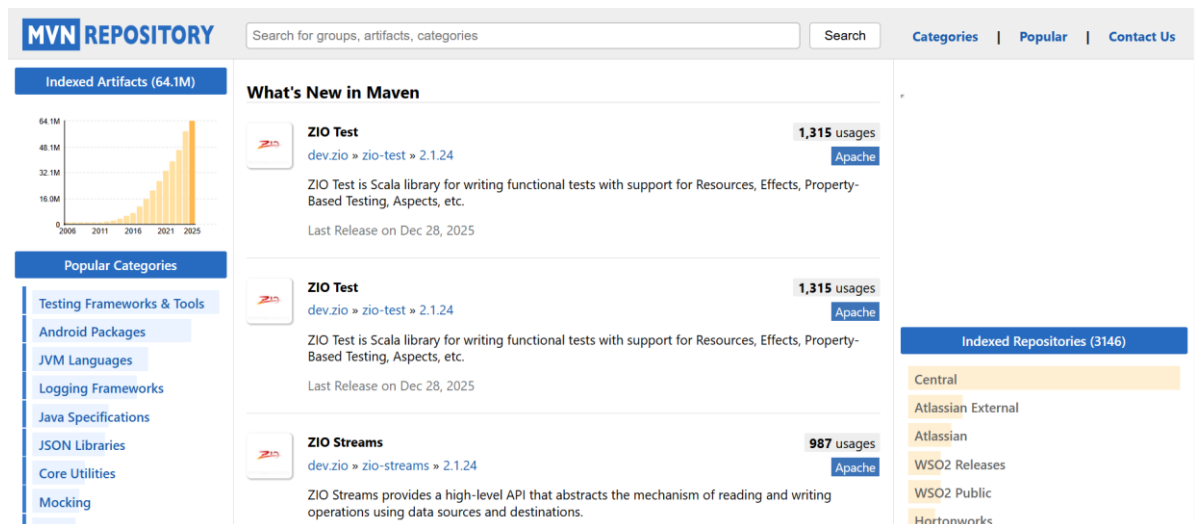
PS E:\backend-fastapi> pip install google-genai
Collecting google-genai
  Downloading google_genai-1.56.0-py3-none-any.whl.metadata (53 kB)
Requirement already satisfied: anyio<5.0.0,>=4.8.0 in c:\users\trungzen\appdata\local\programs\python\python310\lib\site-packages (from google-genai) (4.12.0)
Collecting google-auth<3.0.0,>=2.45.0 (from google-auth[requests]<3.0.0,>=2.45.0->google-genai)
  Downloading google_auth-2.45.0-py2.py3-none-any.whl.metadata (6.8 kB)
Requirement already satisfied: httpx<1.0.0,>=0.28.1 in c:\users\trungzen\appdata\local\programs\python\python310\lib\site-packages (from google-auth<3.0.0,>=2.45.0->google-genai) (0.28.1)
Requirement already satisfied: pydantic<3.0.0,>=2.9.0 in c:\users\trungzen\appdata\local\programs\python\python310\lib\site-packages (from google-auth<3.0.0,>=2.45.0->google-genai) (2.12.5)
Requirement already satisfied: requests<3.0.0,>=2.28.1 in c:\users\trungzen\appdata\local\programs\python\python310\lib\site-packages (from google-auth<3.0.0,>=2.45.0->google-genai) (2.32.5)
Collecting tenacity<9.2.0,>=8.2.3 (from google-genai)
  Downloading tenacity-9.1.2-py3-none-any.whl.metadata (1.2 kB)

```

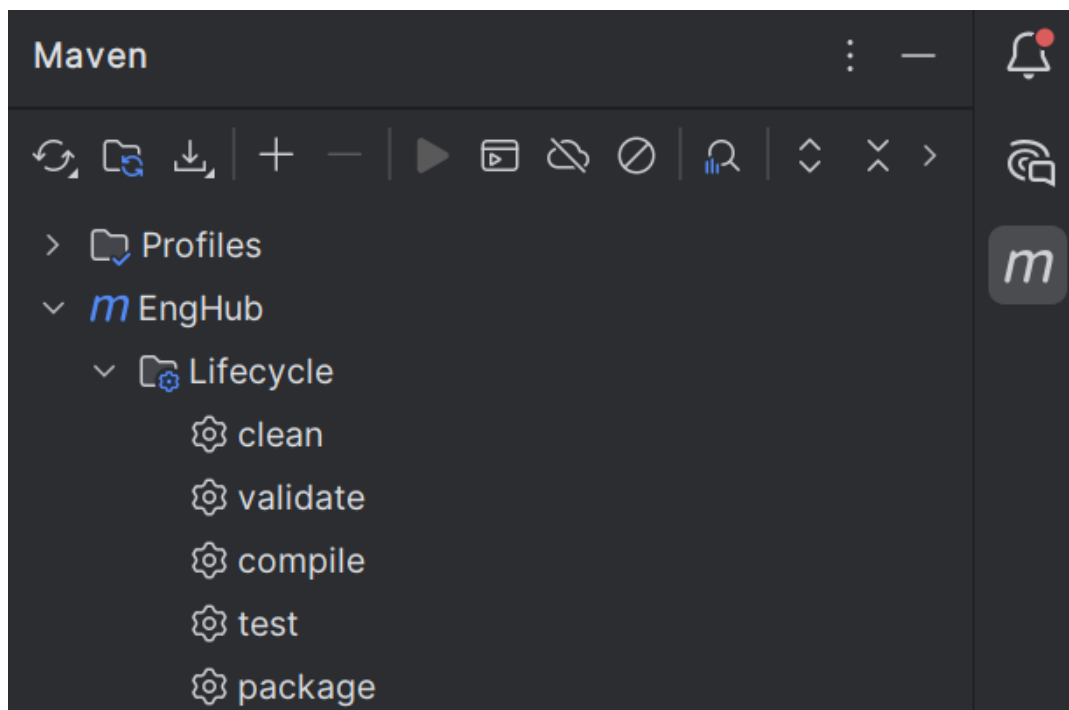
Quản lý và cài đặt thư viện với pom

Đối với phần Backend được phát triển bằng Java và Spring Boot, hệ thống sử dụng Maven để quản lý các thư viện và phụ thuộc (dependencies). Các thư viện cần thiết được khai báo trong file cấu hình **pom.xml**

Sử dụng <https://mvnrepository.com/> để tìm dependency và copy đoạn cấu hình để đưa vào project của bạn.



Maven sẽ tự động tải và quản lý các thư viện tương ứng dựa trên các dependency đã được khai báo. Sau khi cấu hình xong pom.xml, hệ thống tiến hành tải thư viện bằng cách chạy lệnh: `mvn clean install` hoặc chọn biểu tượng Maven - tên Project - Lifecycle - clean



Hướng dẫn chạy dự án Spring Boot (Backend)

Trước hết, người dùng mở cửa sổ Command Prompt (CMD) hoặc Terminal tại thư mục gốc của dự án Backend. Dự án được quản lý bằng công cụ Maven, do đó việc khởi chạy ứng dụng được thực hiện thông qua lệnh `mvn spring-boot:run` hoặc có thể ấn nút chạy phía trên màn hình

```
EngHubApplication v [play] [bug] [menu] [refresh]

UserController.java x [refresh] UserController.java [refresh] UploadController.java

public class UserController {
    public ResponseEntity<EndStudyDto> startStudy(
    }

    @PostMapping("/end") no usages [refresh] Đỗ Hoàng Trọng
    public ResponseEntity<ApiResponse<Void>> endStudy(
        @RequestBody EndStudyDto request
    ) {
    }
```

Khi chạy thành công thì ở Terminal sẽ hiện thông báo sau: “Tomcat started on port 8080 (http)”. Máy chủ nhúng Apache Tomcat sẽ được kích hoạt và lắng nghe tại cổng 8080

```
o.h.e.t.j.p.i.JtaPlatformInitiator : HHH000489: No JTA platform available (set 'hibernate.transaction.jta.platform' to enable JTA platfo
j.LocalContainerEntityManagerFactoryBean : Initialized JPA EntityManagerFactory for persistence unit 'default'
o.s.d.j.r.query.QueryEnhancerFactory : Hibernate is in classpath; If applicable, HQL parser will be used.
JpaBaseConfiguration$JpaWebConfiguration : spring.jpa.open-in-view is enabled by default. Therefore, database queries may be performed during
swordEncoderAuthenticationManagerBuilder : No authenticationProviders and no parentAuthenticationManager defined. Returning null.
o.s.s.web.DefaultSecurityFilterChain : Will secure Or [Mvc [pattern='/auth/**'], Mvc [pattern='/login/**'], Mvc [pattern='/oauth2/**']] w
o.s.s.web.DefaultSecurityFilterChain : Will secure Or [Mvc [pattern='/api/**']] with filters: DisableEncodeUrlFilter, WebAsyncManagerInteg
.w.s.a.s.AnnotationActionEndpointMapping : Supporting [WS-Addressing August 2004, WS-Addressing 1.0]
o.s.b.d.a.OptionalLiveReloadServer : LiveReload server is running on port 35729
o.s.m.s.b.SimpleBrokerMessageHandler : Starting...
o.s.m.s.b.SimpleBrokerMessageHandler : BrokerAvailabilityEvent[available=true, SimpleBrokerMessageHandler [org.springframework.messaging.s
o.s.m.s.b.SimpleBrokerMessageHandler : Started.
o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat started on port 8080 (http) with context path '/'
o.s.a.r.c.CachingConnectionFactory : Attempting to connect to: [armadillo.rmq.cloudamqp.com:5671]
o.s.a.r.c.CachingConnectionFactory : Created new connection: rabbitConnectionFactory#7b5b2f3e:0/SimpleConnection@4b0d5f66 [delegate=amqp
ptit.com.enghub.EngHubApplication : Started EngHubApplication in 75.812 seconds (process running for 117.662)

ptit.com.enghub.config.DataSeeder : Database already contains data. Skipping seeding.
```

Hướng dẫn chạy dự án Frontend

Người dùng mở cửa sổ Command Prompt (CMD) hoặc Terminal tại thư mục gốc của dự án Frontend. Dự án Frontend được quản lý thông qua Node.js cùng trình quản lý gói npm. Do đó, việc khởi chạy ứng dụng được thực hiện bằng cách cài đặt các thư viện phụ thuộc và chạy lệnh khởi động tương ứng.

Cài thư viện

```
PS E:\enghub-fe> npm install

up to date, audited 214 packages in 5s

49 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
```

Chạy lệnh khởi động

```
PS E:\enghub-fe> npm run dev

> enghub-fe@0.0.0 dev
> vite

VITE v7.1.12 ready in 1279 ms

→ Local:   http://localhost:5173/
→ Network: use --host to expose
→ press h + enter to show help
```

Triển khai đưa ra số liệu thực nghiệm

Câu hỏi về ngữ pháp

Data đưa vào chroma

```

{
  "id": "u1_SA_9316",
  "text": "Unit 1: Present continuous (I am doing). Section A. Explanation: Thì hiện tại tiếp diễn diễn tả hành động đang diễn ra tại thời điểm nói.",
  "metadata": {
    "unit_number": "1",
    "unit_title": "Present continuous (I am doing)",
    "section": "A",
    "source": "Unit 1 - A"
  }
},
{
  "id": "u1_SB_5246",
  "text": "Unit 1: Present continuous (I am doing). Section B. Explanation: Diễn tả hành động đã bắt đầu nhưng chưa kết thúc, đang trong quá trình diễn ra.",
  "metadata": {
    "unit_number": "1",
    "unit_title": "Present continuous (I am doing)",
    "section": "B",
    "source": "Unit 1 - B"
  }
},
{
  "id": "u1_SC_9802",
  "text": "Unit 1: Present continuous (I am doing). Section C. Explanation: Dùng thì hiện tại tiếp diễn với các khoảng thời gian xung quanh thời điểm nói.",
  "metadata": {
    "unit_number": "1",
    "unit_title": "Present continuous (I am doing)",
    "section": "C",
    "source": "Unit 1 - C"
  }
}

```

Chuẩn bị test data

```

{
  "question": "Câu trúc ngữ pháp: S + could have + V_past_participle thuộc bài nào?",
  "ground_truth_context": "Unit 27: could (do) and could have (done). Section D. Explanation: Chúng ta dùng 'could have (done)' để nói về quá khứ.",
  "ground_truth_unit": "27",
  "ground_truth_section": "D"
},
{
  "question": "Công thức của Word order 1: verb + object; place and time là gì?",
  "ground_truth_context": "Unit 108: Word order 1: verb + object; place and time. Section B. Explanation: Động từ và nơi chốn (where?) thường đi trước.",
  "ground_truth_unit": "108",
  "ground_truth_section": "B"
},
{
  "question": "Câu trúc ngữ pháp: by + [thời điểm/ngày/sự kiện] thuộc bài nào?",
  "ground_truth_context": "Unit 119: by and until, by the time. Section A. Explanation: 'By' có nghĩa là 'không muộn hơn' một thời điểm cụ thể.",
  "ground_truth_unit": "119",
  "ground_truth_section": "A"
},
{
  "question": "Câu trúc ngữ pháp: N + number (without 'the') thuộc bài nào?",
  "ground_truth_context": "Unit 72: The. Section F. Explanation: Không dùng 'the' với số hiệu (size, platform, room, page, vitamin, section).",
  "ground_truth_unit": "72",
  "ground_truth_section": "F"
},
{
  "question": "Câu trúc ngữ pháp: get back to somebody thuộc bài nào?",
  "ground_truth_context": "Unit 144: Phrasal verbs 9 away/back. Section C. Explanation: get back to somebody = Phản hồi lại ai đó (qua điện thoại).",
  "ground_truth_unit": "144",
  "ground_truth_section": "C"
}

```

Sử dụng Google Colab để dùng GPU chạy llama

⌚ Đang tải model (khoảng 5GB)... Vui lòng đợi 1-2 phút...

```

/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens).
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
warnings.warn(

```

Meta-Llama-3-8B- 4.92G/4.92G [00:31<00:00, 241M/s]

Instruct.Q4_K_M.gguf: 100%

✅ Đã tải xong tại: /root/.cache/huggingface/hub/models--QuantFactory--Meta-Llama-3-8B-Instruct-GGUF

Triển khai function đánh giá

```
# =====
def benchmark_retrieval(engine, test_data, top_k=3):
    print(f"\n... Benchmark Retrieval...")
    methods = ["Similarity", "Sim_Rerank", "Hybrid", "Rerank"]
    stats = {m: {"Hit": 0, "MRR": 0, "Lat": []} for m in methods}

    for item in test_data:
        q = item['question']
        target = item['source_id']

        # Similarity
        res_sim, lat_sim = engine.search_similarity(q, top_k)
        ids_sim = [r['id'] for r in res_sim]
        stats["Similarity"]["Lat"].append(lat_sim)
        if target in ids_sim:
            stats["Similarity"]["Hit"] += 1
            stats["Similarity"]["MRR"] += 1 / (ids_sim.index(target) + 1)

        # simi + rerank
        res_sim_rnk, lat_sim_rnk = engine.search_similarity_reranking(q, top_k)
        ids_sim_rnk = [r['id'] for r in res_sim_rnk]
        stats["Sim_Rerank"]["Lat"].append(lat_sim_rnk)
        if target in ids_sim_rnk:
            stats["Sim_Rerank"]["Hit"] += 1
            stats["Sim_Rerank"]["MRR"] += 1 / (ids_sim_rnk.index(target) + 1)
```

Đánh giá dựa trên context tìm được có đúng unit và section cần tìm không

	Method	Hit rate(%)	MRR	Latency(ms)
	Similarity	55	0.4500	23.19
	Similarity + Reranking	60	0.5417	891.63
	Hybrid	45	0.3583	24.02
	Hybrid + Reranking	65	0.6000	840.84

Đánh giá câu trả lời đưa Gemini chấm điểm

	Method	Faithfulness	Answer Relevance	Context Precision
	Similarity	0.85	0.87	0.53
	Similarity + Reranking	0.90	0.92	0.62
	Hybrid	0.80	0.78	0.43
	Hybrid + Reranking	0.92	0.95	0.70

Câu hỏi về từ vựng

Sử dụng các bài đọc ngắn

Topic: Culture

Culture reflects the values, traditions, and beliefs of a society. Music, art, and literature play important roles in cultural identity. Global communication has led to cultural exchange, allowing people to learn from different traditions. However, some fear that globalization causes cultural homogenization. Preserving cultural heritage is essential while embracing diversity.

Các đoạn text mở rộng từ vựng

The term 'allow' is a significant concept in the field of General / Governance. It is defined as: To give permission for something to happen or to be done.

The term 'society' is a significant concept in the field of Sociology. It is defined as: The aggregate of people living together in a more or less organized community.

The term 'art' is a significant concept in the field of culture / Arts. It is defined as: The expression or application of human creative skill and imagination, typically as an aesthetic form, through visual representation as in painting, sculpture, printmaking, etc., or through music and drama.

The term 'homogenization' is a significant concept in the field of Sociology / Globalization. It is defined as: The process of making things or people alike, especially by standardizing or imposing a common culture or language.

The term 'value' is a significant concept in the field of Ethics / Economics. It is defined as: The importance, worth, or usefulness of something; a principle or standard.

The term 'different' is a significant concept in the field of General. It is defined as: Not the same as another or each other; unlike in nature or quality.

The term 'music' is a significant concept in the field of culture / Arts. It is defined as: Vocal or instrumental sounds (or both) combined in such a way as to produce a composition that is intended to be heard.

The term 'identity' is a significant concept in the field of Psychology / Sociology. It is defined as: The fact of being who or what a person or thing is.

The term 'topic' is a significant concept in the field of Communication. It is defined as: A matter dealt with in a text, discourse, or conversation.

The term 'learn' is a significant concept in the field of Education. It is defined as: Gain or acquire knowledge of or skill in something by study, experience, or instruction.

The term 'important' is a significant concept in the field of General. It is defined as: Of great significance or value; likely to have a profound effect.

The term 'essential' is a significant concept in the field of General. It is defined as: Absolutely necessary; extremely important.. Contextualized.

The term 'play' is a significant concept in the field of Leisure / Arts. It is defined as: Engage in activity for enjoyment and recreation rather than a serious or business purpose.

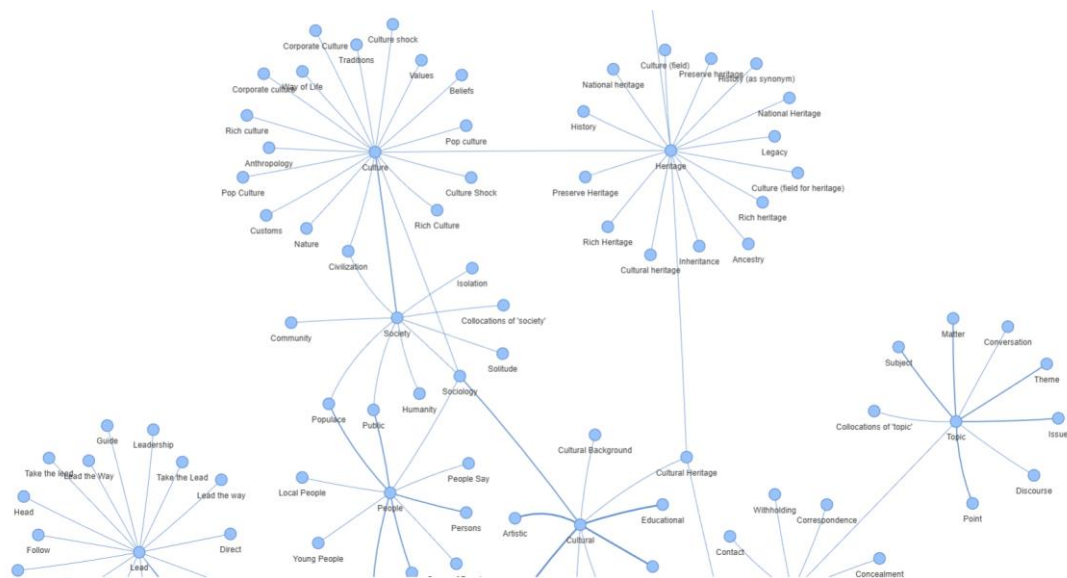
Do LightRAG xây dựng bằng cách gửi đoạn văn cho LLM tách ra entity và relationship nên gọi rất nhiều lần tới LLM, do điều kiện không cho phép nên chỉ có thể test trên tập data nhỏ

```
# -----
# Embedding function
# -----
embedding_func = EmbeddingFunc(
    embedding_dim=768,
    max_token_size=2048,
    func=partial(
        gemini_embed.func,
        api_key=GEMINI_API_KEY,
        model="models/text-embedding-004"
    )
)

# -----
# Initialize RAG
# -----
async def initialize_rag():
    rag = LightRAG(
        working_dir=WORKING_DIR,
        llm_model_func=llm_model_func,
        embedding_func=embedding_func,
        llm_model_name="gemini-2.5-flash",
    )

    await rag.initialize_storages()
    return rag
```

Mối liên hệ giữa các từ sau khi xây dựng bằng LightRAG



Data test

```
{
  "id": 3,
  "question": "Which word is an antonym for 'diversification' and refers to the process of making things uniform?"
},
{
  "id": 4,
  "question": "What term describes 'the expression or application of human creative skill and imagination'?"
},
{
  "id": 5,
  "question": "Which concept represents 'the importance, worth, or usefulness of something' in the field of Ethics?"
},
{
  "id": 6,
  "question": "What word is commonly associated with collocations like 'cultural identity' and 'identity crisis'?"
},
{
  "id": 7,
  "question": "Which term means 'relating to the whole world' and serves as an antonym to 'local'?"
},
{
  "id": 8,
  "question": "What word implies 'thinking deeply or carefully about something' as well as throwing back light or sound?"
},
{

```

Chỉ đơn giản bằng cách cho 1 list câu hỏi rồi đưa Gemini chấm điểm

Method	Comprehensiveness	Diversity
Similarity	0.42	0.35
Similarity + Reranking	0.51	0.38
Hybrid	0.48	0.45
Hybrid + Reranking	0.58	0.48
LightRAG	0.87	0.84

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

Trong khuôn khổ đồ án tốt nghiệp này, nhóm đã tập trung nghiên cứu và xây dựng thành công ứng dụng học tiếng Anh tích hợp trí tuệ nhân tạo, tập trung vào ba trụ cột chính: phân tích ngữ pháp, gợi ý từ vựng và chatbot luyện phản xạ. Dựa trên các kết quả thu được, nhóm rút ra các kết luận sau:

- **Về mặt nghiên cứu:** Nhóm đã làm chủ được quy trình xử lý ngôn ngữ tự nhiên hiện đại, đặc biệt là việc nghiên cứu và ứng dụng kiến trúc LightRAG (Retrieval-Augmented Generation). Qua thực nghiệm, nhóm đã chứng minh được LightRAG có khả năng bảo toàn mạch liên kết tri thức thông qua đồ thị (Knowledge Graph), khắc phục triệt để hiện tượng đứt gãy thông tin mà mô hình Naive RAG thường gặp phải.

- **Về mặt thực tiễn:** Hệ thống đã được hiện thực hóa với đầy đủ các tính năng hỗ trợ người dùng: từ việc sửa lỗi ngữ pháp có giải thích chi tiết, đến việc cung cấp môi trường hội thoại thông minh giúp người dùng cải thiện tư duy phản xạ bằng tiếng Anh. Kết quả thực nghiệm tại Chương 3 cho thấy hệ thống đáp ứng tốt các yêu cầu về độ chính xác và tính thực dụng trong giáo dục.

- **Hạn chế của đồ án:** Mặc dù đạt được những kết quả khả quan, hệ thống vẫn tồn tại một số hạn chế nhất định như: độ trễ trong việc truy vấn các thực thể phức tạp trên đồ thị tri thức còn cao; giao diện người dùng cần được tối ưu hóa hơn cho trải nghiệm đa nền tảng và chưa tích hợp sâu các tính năng tương tác bằng giọng nói.

Để hoàn thiện hệ thống và mở rộng khả năng ứng dụng trong thực tế, nhóm đề xuất các hướng nghiên cứu và phát triển tiếp theo như sau:

- **Tích hợp xử lý đa phương tiện (Multimodal):** Nâng cấp hệ thống bằng cách tích hợp các mô hình nhận dạng giọng nói (STT) và tổng hợp giọng nói (TTS) chất lượng cao. Điều này sẽ cho phép người dùng thực hành kỹ năng Nghe - Nói trực tiếp với chatbot, tạo ra môi trường giao tiếp toàn diện hơn.

- **Cá nhân hóa lộ trình học tập (Adaptive Learning):** Ứng dụng các thuật toán học máy để phân tích hành vi và trình độ của từng cá nhân. Từ đó, hệ thống có thể tự động điều chỉnh độ khó của bài tập và gợi ý nội dung học tập phù hợp nhất với điểm yếu của người dùng.

- **Tối ưu hóa hiệu năng và quy mô tri thức:** Tiếp tục nghiên cứu các kỹ thuật tối

ưu hóa đồ thị tri thức để giảm thiểu thời gian phản hồi (latency). Đồng thời, mở rộng kho dữ liệu huấn luyện để hệ thống có thể hỗ trợ nhiều chuyên ngành tiếng Anh đặc thù hơn như Y khoa, Công nghệ thông tin hay Kinh tế.

- Phát triển cộng đồng học tập: Xây dựng các tính năng tương tác giữa người dùng (Gamification), bảng xếp hạng và thi đấu trực tuyến để tăng tính kết nối và tạo động lực học tập bền vững cho cộng đồng.

TÀI LIỆU THAM KHẢO

- [1] Z. Guo, L. Xia, Y. Yu, T. Ao, and C. Huang, “LightRAG: Simple and Fast Retrieval-Augmented Generation,” *arXiv preprint arXiv:2410.05779*, 2024.
[Online]. Available: <https://arxiv.org/abs/2410.05779>
- [2]<https://www.geeksforgeeks.org/system-design/client-server-architecture-system-design/>
- [3]<https://research.library.kutztown.edu/cisfaculty/10/>

Bảng phân chia công việc

Công việc	Thành viên	Mô tả nhiệm vụ chính
Phân tích và thiết kế hệ thống	Trọng + Tú + Hiếu	Phân tích yêu cầu, xây dựng đặc tả use case, thiết kế kiến trúc tổng thể, sơ đồ lớp, sơ đồ trình tự và mô hình dữ liệu
Module đăng nhập, đăng ký	Tú	Thiết kế và phát triển chức năng xác thực người dùng, đăng ký tài khoản, đăng nhập, quản lý phiên làm việc và bảo mật
Module học và luyện tập	Tú + Hiếu	Xây dựng luồng học tập, luyện tập theo bài học, xử lý logic học tập và lưu trữ tiến trình người dùng
Nghiên cứu và phát triển AI	Tú	Nghiên cứu, tích hợp các chức năng AI hỗ trợ học tập (gợi ý từ vựng, phản hồi thông minh)
Module học Flashcard	Hiếu + Tú	Thiết kế và phát triển chức năng quản lý flashcard, deck flashcard, cơ chế ôn tập và ghi nhớ từ vựng
Thiết kế và phát triển giao diện người dùng	Trọng	Thiết kế bố cục giao diện, trải nghiệm người dùng, xây dựng giao diện web
Module thông báo realtime	Trọng	Xây dựng hệ thống thông báo (notification), nhắc nhở học tập, cập nhật trạng thái và sự kiện trong hệ thống
Module học các kỹ năng	Hiếu	Phát triển các chức năng học theo kỹ năng (Listening, Reading, Writing, Speaking), quản lý nội dung và kết quả học
Module profile, dashboard, theo dõi phiên học realtime	Trọng	Xây dựng profile, dashboard tổng quan, hiển thị thống kê học tập, theo dõi phiên học realtime và trực quan hóa dữ liệu